

OmniLitter

Sistema multiplataforma para la gestión y análisis de datos de
contaminación por residuos



Escuela Técnica Superior de
Ingeniería Informática

Trabajo Fin de Grado

Grado: Ingeniería Informática - Ingeniería del Software

Centro: ETSII — Universidad de Sevilla

Curso académico: 2025/26

Autores:

- Alejandro Castilla — alecasbar@alum.us.es
- Ignacio Martínez Díaz — ignmardial@alum.us.es

Tutor académico: Cruz Mata, Fermín

Cliente / entidad colaboradora: Daniel González Fernández /
Universidad de Cádiz

Resumen

Abstract

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Problema abordado	2
1.3. Objetivos	3
1.4. Alcance del proyecto	3
2. Contexto y antecedentes	5
2.1. Monitorización de residuos	5
2.2. Aplicaciones de campo y ciencia ciudadana	5
2.3. Herramientas existentes	6
2.4. Taxonomías y datos científicos	6
2.5. Conclusiones del análisis	7
3. Descripción del proyecto	8
3.1. Visión general de OmniLitter	8
3.2. Actores y perfiles de usuario	8
3.2.1. Visitante	9
3.2.2. Usuario autenticado	9
3.2.3. Investigador de campo	9
3.2.4. Administrador de grupo	10
3.2.5. Superadministrador	10
3.2.6. Usuario sin grupo activo	10
3.2.7. Actores de apoyo	11
3.3. Plataformas del sistema	11
3.3.1. Aplicación móvil	11
3.3.2. Portal web	12
3.3.3. Backend y almacenamiento centralizado	12
3.3.4. Integración de las plataformas	12
3.4. Funcionalidades principales	13
3.4.1. Gestión de usuarios y grupos	13
3.4.2. Gestión de campañas	13
3.4.3. Sesiones de muestreo	13

3.4.4.	Registro de observaciones	13
3.4.5.	Clasificación mediante taxonomías	14
3.4.6.	Registro fotográfico y geolocalización	14
3.4.7.	Trabajo offline y sincronización	14
3.4.8.	Consulta y visualización de información	14
3.4.9.	Exportación de datos	14
3.4.10.	Resumen funcional	15
3.5.	Restricciones y alcance excluido	15
4.	Requisitos y análisis	17
4.1.	Evolución del registro de requisitos	17
4.1.1.	Versión 1: toma inicial de requisitos	17
4.1.2.	Versión 2: validación mediante prototipos	19
4.1.3.	Versión 3: consolidación tras la última reunión y revisión de dominio	20
4.2.	Casos de uso de referencia	21
4.3.	Requisitos finales consolidados	22
4.3.1.	CU-01 – Registrarse, autenticarse y gestionar la sesión	23
4.3.2.	CU-02 – Gestionar el perfil y el grupo activo	24
4.3.3.	CU-03 – Administrar grupos, miembros e invitaciones	24
4.3.4.	CU-04 – Administrar usuarios de la plataforma	25
4.3.5.	CU-05 – Crear y gestionar campañas de muestreo	25
4.3.6.	CU-06 – Crear y actualizar una sesión de muestreo	26
4.3.7.	CU-07 – Registrar observaciones de residuos	27
4.3.8.	CU-08 – Revisar y finalizar una sesión de muestreo	28
4.3.9.	CU-09 – Trabajar sin conexión y sincronizar datos	28
4.3.10.	CU-10 – Consultar sesiones y observaciones	29
4.3.11.	CU-11 – Visualizar la información en mapa	30
4.3.12.	CU-12 – Consultar métricas y analítica	30
4.3.13.	CU-13 – Exportar datos	31
4.3.14.	CU-14 – Configurar preferencias de la aplicación	31
4.4.	Requisitos no funcionales consolidados	31
4.5.	Requisitos de datos científicos consolidados	33
4.6.	Trazabilidad final	34
5.	Planificación y gestión	36
5.1.	Metodología de trabajo	36
5.1.1.	Adaptación de Scrum	36
5.2.	Planificación Inicial	37
5.2.1.	Identificación de bloques funcionales	38
5.2.2.	Dependencias entre módulos	38
5.2.3.	División inicial en sprints	39
5.2.4.	Riesgos identificados	40
5.3.	Gestión y herramientas de seguimiento	41

5.3.1.	Gestión de la comunicación	41
5.3.2.	Gestión del backlog	43
5.3.3.	Gestión documental	44
5.3.4.	Gestión del código fuente	46
5.3.5.	Gestión de versiones y releases	47
5.3.6.	Gestión de incidencias	48
6.	Tecnologías utilizadas	50
6.1.	Criterios de selección	50
6.2.	Portal web	51
6.3.	Aplicación móvil	52
6.4.	Backend y persistencia	52
6.5.	Despliegue y herramientas auxiliares	53
6.6.	Alternativas consideradas	54
7.	Costes	55
7.1.	Costes de personal	55
7.2.	Costes de infraestructura y despliegue	56
7.3.	Costes de herramientas software	57
7.4.	Costes de equipamiento	58
7.5.	Plan de contingencia	59
7.6.	Costes globales del proyecto	60
8.	Diseño de la solución	62
8.1.	Arquitectura general	62
8.1.1.	Justificación de las decisiones arquitectónicas principales	63
8.2.	Diseño de la base de datos	63
8.2.1.	Modelo relacional	63
8.2.2.	Descripción de entidades	67
8.2.3.	Decisiones de diseño de la base de datos	69
8.3.	Diseño de la API REST	71
8.4.	Seguridad y control de acceso	73
8.5.	Diseño del portal web	74
8.5.1.	Arquitectura de componentes	74
8.5.2.	Gestión de estado	74
8.5.3.	Flujo de navegación y control de acceso por rol	74
8.6.	Diseño de la aplicación móvil	75
8.6.1.	Stack tecnológico y estructura de pantallas	75
8.6.2.	Gestión de estado global	75
8.6.3.	Base de datos local con SQLite	75
8.6.4.	Flujo de captura de una observación	76
8.7.	Sincronización offline	76
8.7.1.	Patrón Outbox Queue	76

8.7.2.	Estados de sincronización	77
8.7.3.	Resolución de conflictos	77
8.7.4.	Garantía de identificadores únicos en entorno offline	77
9.	Desarrollo, implantación y validación	78
9.1.	Sprint 1: Base funcional del sistema	79
9.1.1.	Características y objetivos del sprint	79
9.1.2.	Implementación realizada	80
9.1.3.	Pruebas realizadas	81
9.1.4.	Desviaciones e incidencias	82
9.1.5.	Resultado del sprint	83
9.2.	Sprint 2: Captura de campo y análisis inicial	85
9.2.1.	Características y objetivos del sprint	85
9.2.2.	Implementación realizada	86
9.2.3.	Pruebas realizadas	88
9.2.4.	Desviaciones e incidencias	89
9.2.5.	Resultado del sprint	90
9.3.	Sprint 3: Sincronización, mapas y exportación científica	92
9.3.1.	Características y objetivos del sprint	92
9.3.2.	Implementación realizada	93
9.3.3.	Pruebas realizadas	95
9.3.4.	Desviaciones e incidencias	96
9.3.5.	Resultado del sprint	96
9.4.	Sprint 4: Cierre funcional y refinamiento	98
9.4.1.	Características y objetivos del sprint	98
9.4.2.	Implementación realizada	99
9.4.3.	Pruebas realizadas	101
9.4.4.	Desviaciones e incidencias	101
9.4.5.	Resultado del sprint	102
9.5.	Pruebas globales y validación funcional	104
9.5.1.	Pruebas manuales funcionales	104
9.5.2.	Matriz de trazabilidad	117
10.	Despliegue y operación	119
10.1.	Estrategia de despliegue por entornos	119
10.2.	Entorno local de desarrollo	119
10.2.1.	Orquestación con Docker Compose	120
10.2.2.	Ejecución sin contenedores	120
10.2.3.	Aplicación móvil y emulador	120
10.3.	Preproducción en Render	121
10.4.	Producción en Azure	121
10.5.	Integración y entrega continua	122
10.6.	Configuración y gestión de secretos	122

10.7. Limitaciones operativas	123
11. Conclusiones y trabajo futuro	124
11.1. Cumplimiento de objetivos	124
11.2. Limitaciones	124
11.3. Lecciones aprendidas	125
11.4. Líneas de trabajo futuro	125
Bibliografía	127
A. Declaración del uso de IA	129
A.1. Usos realizados	129
A.2. Ejemplos de prompts utilizados	130
A.3. Autoría y revisión crítica	131
B. Manual de instalación y despliegue	132
B.1. Requisitos previos	132
B.2. Puesta en marcha	132
B.2.1. Clonar el repositorio e instalar dependencias	132
B.2.2. Opción A: arranque con Docker	133
B.2.3. Opción B: desarrollo sin Docker	133
B.3. Aplicación móvil	134
B.4. Scripts útiles	135
B.5. Solución de problemas frecuentes	135
C. Manual de usuario	136

Índice de figuras

5.1. Uso de Microsoft Teams para reuniones de seguimiento y validación.	42
5.2. Uso de Outlook para comunicaciones formales y trazabilidad documental. .	42
5.3. Tablero Kanban utilizado para visualizar el flujo de trabajo del backlog. . .	43
5.4. Plantilla de tarea utilizada para homogeneizar el backlog.	44
5.5. Uso inicial de Overleaf para la redacción colaborativa de la memoria. . . .	45
5.6. Repositorio de documentación utilizado como fuente versionada de la me- moria.	45
5.7. Repositorio documental auxiliar en OneDrive.	46
5.8. Repositorio centralizado utilizado para la gestión del código fuente.	46
5.9. Releases publicadas mediante GitHub.	47
5.10. Plantilla y seguimiento de incidencias en GitHub.	49
8.1. Arquitectura general propuesta para OmniLitter.	63
8.2. Modelo entidad-relación. Bloque de usuarios, grupos, campañas y catálogo de entornos.	65
8.3. Modelo entidad-relación. Bloque de sesiones de muestreo, observaciones, residuos, fotografías y taxonomía.	66
8.4. Modelo entidad-relación. Bloque de autenticación y seguridad.	66

Índice de cuadros

4.1.	Requisitos identificados en la versión 1.	18
4.2.	Requisitos definidos en la versión 2 tras la validación con prototipos.	19
4.3.	Requisitos consolidados o refinados en la versión 3.	21
5.1.	Dependencias principales consideradas en la planificación inicial	39
5.2.	División inicial del trabajo por sprints	40
5.3.	Riesgos principales identificados en la planificación inicial	41
5.4.	Versiones principales publicadas durante el desarrollo	48
6.1.	Tecnologías empleadas en los prototipos funcionales.	51
6.2.	Stack seleccionado para el portal web.	51
6.3.	Stack seleccionado para la aplicación móvil.	52
6.4.	Stack seleccionado para backend y persistencia.	53
6.5.	Herramientas auxiliares, pruebas y despliegue.	54
7.1.	Estimación de costes de personal por actividad	56
7.2.	Estimación de costes de infraestructura	57
7.3.	Estimación de costes de herramientas software	58
7.4.	Estimación de costes de equipamiento amortizado	59
7.5.	Plan de contingencia del proyecto	60
7.6.	Resumen global de costes del proyecto	60
8.1.	Descripción de entidades del modelo de datos.	67
8.2.	Principales <i>endpoints</i> de la API REST de OmniLitter.	71
8.3.	Estados de sincronización y su significado.	77
9.1.	Trazabilidad técnica de la implementación del Sprint 1	81
9.2.	Pruebas automatizadas vinculadas al Sprint 1	82
9.3.	Desviaciones e incidencias del Sprint 1	83
9.4.	Resumen de horas reales del Sprint 1.	84
9.5.	Trazabilidad técnica de la implementación del Sprint 2	88
9.6.	Pruebas automatizadas vinculadas al Sprint 2	89
9.7.	Desviaciones e incidencias del Sprint 2	90
9.8.	Resumen de horas reales del Sprint 2.	91

9.9. Trazabilidad técnica de la implementación del Sprint 3	95
9.10. Pruebas automatizadas vinculadas al Sprint 3	95
9.11. Desviaciones e incidencias del Sprint 3	96
9.12. Resumen de horas reales del Sprint 3.	97
9.13. Trazabilidad técnica de la implementación del Sprint 4	100
9.14. Pruebas automatizadas vinculadas al Sprint 4	101
9.15. Desviaciones e incidencias del Sprint 4	102
9.16. Resumen de horas reales del Sprint 4.	103
9.17. Pruebas manuales funcionales agrupadas por casos de uso	104
9.18. Matriz de trazabilidad entre casos de uso y pruebas manuales	117
10.1. Entornos de despliegue de OmniLitter.	119
10.2. Flujos de integración y entrega continua.	122
A.1. Detalle del uso de IA en el trabajo.	130
B.1. Requisitos previos de instalación.	132
B.2. Servicios disponibles con Docker.	133

1. Introducción

1.1. Contexto y motivación

La contaminación por residuos en entornos terrestres y acuáticos constituye uno de los principales problemas ambientales actuales. La presencia de basura en ríos, playas, zonas costeras, bosques, áreas urbanas y otros espacios naturales afecta a los ecosistemas, a la biodiversidad y a la calidad de vida de las personas. Para comprender la magnitud del problema y evaluar la eficacia de las medidas de prevención o mitigación, las administraciones públicas, las instituciones científicas y las organizaciones ambientales impulsan programas de monitorización de residuos en distintos compartimentos ambientales.

La utilidad de estos programas depende de que los datos recogidos sean comparables, estén correctamente georreferenciados y conserven información suficiente sobre cómo, cuándo y dónde se han obtenido. En el caso de las basuras fluviales, el Joint Research Centre de la Comisión Europea señala la falta de información sistemática sobre la cantidad de residuos transportados por los ríos y la ausencia de metodologías armonizadas capaces de generar datos cuantitativos comparables [9]. De forma similar, trabajos recientes subrayan la necesidad de armonizar las técnicas de monitorización de macroplásticos en ecosistemas acuáticos y terrestres [8].

Un antecedente directo de este proyecto es el trabajo realizado para la monitorización de macrobasuras flotantes en ríos, donde se desarrolló una aplicación para dispositivos tipo *tablet* capaz de registrar los ítems observados, su tamaño y su posición geográfica durante sesiones de monitorización [10]. Este antecedente refuerza la idea de que la herramienta de captura no debe limitarse a digitalizar un formulario, sino que debe orientar la observación y conservar desde el inicio los datos necesarios para que los registros puedan compararse posteriormente.

En este contexto, el cliente planteó la necesidad de renovar y ampliar la aplicación *Floating Litter Monitoring*. La herramienta original estaba orientada principalmente a la monitorización de residuos en ríos y mares, mientras que el nuevo proyecto debía extender esa capacidad a una mayor variedad de entornos naturales y urbanos, incorporar una gestión más flexible de usuarios y grupos, mejorar la recogida de evidencias y permitir la consulta posterior de los datos desde un portal web.

Como respuesta a esta necesidad se propone el desarrollo de OmniLitter, una plataforma formada por una aplicación móvil, un portal web y un backend centralizado. Su finalidad es facilitar la recogida, gestión y análisis de datos de residuos mediante una solución que combine trabajo de campo, sincronización de información, control de permisos y explotación posterior de los datos recopilados.

1.2. Problema abordado

La monitorización de residuos requiere la participación de distintas personas y grupos que realizan campañas en lugares y momentos diferentes. Si la información se registra mediante formularios no normalizados, hojas de cálculo aisladas o campañas de limpieza sin un protocolo común, los datos resultantes suelen quedar dispersos, son difíciles de comparar y pierden parte de su valor para análisis científicos posteriores.

Además, la captura de datos se realiza habitualmente en la naturaleza, donde la conectividad a Internet puede ser limitada o inexistente. Esta situación dificulta el uso de soluciones que dependen de una conexión permanente con servidores remotos. Por ello, resulta necesario que la aplicación móvil pueda almacenar sesiones, observaciones, fotografías y metadatos en el propio dispositivo, y que sincronice esa información con el servidor cuando vuelva a existir conexión.

El proyecto también exige registrar datos con suficiente calidad científica. Cada observación debe asociarse a una taxonomía de residuos, una localización geográfica, una fecha y hora, evidencias fotográficas y metadatos de muestreo como el entorno, la tipología, el método empleado, la duración, el transecto o el área observada. La precisión de la posición GPS y el estado de sincronización de cada registro también son relevantes para interpretar correctamente los datos recogidos.

A estas necesidades se suma la organización colaborativa del trabajo. Los usuarios pertenecen a grupos, los grupos gestionan campañas, las campañas contienen sesiones de muestreo y las sesiones agrupan observaciones. Esta estructura requiere mecanismos de autenticación, control de permisos y aislamiento de datos para que cada equipo pueda gestionar su información de forma segura y privada.

Por tanto, el problema abordado no consiste únicamente en crear un formulario digital de recogida de residuos. El reto es diseñar una plataforma que conecte la captura offline en campo con una infraestructura común de almacenamiento, administración, visualización y exportación, manteniendo la trazabilidad y la comparabilidad de los datos desde su origen.

1.3. Objetivos

El objetivo principal de este Trabajo Fin de Grado es diseñar, desarrollar e implementar OmniLitter, una plataforma software multiplataforma que facilite la monitorización de residuos en distintos entornos naturales y permita transformar observaciones de campo en datos estructurados, comparables y reutilizables.

Para alcanzar este objetivo general se establecen los siguientes objetivos específicos:

1. Diseñar una arquitectura software que integre una aplicación móvil, un portal web, un backend centralizado y una base de datos común.
2. Organizar la información del sistema mediante grupos de trabajo, campañas, sesiones de muestreo y observaciones geolocalizadas.
3. Desarrollar una aplicación móvil destinada a la captura de datos en campo, incluyendo residuos observados, fotografías, ubicación, fecha, hora y metadatos de muestreo.
4. Permitir el funcionamiento de la aplicación móvil en condiciones de conectividad limitada mediante persistencia local y sincronización diferida con el servidor.
5. Incorporar taxonomías estandarizadas, entornos, tipologías y métodos de muestreo configurables para favorecer la calidad científica de la información registrada.
6. Implementar mecanismos de autenticación, roles, permisos y aislamiento de datos entre grupos de trabajo.
7. Desarrollar un portal web que facilite la administración de usuarios, grupos y campañas, así como la consulta de sesiones y observaciones recopiladas.
8. Proporcionar herramientas de visualización y explotación de datos, incluyendo mapas, paneles informativos y exportación en formatos adecuados para análisis posterior.
9. Garantizar la integridad, trazabilidad y consistencia de la información almacenada mediante un backend centralizado y una API autenticada.
10. Aplicar metodologías y herramientas de ingeniería del software para documentar, probar y validar la solución desarrollada dentro del alcance académico del proyecto.

1.4. Alcance del proyecto

El presente Trabajo Fin de Grado aborda el diseño, desarrollo e implementación de una versión funcional de una plataforma software denominada OmniLitter, orientada a facilitar la monitorización y el análisis de la contaminación por residuos en distintos ecosistemas terrestres, acuáticos y urbanos, dentro del alcance académico del proyecto.

La solución desarrollada está compuesta por una aplicación móvil destinada a la recogida de información en campo, un portal web para la gestión y explotación de los datos recopilados, y un backend centralizado encargado de la autenticación, la lógica de negocio, la sincronización y la persistencia de la información.

Dentro del alcance del proyecto se incluyen las siguientes funcionalidades:

- Diseño e implementación de la arquitectura completa del sistema, incluyendo aplicación móvil, portal web, backend y base de datos.
- Registro de observaciones de residuos utilizando una taxonomía jerárquica estandarizada que garantice la consistencia científica de los datos.
- Captura automática de información geográfica y temporal asociada a cada observación mediante los sensores disponibles en los dispositivos móviles.
- Gestión de fotografías y metadatos asociados a las observaciones registradas.
- Gestión de campañas y sesiones de muestreo, incluyendo información contextual como el entorno analizado, el método empleado, la duración de la sesión, el tipo de transecto o cuadrante y las características del área estudiada.
- Funcionamiento en modo offline mediante almacenamiento local de la información y sincronización posterior con el servidor cuando exista conectividad.
- Administración de campañas, usuarios, grupos y permisos mediante un portal web de gestión.
- Visualización de observaciones, sesiones y campañas mediante mapas interactivos y paneles informativos.
- Exportación de los datos recopilados en formatos adecuados para su posterior explotación y análisis científico.
- Implementación de mecanismos de autenticación, control de acceso y protección de la información almacenada.
- Elaboración de la documentación técnica, manuales, pruebas y memoria académica asociada al desarrollo del sistema.

Quedan fuera del alcance del Trabajo Fin de Grado el mantenimiento post-producción, el soporte técnico continuo tras la entrega y la definición de nuevos protocolos científicos oficiales. De esta forma, el alcance del proyecto se centra en proporcionar una plataforma funcional para la recogida, gestión, consulta y explotación de datos relacionados con la monitorización de residuos, estableciendo una base tecnológica que permita futuras ampliaciones y desarrollos.

2. Contexto y antecedentes

2.1. Monitorización de residuos

La monitorización de residuos requiere transformar observaciones realizadas en campo en datos comparables, trazables y útiles para el análisis científico. En el caso de las basuras fluviales, la literatura técnica europea señala una carencia relevante: aunque existen diferentes aproximaciones de muestreo, todavía faltan datos sistemáticos y metodologías armonizadas que permitan obtener información cuantitativa comparable entre ríos, periodos temporales y áreas de estudio [9]. Esta limitación dificulta conocer la distribución espacial y temporal de los residuos, identificar fuentes terrestres y evaluar la eficacia de medidas de prevención o mitigación.

El problema no se limita al ámbito fluvial. Los residuos plásticos y otros tipos de basura aparecen en distintos compartimentos ambientales, tanto acuáticos como terrestres. Estudios recientes sobre macroplásticos destacan la necesidad de seleccionar técnicas de observación fiables y coste-efectivas, como censos visuales o muestreos apoyados por drones, y de avanzar hacia protocolos armonizados de monitorización [8]. Esta necesidad es especialmente relevante en entornos terrestres como bosques o zonas urbanas, donde la disponibilidad de datos científicos estructurados y metodologías consolidadas es menor que en playas o riberas.

En este contexto, las acciones de limpieza ciudadana son valiosas como actividad de sensibilización y retirada de residuos, pero sus registros no siempre alcanzan el nivel de detalle, control metodológico y trazabilidad que exige un estudio científico. Para que los datos puedan utilizarse en análisis temporales, comparaciones espaciales o evaluación de políticas públicas, es necesario documentar el método de muestreo, el área cubierta, la frecuencia, la localización y los metadatos asociados a la recogida de datos [9].

2.2. Aplicaciones de campo y ciencia ciudadana

El uso de aplicaciones digitales permite reducir parte de la brecha entre trabajo de campo, monitorización científica y gestión ambiental. Un precedente directo es el enfoque desarrollado para la observación de macrobasuras flotantes en ríos, en el que una aplicación

de tableta permitió registrar ítems observados, tamaño y geoposición durante sesiones de monitorización, apoyándose en una lista común de elementos y rangos de tamaño para armonizar la recogida y el reporte de datos [10].

Este tipo de herramientas resulta especialmente útil cuando se pretende ampliar la cobertura espacial y temporal de la monitorización. La participación de ONGs, comunidades locales y colaboradores de campo puede multiplicar la capacidad de observación respecto a un equipo científico reducido, siempre que la aplicación guíe la captura de datos mediante protocolos claros, validaciones de entrada, geolocalización, fotografías y metadatos obligatorios. De este modo, la ciencia ciudadana puede generar datos estructurados y reutilizables, no solo registros aislados de limpieza.

2.3. Herramientas existentes

Las iniciativas existentes para el seguimiento de residuos suelen concentrarse en entornos concretos, como playas, superficie del agua, riberas o campañas de limpieza. Las guías y propuestas recientes de OSPAR para basuras en riberas ponen el foco en producir datos y metadatos comparables, detectar diferencias espaciales y cambios temporales, y evaluar la eficacia de acciones regionales de reducción de basuras [12]. También proponen aspectos operativos como la definición de unidades de muestreo, criterios de selección de sitios, estrategias de muestreo, gestión de datos y control de calidad.

OmniLitter toma estas necesidades como referencia de diseño, pero su objetivo no es sustituir un protocolo científico oficial concreto. La plataforma proporciona una base digital flexible para registrar campañas, sesiones y observaciones en playas, riberas, bosques y zonas urbanas, de forma que el equipo científico pueda aplicar el protocolo adecuado en cada caso y conservar los datos mínimos necesarios para su interpretación posterior.

2.4. Taxonomías y datos científicos

La clasificación homogénea de residuos es un requisito esencial para comparar resultados entre campañas. El informe técnico del JRC sobre monitorización de basuras fluviales indica que la identificación de macrobasuras debe apoyarse en listas comunes de categorías y destaca la utilidad de la *MSFD Master List of Categories of Litter Items* como base para la comparabilidad entre estudios en ríos y mares adyacentes [9]. Esta lista organiza los ítems por categorías materiales y proporciona un marco común para relacionar objetos observados con posibles usos o fuentes.

Además de la taxonomía, la calidad científica de los datos depende de los metadatos. La monitorización debe registrar información contextual como método de muestreo, compartimento ambiental, tamaño o tipo de ítem, frecuencia, ubicación, condiciones ambientales

y unidades de reporte [9]. En OmniLitter, esta necesidad se traduce en sesiones de muestreo con metadatos asociados, observaciones georreferenciadas, versionado de taxonomía y exportación de datos en formatos reutilizables.

2.5. Conclusiones del análisis

El análisis del dominio confirma que OmniLitter debe resolver algo más que el registro operativo de residuos. La plataforma debe ayudar a convertir observaciones dispersas en datos armonizados, comparables y georreferenciados, capaces de apoyar estudios científicos y decisiones de gestión. Por ello, los requisitos del sistema se orientan a tres objetivos complementarios: facilitar la recogida de datos en campo, preservar la validez científica mediante protocolos, taxonomías y metadatos, y permitir la explotación posterior de la información para detectar tendencias, identificar fuentes y evaluar medidas de mitigación.

3. Descripción del proyecto

3.1. Visión general de OmniLitter

OmniLitter es una plataforma multiplataforma desarrollada con el objetivo de facilitar la monitorización y el análisis de residuos en distintos tipos de entornos: playas, riberas, bosques y zonas urbanas. El sistema está orientado tanto a investigadores como a colaboradores de campo, proporcionando herramientas que permiten registrar, almacenar y analizar información relacionada con campañas de muestreo de residuos.

La finalidad de la plataforma no es únicamente contabilizar residuos, sino digitalizar y unificar la toma de datos para que las observaciones puedan convertirse en información científica comparable. Para ello, OmniLitter organiza el trabajo en campañas y sesiones de muestreo, registra metadatos de campo, utiliza categorías estandarizadas de residuos y conserva la localización y evidencias asociadas a cada observación.

La arquitectura de OmniLitter se compone de tres elementos principales estrechamente integrados: una API central, una aplicación móvil y un portal web. La aplicación móvil se orienta a la recogida de datos en campo, el portal web a la gestión y explotación de la información, y la API actúa como punto común de autenticación, validación, persistencia y sincronización.

Esta aproximación permite conectar la recogida de datos con la evaluación de tendencias y medidas de mitigación. Al conservar datos cuantitativos, metadatos y trazabilidad por campaña, la plataforma puede apoyar comparaciones antes y después de una intervención, ayudar a identificar fuentes probables de contaminación y aportar evidencias para valorar la eficacia de políticas o acciones de reducción de residuos.

3.2. Actores y perfiles de usuario

OmniLitter es una plataforma colaborativa en la que distintos usuarios participan en la gestión de campañas de monitorización y en la recogida de observaciones de residuos. Debido a esta naturaleza colaborativa, el sistema implementa un modelo de control de acceso basado en dos conceptos diferenciados: el rol global del usuario dentro de la plataforma y el rol que dicho usuario posee dentro de cada grupo de trabajo.

Esta separación permite que una misma persona pueda desempeñar diferentes responsabilidades según el grupo en el que participe, garantizando al mismo tiempo el aislamiento de la información entre equipos de trabajo independientes.

El acceso a los datos se estructura alrededor de tres elementos fundamentales:

- La cuenta de usuario, que identifica a la persona dentro de la plataforma.
- El grupo de trabajo, que delimita el conjunto de campañas, sesiones y observaciones accesibles.
- El rol asignado, que determina las acciones que el usuario puede realizar sobre dichos datos.

A continuación se describen los principales perfiles contemplados por el sistema.

3.2.1. Visitante

El visitante corresponde a cualquier persona que accede a la parte pública de la plataforma sin haber iniciado sesión. Este perfil únicamente puede consultar información general sobre OmniLitter y acceder a las funcionalidades de autenticación, como el inicio de sesión, el registro de una nueva cuenta o la recuperación de contraseña.

No dispone de acceso a campañas, observaciones, métricas ni información perteneciente a los grupos de trabajo.

3.2.2. Usuario autenticado

El usuario autenticado constituye el perfil base del sistema. Se trata de cualquier persona que dispone de una cuenta válida y ha iniciado sesión correctamente.

Este perfil puede gestionar la información asociada a su propia cuenta, consultar los grupos a los que pertenece, seleccionar un grupo activo y aceptar o rechazar invitaciones recibidas. Sin embargo, los permisos operativos sobre campañas y observaciones dependen del rol que posea dentro de cada grupo.

3.2.3. Investigador de campo

El investigador de campo es el perfil responsable de la recogida de información durante las campañas de monitorización. Su actividad principal se desarrolla mediante la aplicación móvil, desde la cual puede crear sesiones de muestreo, registrar observaciones, clasificar residuos mediante la taxonomía disponible, adjuntar fotografías y capturar la localización geográfica de cada registro.

Además, este perfil puede trabajar en modo offline cuando no existe conectividad, almacenando localmente la información para sincronizarla posteriormente con el servidor.

Aunque puede consultar la información necesaria para desarrollar su trabajo, no dispone de permisos para administrar usuarios, gestionar campañas o modificar la estructura organizativa del grupo.

3.2.4. Administrador de grupo

El administrador de grupo es el responsable de coordinar la actividad de un grupo de trabajo concreto. Este perfil puede gestionar los miembros del grupo, enviar invitaciones, asignar roles de membresía y administrar las campañas asociadas a dicho grupo.

Entre sus funciones se encuentran la creación, edición y organización de campañas de monitorización, así como la supervisión de la actividad realizada por los investigadores de campo. También dispone de acceso a métricas agregadas e información de seguimiento relacionada con las campañas del grupo.

Sus permisos se limitan exclusivamente a los grupos en los que posee dicho rol. Por tanto, un mismo usuario puede actuar como administrador en un grupo y como investigador de campo en otro diferente.

3.2.5. Superadministrador

El superadministrador representa el perfil con mayor nivel de autoridad dentro de OmniLitter. Su función principal consiste en administrar la plataforma a nivel global y garantizar su correcto funcionamiento.

Este perfil puede gestionar usuarios, modificar roles globales, supervisar grupos de trabajo y realizar tareas de soporte operativo dentro de la plataforma. A diferencia del administrador de grupo, sus permisos no se encuentran restringidos a un único grupo, sino que abarcan la totalidad del sistema.

Debido a su alcance, este perfil está destinado a tareas de administración y soporte de la plataforma, y no a la realización de actividades habituales de monitorización en campo.

3.2.6. Usuario sin grupo activo

Finalmente, OmniLitter contempla la situación de usuarios registrados que todavía no pertenecen a ningún grupo o que no han seleccionado uno como grupo activo.

En este estado, el usuario puede gestionar su cuenta personal y revisar invitaciones pendientes, pero no tiene acceso a campañas, observaciones ni funcionalidades relacionadas con la monitorización hasta incorporarse a un grupo de trabajo.

3.2.7. Actores de apoyo

Además de los usuarios que interactúan directamente con la plataforma, OmniLitter depende de diversos componentes técnicos que permiten el funcionamiento del sistema.

Entre ellos destacan el servicio de autenticación encargado de validar credenciales y gestionar sesiones, el servicio de correo utilizado para determinadas comunicaciones automáticas, los dispositivos móviles empleados durante las campañas de campo y la infraestructura backend responsable de la lógica de negocio y el almacenamiento persistente de la información.

Aunque estos elementos no representan usuarios propiamente dichos, constituyen actores fundamentales para la correcta ejecución de los procesos definidos por la plataforma.

3.3. Plataformas del sistema

OmniLitter se concibe como una plataforma compuesta por varios elementos que colaboran entre sí para cubrir las distintas fases del proceso de monitorización de residuos. La naturaleza del problema exige disponer de herramientas adaptadas tanto al trabajo de campo como a la gestión y análisis posterior de la información recopilada.

Para dar respuesta a estas necesidades, el sistema se estructura en tres componentes principales: una aplicación móvil, un portal web y un backend centralizado. Esta separación permite adaptar cada componente a las tareas específicas que debe desempeñar, manteniendo al mismo tiempo una gestión unificada de la información.

3.3.1. Aplicación móvil

La aplicación móvil constituye la herramienta principal para la recogida de datos en campo. Está orientada a los usuarios que participan directamente en las campañas de monitorización y les permite registrar observaciones de residuos en el lugar donde se producen.

A través de esta aplicación es posible consultar campañas, iniciar sesiones de muestreo, registrar observaciones, capturar fotografías asociadas a los residuos encontrados y almacenar la localización geográfica de cada registro. Además, la aplicación está diseñada para funcionar en entornos con conectividad limitada, permitiendo continuar el trabajo incluso cuando no existe acceso a Internet y sincronizando posteriormente la información con el servidor.

3.3.2. Portal web

El portal web proporciona las herramientas necesarias para la gestión y supervisión de la información recopilada. Esta plataforma está orientada principalmente a administradores de grupo, investigadores y superadministradores.

Desde el portal web es posible gestionar usuarios y grupos de trabajo, crear y administrar campañas de monitorización, consultar observaciones registradas, visualizar información geográfica mediante mapas interactivos y acceder a métricas agregadas sobre los datos recopilados. Asimismo, permite exportar la información para su utilización en procesos posteriores de análisis científico.

A diferencia de la aplicación móvil, cuyo objetivo principal es facilitar la recogida de datos en campo, el portal web está enfocado a la coordinación, revisión y explotación de la información almacenada.

3.3.3. Backend y almacenamiento centralizado

El backend actúa como elemento de coordinación entre las distintas plataformas del sistema. Su función consiste en centralizar la lógica de negocio, gestionar la autenticación de usuarios, controlar los permisos de acceso y garantizar la consistencia de los datos almacenados.

Tanto la aplicación móvil como el portal web se comunican con este componente para consultar o registrar información. De esta forma, todas las operaciones se realizan sobre una única fuente de datos común, evitando inconsistencias y facilitando la gestión centralizada del sistema.

La información recopilada durante las campañas se almacena en una base de datos centralizada que actúa como repositorio principal de usuarios, grupos, campañas, sesiones de muestreo, observaciones y fotografías asociadas.

3.3.4. Integración de las plataformas

La combinación de estas plataformas permite cubrir el ciclo completo de trabajo de una campaña de monitorización. La aplicación móvil facilita la recogida de datos sobre el terreno, el backend garantiza la gestión y consistencia de la información, y el portal web proporciona las herramientas necesarias para su consulta, supervisión y análisis posterior.

Esta arquitectura permite que cada componente se especialice en una función concreta, favoreciendo la mantenibilidad del sistema y facilitando su evolución futura.

3.4. Funcionalidades principales

OmniLitter proporciona un conjunto de funcionalidades orientadas a cubrir el ciclo completo de una campaña de monitorización de residuos, desde la organización de los equipos de trabajo hasta la recogida y posterior consulta de los datos obtenidos.

3.4.1. Gestión de usuarios y grupos

La plataforma permite el registro y autenticación de usuarios, así como la organización de estos en grupos de trabajo independientes. Cada grupo dispone de sus propias campañas, sesiones y observaciones, garantizando el aislamiento de la información entre equipos.

Además, el sistema incorpora distintos niveles de permisos que permiten diferenciar entre usuarios de campo, administradores de grupo y administradores globales de la plataforma.

3.4.2. Gestión de campañas

OmniLitter permite crear y administrar campañas de monitorización asociadas a un grupo de trabajo. Cada campaña define el contexto en el que se desarrollarán las actividades de muestreo, incluyendo información descriptiva, fechas de realización y características generales del entorno estudiado.

Las campañas actúan como elemento organizador de toda la información recogida durante el trabajo de campo.

3.4.3. Sesiones de muestreo

Dentro de cada campaña, los usuarios pueden crear sesiones de muestreo que representan actividades concretas de recogida de datos.

Las sesiones permiten registrar información contextual relacionada con el trabajo realizado, como el entorno de estudio, el método de muestreo empleado, el protocolo aplicado, la duración de la actividad, el tipo de transecto o cuadrante y la ubicación geográfica asociada.

Esta funcionalidad aporta trazabilidad a los datos recopilados, permitiendo conocer las condiciones bajo las que se registró cada observación.

3.4.4. Registro de observaciones

El registro de observaciones constituye la funcionalidad principal del sistema. Durante una sesión de muestreo, los usuarios pueden documentar los residuos encontrados indicando su localización, fecha de registro y clasificación correspondiente.

Cada observación queda vinculada a una campaña y una sesión concretas, permitiendo mantener la coherencia y organización de los datos almacenados.

3.4.5. Clasificación mediante taxonomías

La plataforma incorpora una taxonomía de residuos que permite clasificar las observaciones de forma homogénea y estandarizada, tomando como referencia listas comunes de categorías utilizadas en monitorización de basuras.

El uso de categorías comunes mejora la calidad de los datos recopilados y facilita su comparación entre campañas, grupos de trabajo, entornos y periodos temporales diferentes.

3.4.6. Registro fotográfico y geolocalización

Las observaciones pueden complementarse mediante fotografías y coordenadas geográficas obtenidas desde el dispositivo móvil.

Estas evidencias permiten contextualizar cada registro, aumentar la fiabilidad de los datos y facilitar posteriores procesos de revisión y análisis.

3.4.7. Trabajo offline y sincronización

La aplicación móvil ha sido diseñada para utilizarse en entornos donde la conectividad puede ser limitada o inexistente.

Por este motivo, los usuarios pueden continuar registrando observaciones sin conexión a Internet. Cuando la conectividad se restablece, la información almacenada localmente se sincroniza con el sistema central, garantizando la disponibilidad de los datos para el resto de usuarios autorizados.

3.4.8. Consulta y visualización de información

El portal web permite consultar la información recopilada durante las campañas mediante diferentes mecanismos de visualización.

Los usuarios autorizados pueden revisar observaciones, consultar información geográfica a través de mapas interactivos y acceder a métricas generales relacionadas con la actividad de monitorización realizada.

3.4.9. Exportación de datos

OmniLitter incorpora mecanismos para exportar la información recopilada en formatos adecuados para su tratamiento posterior.

Esta funcionalidad facilita la reutilización de los datos en herramientas externas de análisis y favorece su utilización en estudios científicos relacionados con la contaminación por residuos, incluyendo la generación de series temporales, comparaciones espaciales y evaluaciones de medidas de mitigación.

3.4.10. Resumen funcional

De forma resumida, OmniLitter permite:

- Gestionar usuarios, grupos y permisos.
- Planificar campañas de monitorización.
- Realizar sesiones de muestreo en campo.
- Registrar observaciones geolocalizadas.
- Clasificar residuos mediante una taxonomía estandarizada.
- Asociar fotografías y metadatos a las observaciones.
- Trabajar sin conexión y sincronizar posteriormente los datos.
- Consultar la información recopilada mediante mapas y métricas.
- Exportar los datos para su análisis posterior.

3.5. Restricciones y alcance excluido

Con el objetivo de acotar el alcance del proyecto y ajustarlo a las limitaciones temporales y de recursos propias de un Trabajo Fin de Grado, se decidió excluir una serie de funcionalidades y actividades que, aunque podrían aportar valor a la plataforma, no resultaban imprescindibles para cumplir los objetivos principales establecidos.

Las principales exclusiones son las siguientes:

- **Soporte multiidioma avanzado.** Aunque la arquitectura de la aplicación permite la internacionalización de la interfaz, el alcance del proyecto se limita al español y al inglés. La incorporación de idiomas adicionales requeriría un esfuerzo significativo de traducción y mantenimiento, además de depender en algunos casos de servicios externos incompatibles con los requisitos de funcionamiento offline de la aplicación móvil.
- **Mantenimiento evolutivo y correctivo posterior a la entrega.** El presente trabajo contempla el análisis, diseño, implementación y validación de la plataforma. Las actividades relacionadas con el mantenimiento posterior, incorporación de nuevas funcionalidades o soporte continuado tras la finalización del proyecto quedan fuera de su alcance.

- **Identificación automática de residuos mediante inteligencia artificial.** Durante las fases iniciales del proyecto se consideró la posibilidad de incorporar mecanismos de reconocimiento automático de residuos a partir de imágenes. Sin embargo, el desarrollo, entrenamiento y validación de modelos de visión artificial requiere una inversión de tiempo, recursos computacionales y conjuntos de datos especializados que exceden las posibilidades de este Trabajo Fin de Grado.
- **Publicación oficial en tiendas de aplicaciones.** Aunque la aplicación móvil ha sido desarrollada para poder ejecutarse en dispositivos reales, la publicación oficial en plataformas como Google Play Store o Apple App Store no forma parte del alcance del proyecto. Este proceso implica requisitos administrativos, costes económicos y procedimientos de revisión que no resultan necesarios para la validación académica de la solución desarrollada.

Estas exclusiones permiten concentrar los esfuerzos de desarrollo en las funcionalidades esenciales de OmniLitter, garantizando una solución completa y funcional dentro de las restricciones temporales y organizativas del proyecto.

4. Requisitos y análisis

Este capítulo recoge el proceso seguido para obtener, contrastar y consolidar los requisitos de OmniLitter. Al tratarse de un proyecto desarrollado para un cliente real y dentro de un dominio científico específico, la definición de requisitos no se cerró en una única fase inicial, sino que fue evolucionando a partir de reuniones con el cliente, validación mediante prototipos funcionales y revisión del alcance del sistema.

Por este motivo, los requisitos se presentan en dos niveles. En primer lugar, se describe la evolución del registro de requisitos, indicando qué requisitos estaban presentes en cada sesión de trabajo y qué cambios se introdujeron en cada versión. En segundo lugar, se recoge la versión final consolidada, relacionando cada requisito con el caso de uso al que da soporte cuando procede.

4.1. Evolución del registro de requisitos

Durante el desarrollo de OmniLitter se manejaron tres versiones principales del registro de requisitos. Cada una representa un momento distinto del proyecto: la toma inicial de requisitos, la validación mediante prototipos y la consolidación posterior a la última reunión de validación. Además, se incorporó un refinamiento de dominio aportado por el cliente, centrado en la necesidad de datos científicos armonizados, comparables y útiles para evaluar medidas de mitigación.

4.1.1. Versión 1: toma inicial de requisitos

Fecha aproximada: 09/02/2026.

Estado: primera versión del registro de requisitos.

Objetivo de la sesión: recoger las necesidades iniciales del cliente y definir una primera lista de requisitos funcionales, no funcionales y científicos.

En la primera reunión se identificó la necesidad de una herramienta capaz de registrar datos sobre residuos en distintos entornos naturales. El cliente planteó la importancia de capturar observaciones geolocalizadas, asociar fotografías, registrar metadatos del muestreo, trabajar sin conexión y sincronizar posteriormente los datos con un servidor. También

se identificaron necesidades de usuarios, grupos, roles, visualización en mapa, estadísticas y exportación.

Cuadro 4.1: Requisitos identificados en la versión 1.

ID	Descripción	Prioridad
RF-01	Registro de tipo de residuo a partir de tipos preestablecidos.	Crítica
RF-02	Registro del tamaño del residuo observado.	Media
RF-03	Geolocalización automática de cada observación mediante GPS.	Crítica
RF-04	Registro automático de fecha y hora del avistamiento.	Alta
RF-05	Asociación de una o varias fotografías al registro de residuo.	Alta
RF-06	Funcionamiento sin conexión, con almacenamiento local de datos.	Crítica
RF-07	Sincronización de datos locales con el servidor al recuperar conectividad.	Crítica
RF-08	Selección del tipo de entorno antes de iniciar el muestreo.	Alta
RF-09	Registro de parámetros de muestreo, como ancho de observación o duración.	Alta
RF-10	Interfaz disponible en español e inglés.	Media
RF-11	Creación de cuentas de usuario.	Alta
RF-12	Creación de grupos de trabajo y asociación de usuarios.	Crítica
RF-13	Soporte de roles de usuario.	Crítica
RF-14	Visualización de observaciones geolocalizadas en mapa.	Alta
RF-15	Cálculo de estadísticas básicas a partir de los datos almacenados.	Media
RF-16	Privacidad por grupo para limitar la visibilidad de datos.	Crítica
RNF-01	Interfaz optimizada para tablet y uso en campo.	Alta
RNF-02	Comunicación segura entre aplicación móvil y servidor mediante HTTPS.	Crítica
RNF-03	Compatibilidad de la aplicación móvil con Android e iOS.	Media
RDC-01	Almacenamiento de datos según la taxonomía proporcionada por el cliente.	Crítica
RDC-02	Exportación de datos en formato CSV para análisis científico posterior.	Alta

Esta versión constituye la línea base del proyecto y permite identificar los bloques iniciales: captura de datos, geolocalización, fotografías, funcionamiento offline, sincronización, usuarios, grupos, roles, mapa, estadísticas, seguridad, compatibilidad, taxonomía y exportación.

4.1.2. Versión 2: validación mediante prototipos

Estado: versión de trabajo posterior a la toma inicial.

Objetivo de la sesión: contrastar los requisitos iniciales con prototipos funcionales del portal web y de la aplicación móvil.

Tras la primera toma de requisitos, se elaboraron prototipos que representaban los principales flujos del sistema: autenticación, selección de grupo activo, creación de sesiones de muestreo, registro de observaciones, consulta de datos, mapa y sincronización. Esta revisión permitió transformar necesidades generales de la V1 en requisitos más cercanos al funcionamiento real del sistema.

Cuadro 4.2: Requisitos definidos en la versión 2 tras la validación con prototipos.

ID	Descripción	Prioridad
RF-001	Inicio de sesión en portal web y aplicación móvil.	Crítica
RF-002	Cierre de sesión y revocación de sesión local.	Alta
RF-004	Roles y control de acceso por grupo.	Crítica
RF-005	Creación de grupos y asociación de usuarios.	Crítica
RF-006	Selección de grupo activo para filtrar datos y acciones.	Alta
RF-007	Trabajo por sesiones de muestreo en la aplicación móvil.	Crítica
RF-008	Captura de coordenadas GPS en sesiones y observaciones.	Crítica
RF-009	Creación y consulta de campañas asociadas a un grupo.	Alta
RF-010	Fotografías asociadas a observaciones.	Alta
RF-012	Navegación por taxonomía jerárquica basada en categorías estandarizadas de residuos.	Crítica
RF-015	Registro de cantidad observada como entero positivo.	Crítica
RF-020	Revisión de sesión antes de finalizarla.	Alta
RF-022	Funcionamiento sin conexión durante la captura de campo.	Crítica
RF-023	Cola de sincronización y subida posterior de datos pendientes.	Crítica
RF-027	Listado de sesiones y observaciones con filtros en el portal.	Alta
RF-028	Visualización de observaciones en mapa.	Alta
RF-030	Dashboard con métricas agregadas.	Media
RF-031	Exportación de datos para análisis externo.	Media
RF-032	Gestión de usuarios desde el portal.	Alta
RF-033	Gestión de grupos y membresías desde el portal.	Alta
RF-035	Soporte de interfaz en español e inglés.	Media
RNF-001	Seguridad en comunicaciones y acceso autenticado.	Crítica
RNF-002	Aislamiento de datos entre grupos de investigación.	Crítica
RNF-003	Enfoque <i>offline-first</i> para la aplicación móvil.	Crítica

ID	Descripción	Prioridad
RNF-004	Fiabilidad de sincronización ante cortes de red o cierres inesperados.	Crítica
RNF-005	Usabilidad orientada al trabajo de campo.	Alta
RNF-008	Trazabilidad entre requisitos, operaciones relevantes y pruebas.	Alta
RDC-001	Taxonomía jerárquica con código identificador estable.	Crítica
RDC-002	Metadatos científicos de sesión.	Crítica
RDC-003	Metadatos técnicos de fotografías cuando estén disponibles.	Media
RDC-004	Exportación científica con campos suficientes para análisis.	Media
RDC-005	Versión de taxonomía asociada a sesiones y observaciones.	Alta

Respecto a la V1, la creación de cuentas pasa a ser un flujo de autenticación completo, los roles se refinan como control de acceso por grupo, los metadatos se organizan alrededor de sesiones de muestreo y la taxonomía deja de ser una lista simple para convertirse en una estructura jerárquica navegable.

4.1.3. Versión 3: consolidación tras la última reunión y revisión de dominio

Fecha aproximada: 10/06/2026.

Estado: versión final consolidada de requisitos.

Objetivo de la sesión: revisar el flujo real de la aplicación móvil, especialmente sesiones, sincronización, mapa y operación posterior del sistema, e incorporar la revisión científica posterior aportada por el cliente.

En la última reunión se revisó el funcionamiento de la aplicación móvil en relación con las sesiones de muestreo. Se reforzó la necesidad de consultar qué sesiones estaban pendientes, cuántas observaciones contenían, si una sesión seguía activa y cómo debía finalizarse antes de sincronizar. También se aclaró que la memoria debía documentar el despliegue y la administración básica, dejando el mantenimiento evolutivo posterior fuera del alcance académico del proyecto.

La información aportada por el cliente reforzó la motivación científica del sistema: la falta de datos espaciales y temporales, la ausencia de metodologías armonizadas en algunos entornos terrestres y la necesidad de capturar datos y metadatos de campo con suficiente calidad para comparar campañas, detectar tendencias y evaluar la eficacia de políticas o medidas de mitigación. Esta revisión se alinea con la literatura sobre basuras fluviales, que reclama datos comparables, listas comunes de categorías, metadatos y compatibilidad entre evaluaciones continentales, costeras y marinas [9, 10, 12].

Cuadro 4.3: Requisitos consolidados o refinados en la versión 3.

ID	Descripción	Prioridad
RF-021	Estado de sincronización de sesiones y observaciones: sincronizado, pendiente, fallido o en conflicto.	Alta
RF-022	Funcionamiento offline durante la captura de campo con campañas descargadas, sesiones, observaciones y elementos asociados.	Crítica
RF-023	Sincronización por lotes de cambios pendientes, con resultado individual por operación.	Crítica
RF-024	Consulta de cambios remotos del grupo activo desde una marca temporal.	Alta
RF-025	Persistencia local de sesiones y observaciones mediante SQLite.	Alta
RF-028	Visualización en mapa con filtros por entorno, campaña, sesión o localización cuando proceda.	Alta
RF-035	Interfaz en español e inglés, sin soporte multiidioma avanzado dentro del alcance.	Media
RF-036	Configuración de preferencias de sincronización y localización.	Media
RNF-006	Operabilidad mediante documentación suficiente para despliegue y administración básica.	Alta
RNF-007	Mantenibilidad orientada a facilitar futuras modificaciones de categorías o metadatos.	Media
RDC-006	Versionado y documentación de futuras evoluciones de categorías, taxonomías y metadatos.	Media

La V3 no sustituye al conjunto completo de la V2, sino que lo consolida. En particular, precisa el flujo *offline-first*, refuerza la trazabilidad científica, explicita el estado de sincronización de las sesiones y acota el alcance operativo posterior a la entrega.

4.2. Casos de uso de referencia

Los casos de uso de referencia permiten relacionar los requisitos funcionales finales con interacciones concretas del sistema. La relación se ha actualizado a partir del documento específico de casos de uso, que organiza OmniLitter en torno a catorce flujos principales.

- **CU-01 – Registrarse, autenticarse y gestionar la sesión.** El usuario accede al sistema, crea una cuenta, recupera el acceso cuando procede y gestiona su sesión activa.
- **CU-02 – Gestionar el perfil y el grupo activo.** El usuario mantiene sus datos

básicos y selecciona el grupo de trabajo sobre el que opera.

- **CU-03 – Administrar grupos, miembros e invitaciones.** Los usuarios con permisos organizan equipos de trabajo, membresías e invitaciones.
- **CU-04 – Administrar usuarios de la plataforma.** El superadministrador gestiona cuentas, roles globales y estado de los usuarios.
- **CU-05 – Crear y gestionar campañas de muestreo.** El administrador de grupo planifica campañas asociadas al grupo activo.
- **CU-06 – Crear y actualizar una sesión de muestreo.** El investigador inicia una sesión dentro de una campaña y registra el progreso del trabajo de campo.
- **CU-07 – Registrar observaciones de residuos.** El investigador registra residuos dentro de una sesión, con taxonomía, cantidad, fotografía, coordenadas y notas cuando proceda.
- **CU-08 – Revisar y finalizar una sesión de muestreo.** El investigador revisa la información recogida antes de cerrar la sesión.
- **CU-09 – Trabajar sin conexión y sincronizar datos.** La aplicación móvil conserva datos localmente y los sincroniza cuando existe conectividad.
- **CU-10 – Consultar sesiones y observaciones.** Los usuarios autorizados consultan la información registrada desde web o móvil respetando el alcance del grupo activo.
- **CU-11 – Visualizar la información en mapa.** Los usuarios interpretan espacialmente observaciones, rutas y áreas muestreadas.
- **CU-12 – Consultar métricas y analítica.** Los usuarios autorizados analizan información agregada del grupo activo.
- **CU-13 – Exportar datos.** Los usuarios autorizados extraen datos en formatos reutilizables para análisis externo.
- **CU-14 – Configurar preferencias de la aplicación.** El usuario adapta idioma, localización y comportamiento de sincronización, especialmente en la aplicación móvil.

4.3. Requisitos finales consolidados

En esta sección se presenta la versión final consolidada de requisitos funcionales. Para cada bloque se introduce el caso de uso correspondiente y se describen los requisitos asociados mediante tarjetas. Los criterios de aceptación se tratan en la planificación y validación de pruebas, por lo que no se duplican dentro de las tarjetas de requisitos.

4.3.1. CU-01 – Registrarse, autenticarse y gestionar la sesión

El usuario debe poder acceder al sistema de forma segura, tanto desde el portal web como desde la aplicación móvil. Este caso de uso cubre el registro de una nueva cuenta, el inicio de sesión, la renovación de la sesión, el cierre de sesión y la recuperación de acceso.

RF-001 – Inicio de sesión en portal web y aplicación móvil

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-01.

Descripción: el sistema deberá permitir que un usuario con credenciales válidas inicie sesión desde cualquiera de las plataformas disponibles. Tras autenticarse, el usuario recibirá una sesión válida y accederá únicamente a las funcionalidades permitidas por su perfil y sus grupos.

RF-002 – Cierre de sesión y revocación de sesión

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-01.

Descripción: el sistema deberá permitir cerrar la sesión activa y revocar el token de refresco asociado. Tras el cierre de sesión, el usuario no podrá acceder a rutas protegidas sin autenticarse de nuevo.

RF-003 – Registro de nuevos usuarios

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-01.

Descripción: el sistema deberá permitir la creación de nuevas cuentas mediante nombre, correo electrónico y contraseña. La contraseña deberá cumplir las reglas mínimas de seguridad definidas por la aplicación.

RF-018 – Recuperación y restablecimiento de contraseña

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-01.

Descripción: el sistema deberá permitir solicitar la recuperación de contraseña mediante el correo electrónico de la cuenta y restablecerla mediante un token temporal válido.

RF-019 – Cambio de contraseña del usuario autenticado

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-01.

Descripción: el sistema deberá permitir que un usuario autenticado cambie su contraseña indicando la contraseña actual y una nueva contraseña válida.

4.3.2. CU-02 – Gestionar el perfil y el grupo activo

El usuario autenticado debe poder mantener sus datos básicos y seleccionar el grupo de trabajo sobre el que desea operar. El grupo activo determina campañas, sesiones, observaciones, métricas, mapas y operaciones permitidas.

RF-006 – Selección de grupo activo

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-02.

Descripción: el sistema deberá permitir seleccionar un grupo activo entre los grupos a los que pertenece el usuario. Los datos mostrados y las operaciones disponibles deberán corresponderse con dicho grupo.

RF-011 – Edición del perfil básico del usuario

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-02.

Descripción: el sistema deberá permitir consultar y modificar los datos básicos del usuario autenticado, como el nombre visible y el correo electrónico.

4.3.3. CU-03 – Administrar grupos, miembros e invitaciones

Los usuarios con permisos adecuados deben poder organizar equipos de trabajo mediante grupos, gestionar miembros y controlar invitaciones pendientes. Este caso de uso garantiza la colaboración sin mezclar datos de distintos equipos.

RF-004 – Roles y control de acceso por grupo

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-03, CU-04, CU-05, CU-10, CU-12 y CU-13.

Descripción: el sistema deberá aplicar permisos en función del rol global del usuario y de su rol dentro de cada grupo. Un usuario solo podrá acceder a los datos y acciones permitidos en el grupo correspondiente.

RF-005 – Creación de grupos y asociación de usuarios

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-03.

Descripción: el sistema deberá permitir crear grupos de trabajo y asociar usuarios a dichos grupos mediante membresías.

RF-033 – Gestión de grupos y membresías

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-03.

Descripción: el sistema deberá permitir consultar, editar, archivar, restaurar y eliminar grupos cuando el usuario tenga permisos suficientes. También deberá permitir añadir miembros, modificar sus roles y expulsarlos del grupo.

RF-034 – Gestión de invitaciones a grupos

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-03.

Descripción: el sistema deberá permitir invitar usuarios mediante correo electrónico y permitir que los destinatarios acepten o rechacen dichas invitaciones.

4.3.4. CU-04 – Administrar usuarios de la plataforma

El superadministrador debe poder gestionar las cuentas de usuario de la plataforma. Esta administración global queda separada de la gestión ordinaria de los grupos.

RF-032 – Gestión de usuarios desde el portal

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-04.

Descripción: el portal web deberá permitir a un superadministrador consultar usuarios, crear cuentas, cambiar roles globales y activar o desactivar cuentas.

4.3.5. CU-05 – Crear y gestionar campañas de muestreo

El administrador de grupo debe poder planificar el trabajo de campo mediante campañas asociadas al grupo activo. Las campañas contextualizan las sesiones y permiten analizar posteriormente los datos por proyecto, periodo o entorno.

RF-009 – Creación y consulta de campañas asociadas a un grupo

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-05.

Descripción: el sistema deberá permitir crear, consultar, modificar y eliminar campañas asociadas al grupo activo. Cada campaña deberá contener información básica como nombre, descripción, estado, fechas y contexto de muestreo.

4.3.6. CU-06 – Crear y actualizar una sesión de muestreo

El investigador de campo debe poder iniciar una sesión de muestreo dentro de una campaña. La sesión agrupa el trabajo realizado durante una salida de campo e incluye información científica necesaria para interpretar las observaciones.

RF-007 – Trabajo por sesiones de muestreo

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-06.

Descripción: la aplicación móvil deberá organizar el trabajo de campo mediante sesiones de muestreo asociadas a una campaña y a un observador.

RF-008 – Captura de coordenadas GPS

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-06 y CU-07.

Descripción: la aplicación móvil deberá capturar coordenadas GPS al iniciar una sesión y al registrar observaciones, siempre que el dispositivo disponga de permisos y señal suficiente.

RF-017 – Inclusión de notas de campo

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-06 y CU-07.

Descripción: el sistema deberá permitir registrar notas descriptivas en sesiones y observaciones para completar la información estrictamente estructurada.

RF-026 – Registro del progreso de la sesión

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-06 y CU-08.

Descripción: el sistema deberá permitir actualizar el progreso de una sesión mediante puntos de ruta, duración, área muestreada y parámetros asociados al método de muestreo, como cuadrícula, transecto o muestreo libre.

4.3.7. CU-07 – Registrar observaciones de residuos

Durante una sesión de muestreo, el investigador de campo debe poder registrar observaciones de residuos. Cada observación queda vinculada a la sesión, al usuario observador, a la localización, a una categoría taxonómica y a los elementos de residuo identificados.

RF-013 – Registro de observaciones dentro de una sesión

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-07.

Descripción: la aplicación móvil deberá permitir crear observaciones asociadas a una sesión de muestreo activa.

RF-014 – Asociación de observaciones a campaña, grupo y usuario observador

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-07.

Descripción: el sistema deberá conservar la trazabilidad de cada observación mediante su relación con la sesión, la campaña, el grupo y el usuario que la registró.

RF-012 – Navegación por taxonomía jerárquica

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-07.

Descripción: el sistema deberá permitir seleccionar residuos desde una taxonomía estructurada por códigos, categorías y versiones activas.

RF-015 – Cantidad observada

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-07.

Descripción: el sistema deberá registrar la cantidad observada como un número entero positivo y no deberá admitir cantidades vacías, negativas o iguales a cero.

RF-016 – Registro de material y categoría del residuo

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-07.

Descripción: el sistema deberá permitir registrar la categoría del residuo y, cuando proceda, el material asociado a cada elemento observado.

RF-010 – Fotografías asociadas a observaciones

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-07.

Descripción: la aplicación deberá permitir adjuntar fotografías a una observación y almacenar los metadatos necesarios para relacionarlas con el registro correspondiente.

4.3.8. CU-08 – Revisar y finalizar una sesión de muestreo

Antes de cerrar una sesión, el investigador debe poder revisar la información recogida y comprobar que las observaciones registradas son coherentes. La finalización marca el cierre del trabajo de campo asociado.

RF-020 – Revisión de sesión antes de finalizarla

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-08.

Descripción: la aplicación móvil deberá permitir consultar las observaciones registradas en una sesión antes de finalizarla.

4.3.9. CU-09 – Trabajar sin conexión y sincronizar datos

La aplicación móvil debe permitir trabajar en campo en condiciones de conectividad limitada. Los datos creados localmente deberán conservarse en el dispositivo y sincronizarse con el servidor cuando exista conexión disponible.

RF-022 – Funcionamiento offline durante la captura de campo

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-09.

Descripción: la aplicación móvil deberá permitir crear y consultar datos relevantes sin conexión, incluyendo campañas descargadas, sesiones, observaciones y elementos asociados.

RF-025 – Persistencia local de sesiones y observaciones

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-09.

Descripción: la aplicación móvil deberá almacenar localmente los datos necesarios mediante SQLite para evitar pérdida de información durante el trabajo de campo.

RF-021 – Estado de sincronización de sesiones y observaciones

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-09.

Descripción: la aplicación deberá mostrar el estado de sincronización de los datos locales, diferenciando registros sincronizados, pendientes, fallidos o en conflicto.

RF-023 – Sincronización por lotes de cambios pendientes

Tipo: funcional.

Prioridad: crítica.

Caso de uso asociado: CU-09.

Descripción: el sistema deberá disponer de una cola de sincronización que envíe al servidor las operaciones pendientes. Cada operación del lote deberá recibir un resultado individual.

RF-024 – Consulta de cambios remotos del grupo activo

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-09.

Descripción: la aplicación móvil deberá poder consultar cambios remotos desde una marca temporal para actualizar la copia local de los datos del grupo activo.

4.3.10. CU-10 – Consultar sesiones y observaciones

Los usuarios autorizados deben poder consultar la información registrada desde el portal web y desde la aplicación móvil. La consulta debe respetar el alcance del grupo activo y

los permisos del usuario.

RF-027 – Listados de sesiones y observaciones con filtros

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-10, CU-12 y CU-13.

Descripción: el sistema deberá permitir consultar sesiones y observaciones mediante listados filtrables por campaña, sesión, entorno, fecha u otros criterios disponibles.

4.3.11. CU-11 – Visualizar la información en mapa

Los usuarios autorizados deben poder interpretar espacialmente los datos registrados. El mapa permite visualizar observaciones, rutas de sesiones y áreas muestreadas para facilitar el análisis territorial.

RF-028 – Visualización de observaciones en mapa

Tipo: funcional.

Prioridad: alta.

Caso de uso asociado: CU-11.

Descripción: el portal web y la aplicación deberán permitir representar observaciones geolocalizadas en una vista cartográfica.

RF-029 – Visualización de rutas y áreas muestreadas

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-11.

Descripción: el sistema deberá permitir consultar rutas de sesiones y áreas muestreadas a partir de los puntos de recorrido registrados durante el trabajo de campo.

4.3.12. CU-12 – Consultar métricas y analítica

Los usuarios autorizados deben poder analizar la información agregada del grupo activo. Este caso de uso transforma los registros individuales en indicadores útiles para seguimiento, revisión y toma de decisiones.

RF-030 – Dashboard con métricas agregadas

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-12.

Descripción: el portal web deberá mostrar indicadores agregados sobre campañas, sesiones, observaciones, fotografías y residuos registrados.

4.3.13. CU-13 – Exportar datos

Los usuarios autorizados deben poder extraer la información registrada para su tratamiento externo, elaboración de informes o análisis científico. La exportación favorece la reutilización y reproducibilidad de los datos.

RF-031 – Exportación de datos en formatos reutilizables

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-13.

Descripción: el sistema deberá permitir exportar datos seleccionados en formatos adecuados para análisis posterior, incluyendo CSV, JSON y GeoJSON cuando proceda.

4.3.14. CU-14 – Configurar preferencias de la aplicación

El usuario debe poder adaptar la aplicación a sus condiciones de uso, especialmente en el dispositivo móvil. Las preferencias permiten ajustar idioma, localización y comportamiento de sincronización.

RF-035 – Soporte de interfaz en español e inglés

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-14.

Descripción: el sistema deberá permitir utilizar la interfaz al menos en español e inglés.

RF-036 – Configuración de preferencias de sincronización y localización

Tipo: funcional.

Prioridad: media.

Caso de uso asociado: CU-14.

Descripción: la aplicación móvil deberá permitir configurar opciones como sincronización automática, sincronización solo por Wi-Fi y uso de localización de alta precisión.

4.4. Requisitos no funcionales consolidados

Además de los requisitos funcionales, OmniLitter debe cumplir una serie de condiciones de calidad que afectan al sistema en su conjunto.

RNF-001 – Seguridad

Prioridad: crítica.

Descripción: la comunicación entre clientes y backend deberá realizarse mediante HTTPS/TLS en entornos de despliegue, y el acceso a datos deberá estar protegido mediante autenticación y autorización por grupo.

RNF-002 – Aislamiento de datos

Prioridad: crítica.

Descripción: los usuarios solo podrán acceder a datos de los grupos a los que pertenecen. Esta separación deberá aplicarse en listados, mapas, métricas, campañas y observaciones.

RNF-003 – Offline-first

Prioridad: crítica.

Descripción: las operaciones principales de captura en campo no deberán depender de conectividad.

RNF-004 – Fiabilidad de sincronización

Prioridad: crítica.

Descripción: la sincronización deberá ser reintentable y no deberá perder datos ante cortes de red, cierres inesperados de la aplicación o sesiones pendientes.

RNF-005 – Usabilidad en campo

Prioridad: alta.

Descripción: la aplicación móvil deberá facilitar capturas rápidas, revisión de sesiones y comprensión clara del estado pendiente, activo, sincronizado o con error.

RNF-006 – Operabilidad

Prioridad: alta.

Descripción: el sistema deberá acompañarse de documentación suficiente para que una persona con perfil técnico pueda instalar, desplegar, configurar y administrar la plataforma de forma básica.

RNF-007 – Mantenibilidad

Prioridad: media.

Descripción: el diseño deberá facilitar futuras modificaciones de categorías, taxonomías o metadatos, aunque dichas modificaciones quedan fuera del alcance de mantenimiento definido para el proyecto.

4.5. Requisitos de datos científicos consolidados

Al tratarse de una herramienta para captura y análisis de datos científicos, OmniLitter debe preservar suficiente información contextual para que las observaciones puedan interpretarse y compararse posteriormente.

RDC-001 – Taxonomía jerárquica con códigos estables

Prioridad: crítica.

Descripción: la taxonomía de residuos deberá presentarse de forma jerárquica y conservar el código identificador de cada tipo de residuo, de acuerdo con la necesidad de utilizar listas comunes de categorías para favorecer la comparabilidad entre estudios [9].

RDC-002 – Metadatos de muestreo

Prioridad: crítica.

Descripción: las sesiones deberán almacenar metadatos de muestreo suficientes para interpretar los datos recopilados, incluyendo entorno, protocolo, método de muestreo, unidad o área observada, fecha, duración y localización cuando proceda.

RDC-003 – Metadatos de fotografías

Prioridad: media.

Descripción: las fotografías deberán conservar metadatos relevantes cuando estén disponibles, como fecha de captura, dimensiones, formato o coordenadas EXIF.

RDC-004 – Exportación científica

Prioridad: media.

Descripción: los datos exportados deberán incluir campos suficientes para análisis posterior y, cuando sea posible, un diccionario de columnas, unidades, metadatos de muestreo y versión de taxonomía utilizada.

RDC-005 – Versionado de taxonomía

Prioridad: alta.

Descripción: la versión de taxonomía utilizada deberá quedar asociada a sesiones y observaciones para garantizar reproducibilidad.

RDC-006 – Evolución futura de categorías y metadatos

Prioridad: media.

Descripción: las futuras modificaciones de categorías, taxonomías o metadatos deberán documentarse y versionarse para no comprometer la comparabilidad de los datos históricos.

4.6. Trazabilidad final

La trazabilidad permite justificar el origen de los requisitos consolidados y relacionar cada decisión con las fuentes empleadas durante el proyecto.

Trazabilidad desde la V1

El registro inicial de requisitos permitió identificar las necesidades base del sistema: registro de residuos, GPS, fotografías, fecha y hora, funcionamiento offline, sincronización, usuarios, grupos, roles, mapa, estadísticas, privacidad, seguridad, compatibilidad, taxonomía y exportación. Estos requisitos se transformaron posteriormente en los bloques funcionales de captura de campo, gestión de usuarios y grupos, consulta en portal, sincronización y requisitos científicos.

Trazabilidad desde la V2

La validación mediante prototipos permitió refinar los requisitos iniciales y organizarlos según los flujos reales del sistema. De esta sesión derivan la selección de grupo activo, la organización por sesiones de muestreo, la revisión de sesión, la cola de sincronización, los listados con filtros, el dashboard y la separación formal entre requisitos funcionales, no funcionales y de datos científicos.

Trazabilidad desde la V3

La última reunión permitió consolidar el flujo offline y de sincronización. De esta sesión derivan de forma explícita el estado de sincronización de sesiones y observaciones, la persistencia local, la sincronización por lotes, la consulta de cambios remotos, el refinamiento de la vista de mapa con filtros y la aclaración de la documentación técnica y del mantenimiento posterior fuera del alcance del proyecto.

Relación entre fuentes, cambios y requisitos finales

- La necesidad inicial de registrar residuos se consolida en RF-007, RF-008, RF-010, RF-012, RF-013, RF-014, RF-015, RF-016, RF-017, RF-020, RF-026, RDC-001 y RDC-002.
- La necesidad inicial de trabajar sin conexión se consolida en RF-021, RF-022, RF-023, RF-024, RF-025, RNF-003 y RNF-004.
- La necesidad inicial de gestionar usuarios, grupos y privacidad se consolida en RF-001, RF-002, RF-004, RF-005, RF-006, RF-032, RF-033, RNF-001 y RNF-002.

- La necesidad inicial de consultar y analizar datos se consolida en RF-027, RF-028, RF-030, RF-031 y RDC-004.
- La necesidad de preservar valor científico se consolida en RDC-001, RDC-002, RDC-003, RDC-004, RDC-005 y RDC-006.
- La revisión científica aportada por el cliente y las fuentes consultadas refuerzan RF-012, RF-013, RF-014, RF-031, RDC-001, RDC-002, RDC-004 y RDC-005, al vincular la aplicación con la toma de datos armonizada, el uso de listas comunes de categorías, la captura de metadatos y la generación de datos comparables para evaluar tendencias y medidas de mitigación.
- Las dudas planteadas en la última reunión sobre despliegue y mantenimiento se consolidan como requisitos no funcionales de operabilidad y mantenibilidad, principalmente en RNF-006 y RNF-007.

En conclusión, la versión final de requisitos mantiene el objetivo principal de OmniLitter: desarrollar una plataforma formada por aplicación móvil, portal web y backend centralizado para la captura, gestión y análisis de datos de residuos. La evolución respecto a las versiones iniciales se centra en precisar el flujo offline y de sincronización, reforzar el estado de las sesiones, completar la trazabilidad científica y aclarar el alcance operativo posterior a la entrega.

5. Planificación y gestión

5.1. Metodología de trabajo

5.1.1. Adaptación de Scrum

Para la organización del trabajo se tomó Scrum como referencia metodológica, pero se aplicó de forma adaptada a las condiciones reales del proyecto. OmniLitter no se desarrolló dentro de un equipo Scrum completo, sino como un trabajo académico realizado por dos personas, en coordinación con un cliente externo y bajo la supervisión del tutor. Por tanto, la metodología se utilizó como marco de trabajo iterativo e incremental, no como una implantación estricta de todos sus roles, eventos y artefactos.

La principal adaptación afectó a los roles. El cliente asumió una función próxima a la de *Product Owner*, ya que aportó la visión de dominio, explicó las necesidades científicas del sistema, validó los prototipos y ayudó a priorizar las funcionalidades con mayor valor para el uso real de la plataforma. Esta figura fue especialmente importante en aspectos como la captura de datos en campo, la organización por sesiones de muestreo, la taxonomía de residuos, el funcionamiento sin conexión y la utilidad posterior de los datos exportados. Sin embargo, al tratarse de un proyecto académico, la responsabilidad final de planificación, desarrollo y documentación permaneció en el equipo de trabajo.

No existió una figura formal de *Scrum Master*. Las tareas habituales de seguimiento, eliminación de bloqueos y coordinación se repartieron entre los dos integrantes del equipo. Esta decisión resultó coherente con el tamaño del proyecto: introducir un rol específico de facilitación habría sido poco realista para un equipo tan reducido. En su lugar, se mantuvo una coordinación directa y continua, centrada en revisar avances, detectar dependencias entre backend, portal web y aplicación móvil, y ajustar el reparto de tareas según la carga de cada momento.

La planificación se estructuró en sprints, entendidos como bloques de trabajo con objetivos funcionales concretos. No obstante, su duración no se fijó de manera completamente rígida. El calendario académico, la disponibilidad de los miembros del equipo, las reuniones con cliente y tutor, y la complejidad variable de algunas tareas obligaron a adaptar la duración efectiva de cada iteración. Por este motivo, los sprints se utilizaron principalmente para ordenar el avance incremental del producto: primero la base funcional del sistema,

después la captura de campo y el análisis inicial, posteriormente la sincronización, mapas y exportación, y finalmente el cierre funcional y refinamiento.

También se adaptaron los eventos de Scrum. El *Daily Scrum* no se realizó como una reunión diaria formal, ya que el equipo estaba formado únicamente por dos personas. En la práctica, se sustituyó por una coordinación constante mediante comunicación directa, revisión frecuente del estado de las tareas y resolución rápida de dudas técnicas. Este formato permitió mantener visibilidad sobre el progreso sin introducir una ceremonia artificial que no aportaba valor adicional al tamaño del equipo.

Las revisiones de sprint se realizaron cuando fue posible con la participación del cliente y del tutor. Estas sesiones permitieron contrastar incrementos funcionales, validar decisiones de diseño y ajustar el backlog a partir de la retroalimentación recibida. Cuando no fue viable realizar una revisión formal al cierre exacto de una iteración, la validación se produjo mediante reuniones de seguimiento, revisión de prototipos, comprobación de funcionalidades implementadas o discusión de cambios en los requisitos. De esta forma, se conservó el principio de inspección y adaptación propio de Scrum, aunque con una ejecución flexible.

En conjunto, la adaptación de Scrum permitió mantener una forma de trabajo ordenada, incremental y revisable, adecuada tanto al contexto académico como a la existencia de un cliente real. La metodología ayudó a dividir el desarrollo en entregas parciales, priorizar funcionalidades según su valor, incorporar cambios de requisitos sin romper la planificación general y conservar trazabilidad entre necesidades, backlog, implementación y validación.

5.2. Planificación Inicial

La planificación inicial de OmniLitter tuvo como objetivo transformar los requisitos obtenidos con el cliente en un conjunto de bloques funcionales ordenados, asumibles por un equipo de dos personas y coherentes con el calendario académico. Desde el inicio se identificó que el proyecto no consistía en una única aplicación aislada, sino en una plataforma formada por tres piezas principales: backend, portal web y aplicación móvil. Por ello, la planificación se centró en reducir riesgos de integración y en construir primero las capacidades sobre las que dependían las funcionalidades posteriores.

La planificación no se entendió como un documento cerrado, sino como una línea de trabajo inicial revisable. Las reuniones con el cliente, la validación de prototipos, las restricciones técnicas del entorno móvil y la evolución del alcance hicieron necesario ajustar prioridades durante el desarrollo. Aun así, se mantuvo una estructura común: identificar los bloques principales, ordenar sus dependencias, repartir el trabajo por sprints y revisar los riesgos más relevantes en cada etapa.

5.2.1. Identificación de bloques funcionales

El primer paso consistió en agrupar los requisitos en bloques funcionales. Esta agrupación permitió evitar una planificación basada únicamente en tareas pequeñas y facilitó razonar sobre qué partes del sistema aportaban valor de forma conjunta. Los bloques identificados fueron los siguientes:

- **Autenticación, usuarios y permisos.** Incluye registro, inicio de sesión, cierre de sesión, roles, grupos de trabajo y control de acceso. Este bloque era necesario para poder asociar datos a usuarios reales y separar la información entre grupos.
- **Gestión de campañas y sesiones de muestreo.** Define el contexto organizativo y científico de la captura de datos. Las campañas agrupan el trabajo de campo y las sesiones representan actividades concretas de recogida.
- **Captura de observaciones en aplicación móvil.** Comprende el registro de residuos, selección de taxonomía, cantidades, notas, coordenadas y fotografías. Es el bloque central desde la perspectiva del usuario de campo.
- **Persistencia local y sincronización.** Permite trabajar sin conexión, almacenar datos pendientes y sincronizarlos posteriormente con el backend. Se identificó desde el inicio como una de las áreas de mayor complejidad técnica.
- **Consulta, mapa, analítica y exportación.** Agrupa las funcionalidades del portal web orientadas a revisar, interpretar y reutilizar los datos recogidos: listados, filtros, visualización geográfica, métricas y exportaciones.
- **Administración, configuración y despliegue.** Incluye la gestión operativa de usuarios, grupos, catálogo científico, preferencias, documentación técnica y preparación del sistema para su ejecución en entornos reales.

Esta división ayudó a mantener la trazabilidad entre requisitos, backlog, implementación y pruebas. Además, permitió distinguir entre funcionalidades fundacionales, como autenticación y permisos, y funcionalidades de valor final, como sincronización, mapas o exportación científica.

5.2.2. Dependencias entre módulos

La planificación inicial tuvo en cuenta que muchos módulos no podían desarrollarse de forma completamente independiente. El backend actuaba como capa común de datos y reglas de negocio, mientras que el portal web y la aplicación móvil consumían sus servicios desde perspectivas distintas. Por este motivo, se priorizó construir primero los contratos, modelos y permisos compartidos antes de abordar flujos más avanzados.

Cuadro 5.1: Dependencias principales consideradas en la planificación inicial

Módulo o bloque	Dependencias principales	Justificación de planificación
Autenticación y permisos	Ninguna dependencia funcional previa relevante.	Debía implementarse al inicio para proteger rutas, asociar datos a usuarios y aplicar visibilidad por grupos.
Grupos y membresías	Autenticación.	Los grupos determinan el alcance de campañas, sesiones, observaciones, métricas y administración.
Campañas y sesiones	Autenticación, grupos y permisos.	Cada sesión necesita un contexto de grupo, campaña, usuario observador y reglas de acceso.
Observaciones y fotografías	Sesiones, taxonomía, permisos y almacenamiento.	Las observaciones solo tienen sentido dentro de una sesión y deben conservar trazabilidad científica.
Persistencia offline	Modelos de campañas, sesiones, observaciones y usuario autenticado.	La copia local debía reflejar el modelo del backend para poder sincronizar sin pérdida de información.
Sincronización	Persistencia offline, API de sesiones y observaciones, estados locales y remotos.	Era necesario disponer antes de captura local y contratos estables para resolver el envío de cambios pendientes.
Portal de consulta y análisis	Observaciones sincronizadas, permisos y filtros por grupo.	El portal depende de datos ya consolidados para listados, métricas, mapa y exportaciones.
Despliegue y operación	Backend, web, configuración de entorno y build móvil.	La puesta en marcha requiere que los módulos principales estén integrados y documentados.

Estas dependencias condicionaron el orden de los sprints. En particular, no era conveniente abordar la sincronización antes de disponer de captura local, ni desarrollar el análisis web antes de contar con observaciones suficientemente estructuradas. Del mismo modo, los permisos por grupo debían estar presentes desde las primeras iteraciones, ya que afectaban transversalmente a casi todos los módulos.

5.2.3. División inicial en sprints

La planificación se organizó en cuatro sprints principales. Cada sprint se definió como un incremento con un objetivo funcional reconocible, evitando que las iteraciones fueran simples agrupaciones técnicas sin valor observable. La duración real de cada sprint se ajustó a la disponibilidad del equipo y al calendario académico, pero la secuencia prevista se mantuvo como guía general del desarrollo.

Cuadro 5.2: División inicial del trabajo por sprints

Sprint	Objetivo principal	Bloques funcionales previstos
Sprint 1	Base funcional del sistema.	Autenticación, usuarios, grupos, permisos, campañas, sesiones iniciales, taxonomía y primeras observaciones.
Sprint 2	Captura de campo y análisis inicial.	Mejora de sesiones y observaciones, GPS, fotografías, validaciones, persistencia local inicial y consulta web de datos capturados.
Sprint 3	Sincronización, mapas y exportación científica.	Cola de sincronización, publicación de datos offline, visualización geográfica, métricas, exportación CSV/JSON/GeoJSON y preferencias de uso.
Sprint 4	Cierre funcional y refinamiento.	Administración, ajustes de campañas y sesiones, catálogo científico, correcciones finales, pruebas regresivas, documentación y preparación de despliegue.

Esta división permitió construir el sistema de forma incremental. El primer sprint dejaba una arquitectura mínima conectada; el segundo acercaba la aplicación móvil al trabajo real de campo; el tercero cerraba el ciclo entre captura offline, sincronización y explotación de datos; y el cuarto estabilizaba la plataforma para su entrega y validación final.

5.2.4. Riesgos identificados

Durante la planificación inicial se identificaron varios riesgos que podían afectar al alcance, al calendario o a la calidad del producto. Estos riesgos se revisaron durante el desarrollo y explican algunas decisiones de priorización, como construir primero una base técnica estable, validar pronto el flujo móvil o reservar margen para integración y despliegue.

Cuadro 5.3: Riesgos principales identificados en la planificación inicial

Riesgo	Impacto	Medidas de mitigación
Cambios de requisitos del cliente	Alto	Mantener un backlog revisable, validar prototipos e incrementos funcionales y documentar la evolución de requisitos.
Complejidad de sincronización offline	Alto	Desarrollar primero persistencia local, separar estados de sincronización, probar recorridos manuales completos y limitar el alcance a los flujos principales.
Coordinación entre portal web y aplicación móvil	Alto	Centralizar reglas en el backend, compartir contratos y esquemas cuando fuese posible, y validar los flujos de extremo a extremo.
Curva de aprendizaje de Expo, React Native y emuladores	Medio	Reservar tiempo para configuración, pruebas en emulador y dispositivo real, y priorizar funcionalidades móviles críticas antes del cierre.
Problemas de despliegue e infraestructura	Medio	Documentar entornos, utilizar configuración reproducible, preparar despliegue progresivo de backend, web y build móvil, y dejar constancia de limitaciones operativas.
Sobrecarga por equipo reducido	Medio	Repartir tareas por módulos, mantener coordinación constante y priorizar funcionalidades imprescindibles frente a mejoras secundarias.

El riesgo más relevante desde el punto de vista técnico fue la sincronización *offline-first*, porque implicaba coordinar almacenamiento local, estados pendientes, API remota, reintentos y validación de datos. Desde el punto de vista de gestión, el riesgo principal fue la evolución de requisitos, propia de un proyecto con cliente real y dominio científico específico. La planificación trató ambos riesgos mediante iteraciones cortas, revisiones frecuentes y una separación clara entre el núcleo funcional imprescindible y las ampliaciones deseables.

5.3. Gestión y herramientas de seguimiento

5.3.1. Gestión de la comunicación

La comunicación del proyecto se organizó separando los canales según el nivel de formalidad, el tipo de interlocutor y la necesidad de trazabilidad. Esta separación evitó mezclar decisiones funcionales con coordinación operativa diaria, y permitió mantener un flujo de comunicación ligero pero suficientemente controlado.

Con el cliente y el tutor se utilizó Microsoft Teams como canal principal de comunicación síncrona. A través de esta herramienta se realizaron reuniones de seguimiento, revisiones de avance, aclaración de requisitos, validación de decisiones funcionales y resolución rápida.

da de dudas. Teams funcionó, por tanto, como espacio de coordinación externa y como mecanismo de revisión incremental del producto.

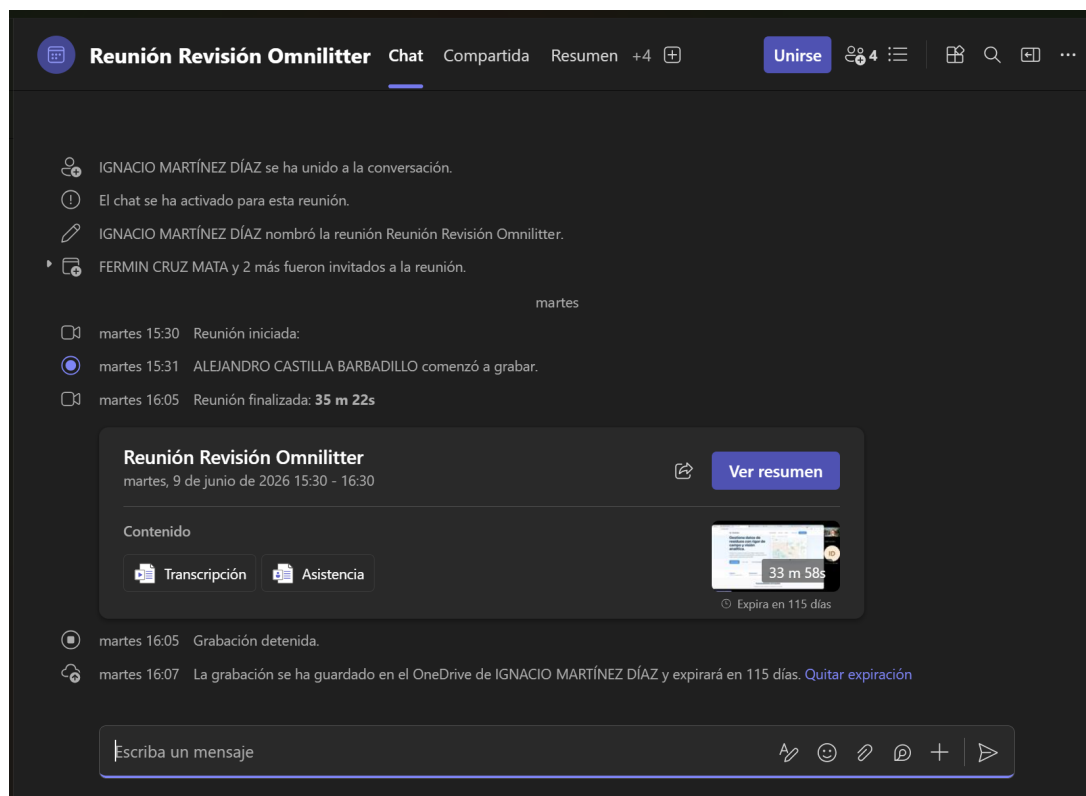


Figura 5.1: Uso de Microsoft Teams para reuniones de seguimiento y validación.

Outlook se reservó para comunicaciones con mayor valor documental: convocatorias, confirmaciones, envío de documentos y acuerdos relevantes. De este modo, Teams asumió el papel de canal ágil de interacción y Outlook el de canal formal de comunicación y archivo.

Entre los dos integrantes del equipo se emplearon canales informales, principalmente WhatsApp, para coordinación diaria, desbloqueo de tareas, sincronización de disponibilidad y revisión rápida de incidencias. Este esquema se aproxima a una comunicación de baja ceremonia, adecuada para equipos reducidos: menos carga administrativa, más frecuencia de contacto y mayor capacidad de reacción ante cambios de prioridad o problemas técnicos.

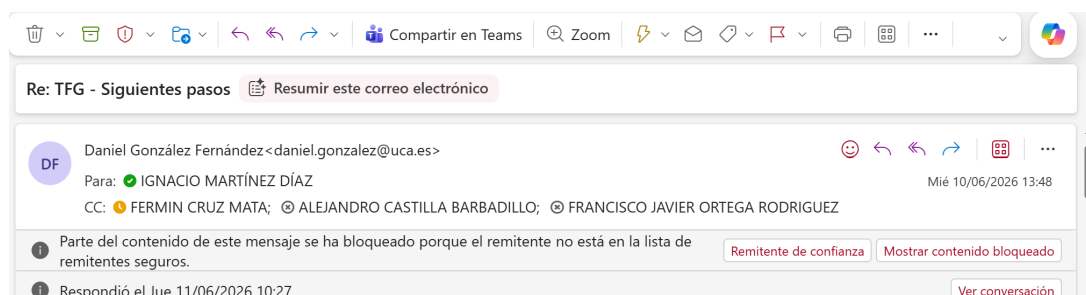


Figura 5.2: Uso de Outlook para comunicaciones formales y trazabilidad documental.

5.3.2. Gestión del backlog

La gestión del backlog se realizó mediante GitHub Projects, utilizando un tablero Kanban asociado al repositorio del proyecto. El tablero actuó como radiador de información del desarrollo: permitía visualizar el flujo de trabajo, identificar tareas pendientes, trabajo en curso, elementos bloqueados y tareas completadas. De esta forma, el backlog no se limitó a una lista de funcionalidades, sino que se convirtió en una herramienta de seguimiento operativo por sprint.

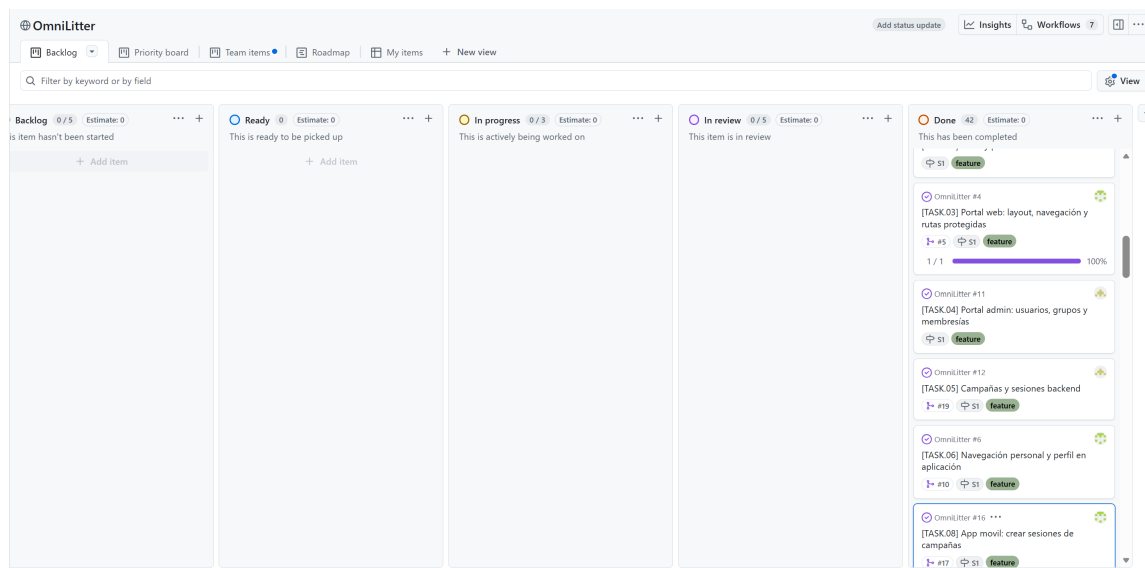


Figura 5.3: Tablero Kanban utilizado para visualizar el flujo de trabajo del backlog.

Cada elemento del backlog se registró como una issue de GitHub para mantener trazabilidad entre planificación e implementación. Para homogeneizar la información se definió una plantilla específica de tareas, con campos de resumen, módulo afectado, tipo de tarea, prioridad, sprint, estimación, rama sugerida, dependencias o bloqueos y Definition of Done. Esta estructura permitió aplicar una técnica de refinamiento ligero del backlog: cada tarea debía estar suficientemente descrita para poder implementarse, revisarse y cerrarse sin ambigüedad.

← Create new issue in Omnilitter-TFG/OmniLitter: ★ Tarea ×

Add a title *

[TASK.XX]

★ Nueva tarea

Completa esta plantilla para que la tarea sea clara, trazable y fácil de revisar.

📄 Resumen *

Describe la tarea en una frase corta.

Ej: Implementar endpoint de login con JWT

🌱 Módulo afectado *

None ▾

🔧 Tipo de tarea *

Feature ▾

🔥 Prioridad *

Crítica — Bloquea el camino crítico ▾

🏆 Sprint *

S1 — Base funcional (25-29 mayo) ▾

🕒 Estimación *

XS — Menos de 1 hora ▾

🌿 Rama sugerida

☐ Create more Cancel Create ^ ↗

Figura 5.4: Plantilla de tarea utilizada para homogeneizar el backlog.

Además, se utilizó una identificación organizada de tareas y ramas mediante patrones como `feature/tXX-descripcion` o `fix/bugXX-descripcion`. Con ello se obtuvo trazabilidad horizontal entre issue, rama de trabajo, Pull Request, commit funcional, sprint y release. Esta relación resultó útil para reconstruir la evolución del producto y justificar qué incremento incorporaba cada funcionalidad.

5.3.3. Gestión documental

La gestión documental combinó varias herramientas según la fase del proyecto. Al inicio, la memoria se redactó en Overleaf, lo que facilitaba la edición colaborativa en LaTeX y la revisión rápida del documento. Sin embargo, a medida que la memoria creció en tamaño, número de capítulos, tablas, figuras y anexos, aparecieron limitaciones de compilación. Por este motivo, la documentación principal se trasladó posteriormente a un repositorio de GitHub.

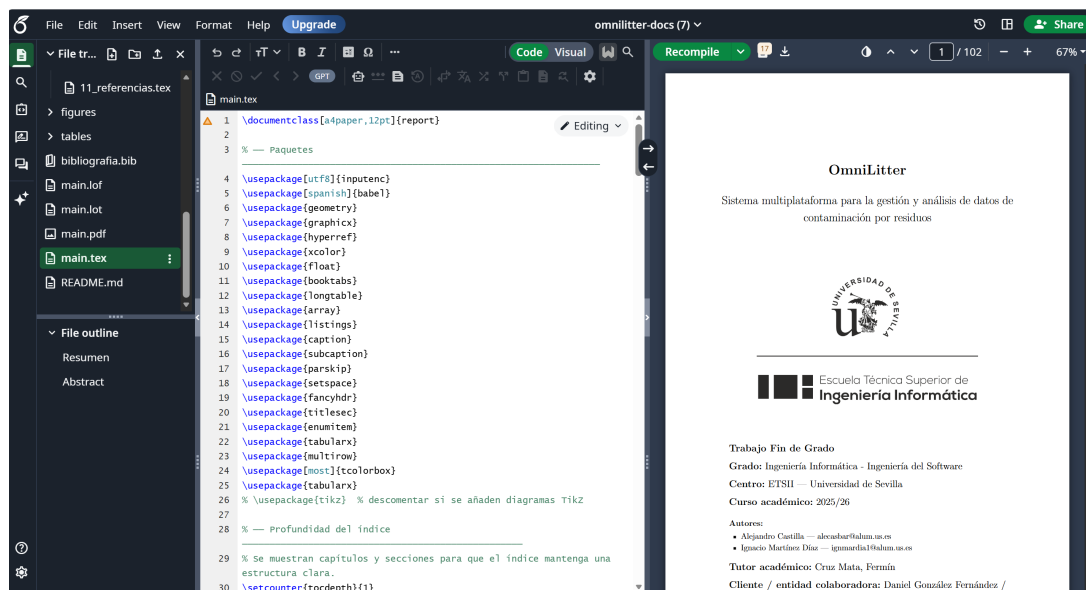


Figura 5.5: Uso inicial de Overleaf para la redacción colaborativa de la memoria.

GitHub se utilizó como fuente versionada de la memoria, manteniendo control sobre los ficheros `.tex`, la bibliografía, las figuras, los anexos y el PDF generado. Este cambio convirtió la memoria en un artefacto de configuración del proyecto, con un flujo similar al del código fuente: cambios incrementales, historial recuperable y posibilidad de revisar diferencias entre versiones. La documentación quedó así alineada con la evolución real del sistema y no como un entregable aislado al final del desarrollo.

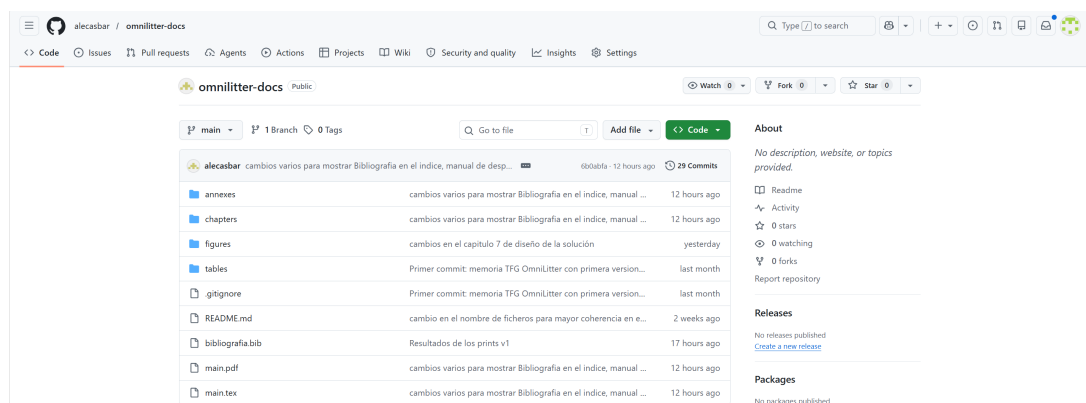


Figura 5.6: Repositorio de documentación utilizado como fuente versionada de la memoria.

Además, se creó una carpeta compartida en OneDrive para conservar documentos iniciales de planificación y trabajo, elaborados principalmente con Microsoft Word. Esta carpeta sirvió como repositorio documental auxiliar para borradores, planificaciones iniciales, notas de reunión y materiales que todavía no formaban parte de la memoria final en LaTeX.

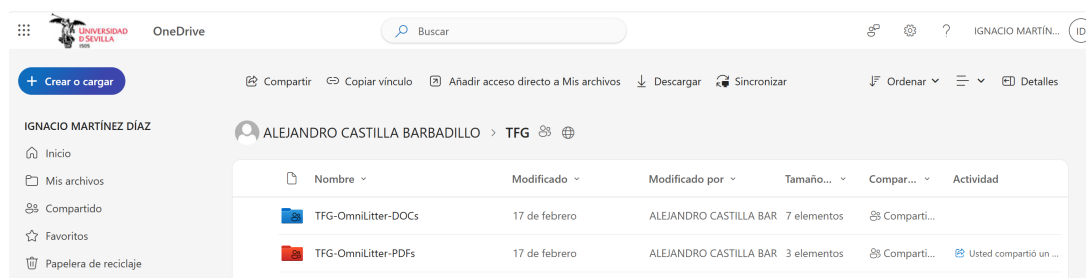


Figura 5.7: Repositorio documental auxiliar en OneDrive.

5.3.4. Gestión del código fuente

El código fuente de OmniLitter se gestionó mediante un repositorio centralizado en GitHub dentro de la organización **Omnilitter-TFG**. El repositorio principal, **OmniLitter**, agrupó las distintas partes del sistema: backend, portal web, aplicación móvil, paquete compartido, configuración de despliegue y documentación técnica auxiliar. Esta estructura responde a una estrategia de monorepo: todos los módulos evolucionan en un mismo repositorio, lo que facilita coordinar cambios transversales y mantener contratos compartidos entre API, web y móvil.

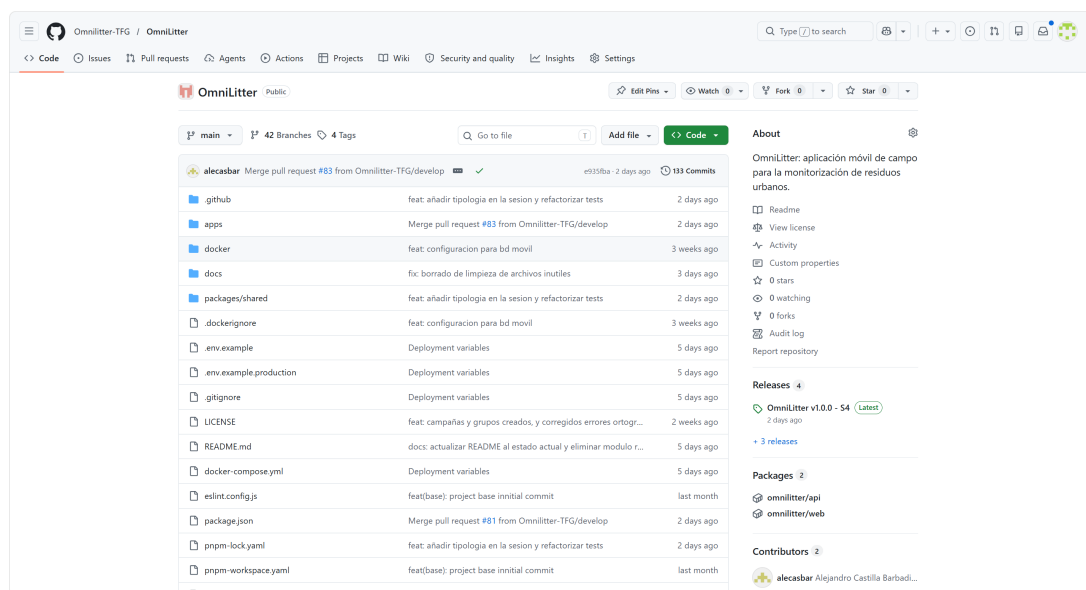


Figura 5.8: Repositorio centralizado utilizado para la gestión del código fuente.

Para el control de versiones se utilizó Git, sistema distribuido que modela el historial como una secuencia de instantáneas y permite trabajar con ramas locales y remotas. El flujo seguido puede entenderse como un GitFlow simplificado: **main** actuó como rama estable, **develop** como rama de integración y las ramas **feature/** y **fix/** como ramas temporales para nuevas funcionalidades y correcciones. Las integraciones se realizaron mediante Pull Requests hacia **develop** y, al consolidar un incremento, hacia **main**.

Los Pull Requests actuaron como mecanismo de control de cambios. Antes de integrar una rama se revisaba su alcance, se comprobaba su relación con la tarea del backlog, se validaba que no rompiera funcionalidades existentes y se dejaba rastro del merge en el historial. En el repositorio se observa este patrón mediante merges desde ramas **feature/** y **fix/** asociadas a tareas e incidencias concretas.

También se aplicó una convención de mensajes próxima a Conventional Commits, cuyo objetivo es aportar significado legible por personas y herramientas al historial de commits [?]. Se utilizaron prefijos como **feat:**, **fix:**, **test:**, **docs:** o **ci:**, lo que permitió distinguir nuevas funcionalidades, correcciones, pruebas, documentación y cambios de integración continua. Esta técnica mejora la trazabilidad del historial y facilita relacionar commits, tareas, incidencias y releases.

5.3.5. Gestión de versiones y releases

La evolución del producto se documentó mediante versiones asociadas a incrementos del desarrollo. Se siguió una estrategia inspirada en el versionado semántico, que utiliza la forma **MAJOR.MINOR.PATCH** para comunicar el tipo de cambio incorporado [?]. En el proyecto se generaron releases intermedias por sprint, utilizando etiquetas como **v0.1.0**, **v0.2.0** y **v0.3.0**, y una versión final **v.1.0.0**, correspondiente al cierre funcional del proyecto.

Las etiquetas Git funcionaron como puntos de control o baselines del producto. Cada release representaba un estado estable del repositorio, asociado a un conjunto de funcionalidades integradas y validables. Esta estrategia facilitó reconstruir la evolución de OmniLitter por incrementos y separar versiones de desarrollo intermedias de la versión final entregable.

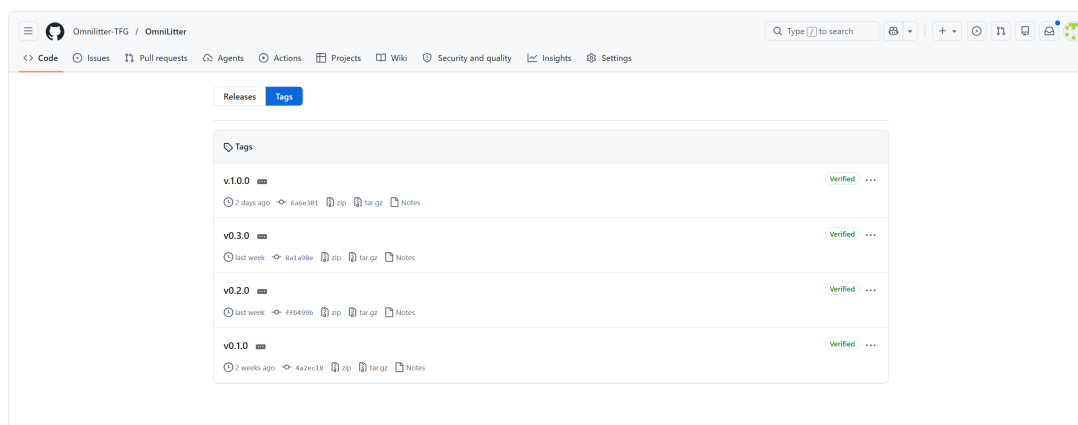


Figura 5.9: Releases publicadas mediante GitHub.

Las releases se gestionaron con herramientas de GitHub. El repositorio incluye un workflow de GitHub Actions que crea una release al publicar una etiqueta que coincida con el patrón **v***. GitHub Actions permite automatizar flujos de CI/CD dentro del repositorio mediante workflows definidos en YAML [?]. En este caso, la automatización generaba notas básicas

a partir de commits y Pull Requests. Además, se incorporaron otros flujos de integración y despliegue, como validación continua, construcción de imágenes Docker y generación manual de una build Android de previsualización con Expo/EAS.

Cuadro 5.4: Versiones principales publicadas durante el desarrollo

Versión	Momento	Finalidad
v0.1.0	Release intermedia	Marcar una primera base funcional integrada del sistema.
v0.2.0	Release intermedia	Consolidar avances de captura de campo, persistencia inicial y módulos web asociados.
v0.3.0	Release intermedia	Recoger funcionalidades avanzadas de sincronización, mapas, análisis y exportación.
v.1.0.0	Release final	Señalar la versión final del proyecto tras el cierre funcional, refinamiento y validación.

5.3.6. Gestión de incidencias

La gestión de incidencias se integró en el mismo sistema de seguimiento utilizado para el backlog. Los errores detectados durante el desarrollo, las pruebas manuales o la revisión de incrementos se registraron en GitHub Issues y se visualizaron en el tablero Kanban de GitHub Projects. De esta forma, las incidencias se sometían al mismo flujo de trabajo que las tareas funcionales: registro, priorización, asignación, resolución, revisión y cierre.

Para homogeneizar el reporte de errores se creó una plantilla específica de bug en el repositorio. Esta plantilla recogía descripción del problema, pasos para reproducirlo, comportamiento esperado, comportamiento actual, módulo afectado, sprint y severidad. Estos campos permitían realizar un triage básico de incidencias, diferenciando errores críticos, altos, medios o bajos según su impacto en la demo, en una funcionalidad principal o en aspectos visuales menores.

La plantilla también incluía una Definition of Done orientada a confirmar que el error había sido corregido, probado localmente, revisado por el compañero y fusionado en `develop`. El repositorio conserva ramas de corrección siguiendo el patrón `fix/bugXX-descripcion`, lo que muestra la relación entre incidencia, rama, Pull Request y validación regresiva.

← Create new issue in Omnilitter-TFG/OmniLitter: 🐛 Bug ×

Add a title *

[BUG]

🐛

Reporte de bug

Describe el problema con el mayor detalle posible.

📖 Descripción del bug *

Write

Preview

H B I ≡ <> 🔗 ≡ ≡ ≡ @ ↗ 📎 ↶

Al intentar...

📎 Paste, drop, or click to add files

📋 Pasos para reproducirlo *

Write

Preview

H B I ≡ <> 🔗 ≡ ≡ ≡ @ ↗ 📎 ↶

1.

2.

3.

📎 Paste, drop, or click to add files

☐ Create more

Cancel

Create ^ 📎

Figura 5.10: Plantilla y seguimiento de incidencias en GitHub.

49

6. Tecnologías utilizadas

En este capítulo se describen las tecnologías seleccionadas para el desarrollo de OmniLitter y la justificación de su elección. La selección se ha realizado teniendo en cuenta tres restricciones principales: el alcance de un Trabajo de Fin de Grado desarrollado por dos personas, la necesidad de disponer de una aplicación móvil capaz de trabajar sin conexión y la conveniencia de mantener un stack homogéneo basado en TypeScript.

6.1. Criterios de selección

Los criterios utilizados para seleccionar las tecnologías han sido los siguientes:

- **Homogeneidad tecnológica:** se prioriza el uso de TypeScript tanto en el frontend como en el backend para reducir el cambio de contexto durante el desarrollo.
- **Capacidad offline:** la aplicación móvil debe poder crear sesiones, observaciones y fotografías sin depender de la conexión a Internet.
- **Madurez del ecosistema:** se eligen tecnologías ampliamente usadas, con documentación suficiente y comunidad activa.
- **Facilidad de despliegue:** el sistema debe poder desplegarse en servicios accesibles para un proyecto académico, como Render o Microsoft Azure, a partir de imágenes de contenedor.
- **Escalabilidad razonable:** aunque el proyecto no persigue una escala industrial, el diseño debe permitir incorporar nuevos grupos, campañas y datos de campo sin rehacer la arquitectura.

Tecnologías empleadas en los prototipos Durante la fase inicial se construyeron prototipos funcionales con datos en memoria: un portal web y una simulación de la aplicación móvil en navegador. Estos prototipos permitieron validar flujos de usuario con el cliente antes de trasladar las decisiones principales a la implementación final.

Cuadro 6.1: Tecnologías empleadas en los prototipos funcionales.

Tecnología	Uso en el proyecto
React 18	Construcción de interfaces de usuario para el portal web y el prototipo móvil.
TypeScript	Tipado estático de componentes, rutas, estado y datos de ejemplo.
Vite	Servidor de desarrollo y empaquetado rápido de los prototipos.
Tailwind CSS	Sistema de estilos utilitario para construir pantallas responsive.
React Router	Definición de rutas públicas, rutas protegidas y navegación entre pantallas.
Recharts	Gráficas y visualizaciones estadísticas del dashboard del portal web.
Lucide React / Material Icons	Iconografía de la interfaz.

6.2. Portal web

El portal web se plantea como una aplicación *Single Page Application* desarrollada con React, TypeScript y Vite. Su responsabilidad principal es permitir a investigadores y administradores consultar observaciones, gestionar usuarios y grupos, visualizar mapas y exportar datos.

Cuadro 6.2: Stack seleccionado para el portal web.

Tecnología	Responsabilidad
React 18	Construcción de componentes reutilizables y pantallas del portal.
TypeScript	Definición de tipos de dominio, respuestas de API y props de componentes.
Vite	Compilación, servidor de desarrollo y generación del bundle final.
Tailwind CSS	Sistema visual responsive y consistente con los prototipos.
Enrutado propio (History API)	Gestión de rutas públicas y protegidas en el cliente, sin librería de enrutado externa.
Fetch API	Comunicación con la API REST del backend.
Leaflet y React Leaflet	Mapa interactivo de observaciones con agrupación de marcadores (<i>clustering</i>) y capa de densidad (<i>heatmap</i>).
Recharts	Gráficos del dashboard y de la analítica.
i18next / react-i18next	Interfaz en español e inglés.

6.3. Aplicación móvil

La aplicación móvil se plantea con React Native y Expo, utilizando SQLite mediante el paquete `expo-sqlite` como base de datos local. El objetivo es que el usuario pueda completar una jornada de muestreo aunque el dispositivo no tenga conexión a Internet.

Cuadro 6.3: Stack seleccionado para la aplicación móvil.

Tecnología	Responsabilidad
React Native	Desarrollo multiplataforma para Android e iOS.
Expo (<i>bare workflow</i> , <code>expo-dev-client</code>)	Toolchain de desarrollo y compilación con acceso a módulos nativos.
TypeScript	Tipado de pantallas, servicios, modelos locales y respuestas de API.
<code>expo-sqlite</code> / SQLite	Persistencia local de sesiones, observaciones, residuos, taxonomía y cola de sincronización.
<code>expo-location</code>	Captura de coordenadas GPS asociadas a sesiones y observaciones.
<code>expo-camera</code> / <code>expo-image-picker</code>	Captura de fotografías con la cámara y selección desde la galería.
<code>expo-file-system</code>	Almacenamiento local de las fotografías en el dispositivo.
<code>expo-secure-store</code>	Guardado seguro de credenciales y <i>token</i> para el inicio de sesión sin conexión.
<code>@react-native-community/netinfo</code>	Detección del estado de conectividad para disparar la sincronización al recuperar cobertura.
<code>react-native-webview</code>	Mapa de sesión embebido para mostrar el recorrido y el área muestreada.
<code>il8next</code> / <code>react-il8next</code>	Interfaz en español e inglés.
Navegación propia (estado de React)	Cambio entre las pestañas principales y los flujos de captura, sin librería de navegación externa.

6.4. Backend y persistencia

El backend centraliza la lógica de negocio, la autenticación, el control de acceso por grupo y la sincronización de datos procedentes de dispositivos móviles. Se plantea como una API REST desarrollada con Node.js, Express y TypeScript.

Cuadro 6.4: Stack seleccionado para backend y persistencia.

Tecnología	Responsabilidad
Node.js 20	Entorno de ejecución del backend.
Express	Definición de rutas, middlewares y controladores REST.
TypeScript	Tipado de servicios, entidades y errores.
PostgreSQL 16	Base de datos relacional principal del sistema.
Prisma	ORM para modelar entidades, generar migraciones y acceder a los datos.
Zod	Validación de esquemas de entrada, compartida entre cliente y servidor.
JWT (jsonwebtoken)	Autenticación mediante <i>tokens</i> de acceso y refresco.
bcryptjs	Almacenamiento de contraseñas mediante <i>hash</i> .
Nodemailer	Envío de correo para la recuperación de contraseña.
swagger-ui-express (OpenAPI)	Documentación interactiva de la API en <code>/api/docs</code> .
Helmet, CORS y Morgan	Cabeceras de seguridad, control de origen y registro de peticiones.
Almacenamiento de ficheros	Las fotografías llegan codificadas en <i>base64</i> y se guardan en el sistema de archivos con el módulo de ficheros de Node.

6.5. Despliegue y herramientas auxiliares

Además de las tecnologías propias de cada componente, el proyecto se apoya en un conjunto de herramientas de organización, calidad, pruebas y despliegue. La tabla 6.5 las recoge.

Cuadro 6.5: Herramientas auxiliares, pruebas y despliegue.

Tecnología	Responsabilidad
pnpm <i>workspaces</i>	Gestión del monorepo con los paquetes de API, portal web, móvil y código compartido.
TypeScript	Lenguaje común de todos los paquetes, con un paquete compartido de tipos y esquemas.
ESLint y Prettier	Análisis estático y formateo homogéneo del código.
Vitest	Pruebas automatizadas de la API, el portal web y el paquete compartido, con medición de cobertura.
Docker y Docker Compose	Empaquetado y orquestación de PostgreSQL, la API y el portal web.
Nginx	Servidor del portal web dentro de su contenedor.
Git, GitHub y GitHub Actions	Control de versiones e integración continua: validación, publicación de imágenes y generación de versiones.
EAS (Expo Application Services)	Compilación del APK de Android de la aplicación móvil.
Render y Microsoft Azure	Despliegue del sistema en los entornos de preproducción y producción.

6.6. Alternativas consideradas

Justificación de SQLite frente a WatermelonDB Aunque inicialmente se valoró WatermelonDB como solución local *offline-first*, se ha decidido emplear SQLite directamente para ajustar mejor el alcance del TFG. SQLite permite cubrir los requisitos principales de la aplicación móvil: almacenar sesiones, observaciones, fotografías pendientes y una cola de sincronización local.

La decisión reduce la complejidad técnica, evita depender de una capa adicional de sincronización y facilita que los autores controlen de forma explícita el esquema local y el flujo de subida de datos. El patrón *offline-first* se mantiene, pero se implementa mediante una tabla local de cola de sincronización y servicios propios de lectura/escritura sobre SQLite.

7. Costes

Este capítulo presenta una estimación económica del proyecto OmniLitter. Al tratarse de un Trabajo de Fin de Grado, una parte importante de los recursos no ha supuesto un desembolso directo, ya que se han utilizado equipos personales, servicios con planes gratuitos y herramientas disponibles para uso académico. No obstante, resulta útil valorar el coste equivalente del proyecto para comprender su dimensión real y justificar el esfuerzo necesario para construir una plataforma compuesta por backend, portal web, aplicación móvil, documentación y despliegue.

Las estimaciones se han realizado diferenciando entre costes de personal, infraestructura, herramientas software, equipamiento y otros costes operativos. En los casos en los que no existió un gasto directo, se indica como coste imputado o amortizado, ya que representa el valor económico aproximado de los recursos utilizados durante el desarrollo.

7.1. Costes de personal

El coste principal del proyecto corresponde al tiempo dedicado por los dos integrantes del equipo. Aunque no se trata de un coste salarial real, se ha estimado su valor económico utilizando una tarifa de referencia de **15 EUR/hora**, razonable para un perfil junior o en prácticas dentro de un proyecto académico de ingeniería del software.

La estimación de horas se basa en la duración del trabajo, la división en cuatro sprints principales y las actividades realizadas: análisis de requisitos, diseño, implementación de los tres módulos principales, integración, pruebas, despliegue y elaboración de la memoria. Se considera una dedicación aproximada de **300 horas por persona**, lo que da un total de **600 horas** de trabajo.

Cuadro 7.1: Estimación de costes de personal por actividad

Actividad	Horas estimadas	Coste estimado	Justificación
Análisis, planificación y reuniones	70 h	1.050 EUR	Incluye reuniones con cliente y tutor, adaptación de requisitos, planificación de sprints y gestión del backlog.
Diseño funcional y técnico	60 h	900 EUR	Comprende arquitectura, modelo de datos, contratos entre módulos, diseño de flujos y decisiones de interfaz.
Desarrollo backend	110 h	1.650 EUR	Implementación de API, autenticación, permisos, entidades, persistencia, servicios y lógica de negocio.
Desarrollo portal web	90 h	1.350 EUR	Construcción de pantallas de administración, consulta, mapas, métricas, exportación y consumo de API.
Desarrollo aplicación móvil	130 h	1.950 EUR	Incluye navegación, captura de datos, GPS, fotografías, persistencia local y funcionamiento offline.
Integración, pruebas y despliegue	80 h	1.200 EUR	Validación de flujos extremo a extremo, corrección de incidencias, configuración de entornos y releases.
Documentación y memoria	60 h	900 EUR	Redacción de memoria, anexos, revisión documental, evidencias, manuales y preparación de entrega.
Total	600 h	9.000 EUR	Coste imputado del esfuerzo de desarrollo del equipo.

Esta partida concentra la mayor parte del presupuesto porque OmniLitter es un proyecto intensivo en desarrollo software. La existencia de varios módulos conectados entre sí incrementó el esfuerzo de integración, especialmente en la sincronización *offline-first* y en la coordinación entre backend, web y móvil.

7.2. Costes de infraestructura y despliegue

La infraestructura utilizada durante el desarrollo se apoyó principalmente en servicios con planes gratuitos o recursos de bajo coste. GitHub se empleó para el repositorio, gestión de issues, tablero Kanban, releases y automatizaciones. Además, el despliegue se planteó

con servicios adecuados para un proyecto académico, como Vercel, Render, Railway o entornos equivalentes.

Durante el desarrollo, estos servicios no generaron un coste directo relevante. A efectos de estimación, se contempla un escenario de uso académico durante seis meses, con infraestructura en planes gratuitos y sin contratación de dominio propio.

Cuadro 7.2: Estimación de costes de infraestructura

Recurso	Periodo	Coste estimado	Justificación
Repositorio GitHub y GitHub Projects	6 meses	0 EUR	Uso de funcionalidades gratuitas para repositorio, issues, tablero Kanban, Pull Requests y releases.
GitHub Actions	6 meses	0 EUR	Uso dentro de los límites gratuitos disponibles para automatizaciones ligeras de CI/CD.
Hosting portal web	6 meses	0 EUR	Despliegue compatible con planes gratuitos de servicios como Vercel o equivalentes.
Backend y API	6 meses	0 EUR	Entorno de despliegue académico o gratuito, suficiente para validación y demostración.
Base de datos PostgreSQL	6 meses	0 EUR	Uso de instancia gratuita o entorno local durante desarrollo y pruebas.
Dominio propio	No contratado	0 EUR	No fue necesario adquirir un dominio para la entrega académica del proyecto.
Total	–	0 EUR	Sin gasto directo de infraestructura durante el desarrollo académico.

En un escenario de operación real, estos costes podrían aumentar por la contratación de una base de datos persistente, almacenamiento de fotografías, dominio propio, copias de seguridad y mayor capacidad de cómputo. Sin embargo, para el alcance del trabajo de fin de grado, los planes gratuitos fueron suficientes para validar la solución.

7.3. Costes de herramientas software

Las herramientas de desarrollo y documentación seleccionadas se eligieron también con criterios de coste. Se priorizaron tecnologías libres, servicios gratuitos o licencias disponibles para estudiantes. Esto permitió reducir el presupuesto sin limitar la calidad técnica del producto.

Cuadro 7.3: Estimación de costes de herramientas software

Herramienta	Coste estimado	Uso en el proyecto
Visual Studio Code	0 EUR	Entorno principal de desarrollo y edición de código.
Git y GitHub	0 EUR	Control de versiones, repositorio centralizado, issues, Pull Requests y releases.
Node.js, TypeScript, React, Express y Expo	0 EUR	Stack principal de desarrollo, basado en tecnologías de código abierto.
PostgreSQL y SQLite	0 EUR	Persistencia remota y local mediante motores gratuitos.
Overleaf	0 EUR	Redacción inicial de la memoria mediante plan gratuito.
LaTeX local	0 EUR	Compilación final de la memoria tras las limitaciones de Overleaf.
Microsoft Teams, Outlook, Word y One-Drive	0 EUR	Herramientas disponibles mediante cuentas académicas o personales para comunicación y documentación.
Total	0 EUR	No se contrataron licencias específicas para el proyecto.

La ausencia de coste de licencias fue posible porque el proyecto se apoyó en un ecosistema ampliamente disponible para desarrollo web y móvil. La principal limitación no fue económica, sino de aprendizaje, configuración y coordinación entre herramientas.

7.4. Costes de equipamiento

El desarrollo se realizó con equipos personales ya disponibles. Por ello, no hubo un gasto de adquisición específico. Aun así, para valorar económicamente el uso de estos recursos se aplica una estimación por amortización, considerando el tiempo de uso durante el proyecto frente a la vida útil aproximada de los dispositivos.

Se asume un periodo de proyecto de seis meses, una vida útil de cuatro años para los ordenadores portátiles y tres años para el dispositivo móvil utilizado en pruebas. Esta estimación no representa dinero desembolsado, sino el coste proporcional de uso del equipamiento.

Cuadro 7.4: Estimación de costes de equipamiento amortizado

Recurso		Unidades	Valor unitario	Coste imputado	Justificación
Ordenadores portátiles		2	900 EUR	225 EUR	Amortización de 6 meses sobre una vida útil estimada de 48 meses.
Dispositivo de pruebas	Android	1	300 EUR	50 EUR	Amortización de 6 meses sobre una vida útil estimada de 36 meses.
Conexión a Internet y electricidad		1	—	60 EUR	Estimación proporcional del consumo y conectividad durante el trabajo.
Total		—	—	335 EUR	Coste imputado de recursos físicos y operativos.

Estos costes son reducidos en comparación con el esfuerzo humano, ya que el proyecto no requirió hardware específico ni servidores propios. El elemento más relevante fue disponer de al menos un dispositivo móvil real para validar funcionalidades como GPS, cámara, almacenamiento local y sincronización.

7.5. Plan de contingencia

Además de los costes estimados, se contempla una reserva de contingencia para cubrir desviaciones razonables durante el desarrollo. En un proyecto de estas características, los principales factores de incertidumbre no proceden de la compra de licencias o infraestructura, sino del tiempo adicional necesario para resolver problemas técnicos, adaptar requisitos o estabilizar la integración entre módulos.

La contingencia se calcula como un **10 %** sobre el coste base estimado del proyecto, excluyendo únicamente costes no previstos que requerirían una decisión externa, como contratar un servicio de pago o adquirir nuevo equipamiento. Esta reserva se considera adecuada para un trabajo académico con alcance controlado, pero con riesgos técnicos relevantes asociados al funcionamiento offline, el despliegue y la coordinación entre backend, web y aplicación móvil.

Cuadro 7.5: Plan de contingencia del proyecto

Riesgo cubierto	Prob.	Reserva	Aplicación prevista
Cambios de requisitos del cliente	Media	300 EUR	Revisión de funcionalidades, ajustes de alcance, adaptación del backlog y pequeñas modificaciones de flujo.
Complejidad de sincronización offline	Alta	350 EUR	Tiempo adicional para depurar estados locales, reintentos, conflictos y validación de datos sincronizados.
Problemas de integración entre módulos	Media	150 EUR	Corrección de incompatibilidades entre backend, portal web, aplicación móvil y contratos compartidos.
Incidencias de despliegue o configuración	Media	85 EUR	Ajustes de variables de entorno, builds, servicios externos y documentación operativa.
Margen general de cierre	Baja	48,50 EUR	Correcciones menores, revisión final, pruebas regresivas y ajustes documentales de última hora.
Total contingencia	–	933,50 EUR	Reserva equivalente al 10 % del coste base del proyecto.

Este plan no implica que la contingencia deba consumirse obligatoriamente. Su función es reflejar una práctica de planificación prudente: reservar margen ante riesgos identificados, evitar que una desviación técnica afecte al cierre del proyecto y disponer de una valoración económica más realista si OmniLitter se plantease como un desarrollo profesional.

7.6. Costes globales del proyecto

El coste global se obtiene sumando las partidas anteriores y la reserva de contingencia definida. La mayor parte del importe corresponde al esfuerzo humano, mientras que infraestructura y licencias presentan coste directo nulo gracias al uso de herramientas gratuitas o ya disponibles.

Cuadro 7.6: Resumen global de costes del proyecto

Categoría de coste	Importe estimado
Costes de personal	9.000 EUR
Costes de infraestructura y despliegue	0 EUR
Costes de herramientas software	0 EUR
Costes de equipamiento y operación	335 EUR
Reserva de contingencia	933,50 EUR
Coste total estimado	10.268,50 EUR

Por tanto, el coste económico estimado de OmniLitter asciende a **10.268,50 EUR** incluyendo contingencia. Esta cifra debe interpretarse como una valoración del proyecto, no como un gasto real efectuado. El desembolso directo fue muy reducido, pero el coste imputado refleja mejor la magnitud del trabajo realizado y permite comparar el proyecto con un desarrollo software equivalente en un entorno profesional.

8. Diseño de la solución

Este capítulo describe las decisiones de diseño adoptadas para materializar los requisitos funcionales y no funcionales de OmniLitter en una arquitectura concreta. Se abordan, en orden de abstracción descendente, la arquitectura general del sistema, el diseño de la base de datos, el contrato de la API REST, la estructura de componentes del portal web y de la aplicación móvil, y el mecanismo de sincronización *offline*.

8.1. Arquitectura general

OmniLitter sigue una arquitectura de **tres capas** clásica adaptada a un contexto multi-plataforma con requisito de operación sin conexión:

1. **Capa de presentación.** Comprende dos clientes independientes: el portal web, orientado a la consulta y gestión de datos por parte de investigadores y administradores, y la aplicación móvil, orientada a la captura de observaciones en campo.
2. **Capa de lógica de negocio.** Una API REST centralizada planteada con Node.js + Express actúa como único punto de acceso a los datos, aplica las reglas de negocio y gestiona la autenticación mediante *tokens* JWT.
3. **Capa de persistencia.** PostgreSQL almacena el estado canónico del sistema. Adicionalmente, la aplicación móvil mantiene una base de datos local SQLite que permite operar sin conexión y sincronizarse en diferido.

La figura 8.1 muestra un diagrama de la arquitectura del sistema.

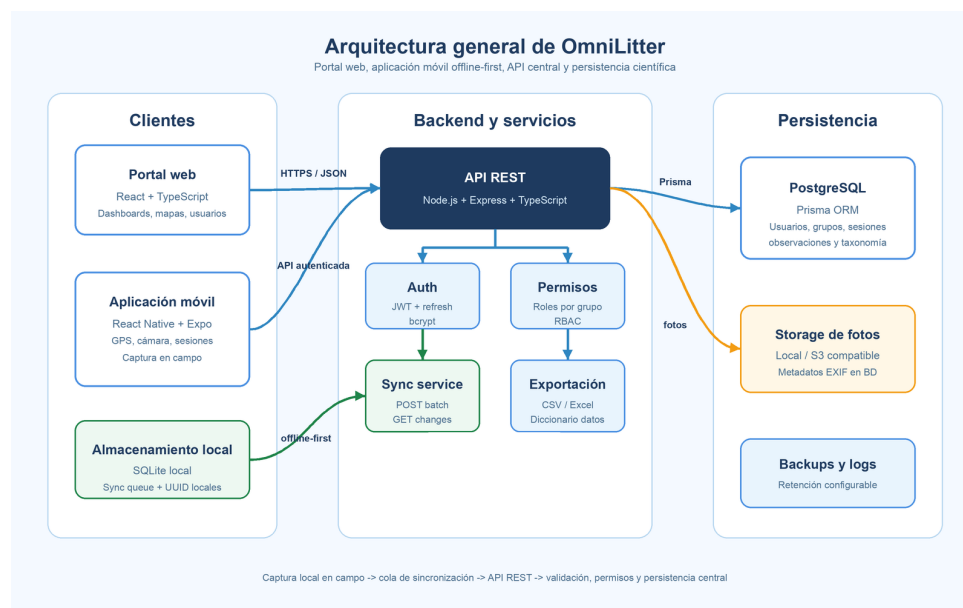


Figura 8.1: Arquitectura general propuesta para OmniLitter.

8.1.1. Justificación de las decisiones arquitectónicas principales

API REST como único punto de acceso. Centralizar la lógica de negocio en la API evita duplicidades entre los dos clientes y facilita la incorporación de nuevos consumidores en el futuro (p.ej., exportaciones automatizadas por *scripts* externos o integraciones con repositorios científicos).

Separación entre portal web y aplicación móvil. Aunque comparten la misma API, los dos clientes tienen requisitos de experiencia de usuario muy distintos: el portal prioriza la visualización analítica y la gestión, mientras que la aplicación móvil prioriza la captura rápida en condiciones de campo adversas (luz solar, guantes, sin conexión). Mantenerlos como proyectos independientes permite evolucionar cada uno a su propio ritmo.

Base de datos local en el móvil. El trabajo de campo se realiza habitualmente en zonas costeras, ribereñas o forestales con cobertura de red intermitente o nula. La decisión de embeber una base de datos local en el dispositivo es, por tanto, un requisito no negociable para la viabilidad del producto.

8.2. Diseño de la base de datos

8.2.1. Modelo relacional

El modelo de datos final de OmniLitter está formado por quince entidades. Para que el diagrama resulte legible y sean más visibles las relaciones, se ha dividido en tres bloques funcionales, cada uno con su propio diagrama entidad-relación:

- **Usuarios, grupos, campañas y catálogo:** `users`, `groups`, `memberships`, `group_invitations`, `campaigns`, `environments`, `typologies` y `typology_clusters`. Recoge la gestión de acceso, la pertenencia a grupos de investigación, las invitaciones, la organización de campañas y el catálogo global de entornos y tipologías.
- **Muestreo, observaciones y taxonomía:** `sampling_sessions`, `observations`, `litter_items`, `observation_photos` y `taxonomy_items`. Es el núcleo operativo del sistema, donde se registran las sesiones de campo, las observaciones de residuos, su desglose y clasificación y las fotografías asociadas.
- **Autenticación y seguridad:** `users`, `refresh_tokens` y `password_reset_tokens`. Agrupa las entidades que dan soporte al inicio de sesión, la renovación de tokens y el restablecimiento de contraseña.

La entidad `sampling_sessions` aparece en los dos primeros bloques porque sirve de unión entre la organización de campañas y la captura de datos en campo. Las figuras 8.2, 8.3 y 8.4 muestran cada uno de estos bloques.

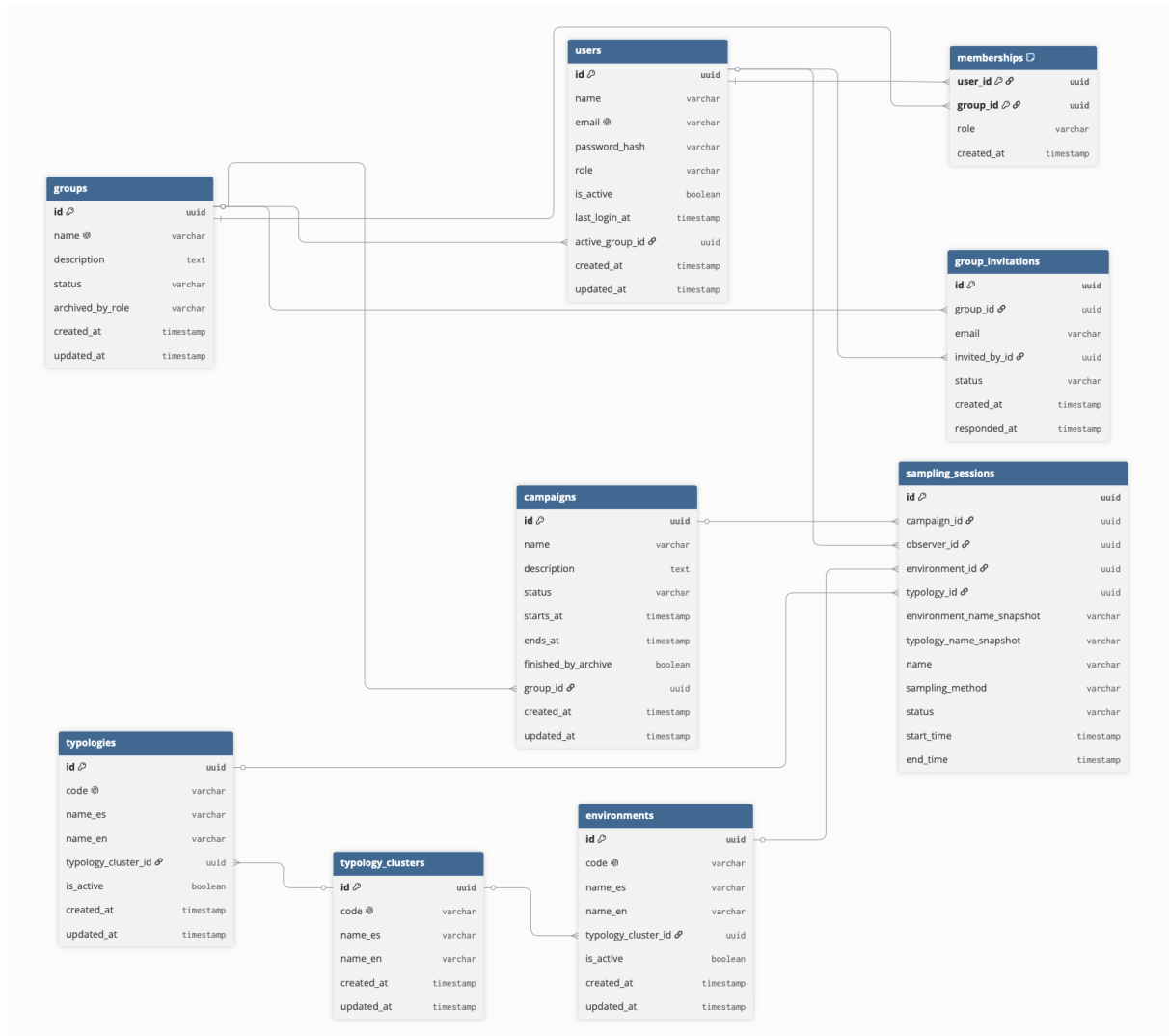


Figura 8.2: Modelo entidad-relación. Bloque de usuarios, grupos, campañas y catálogo de entornos.

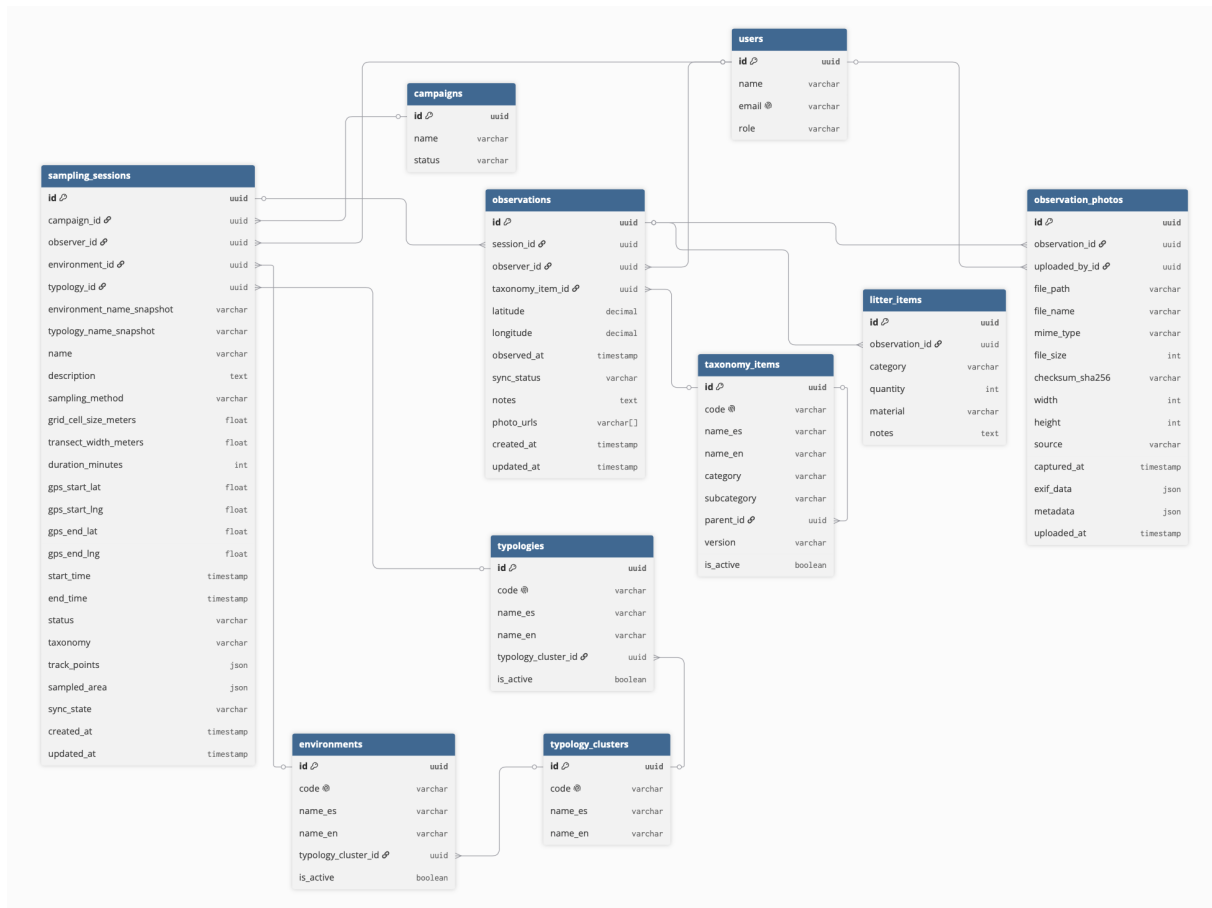


Figura 8.3: Modelo entidad-relación. Bloque de sesiones de muestreo, observaciones, residuos, fotografías y taxonomía.

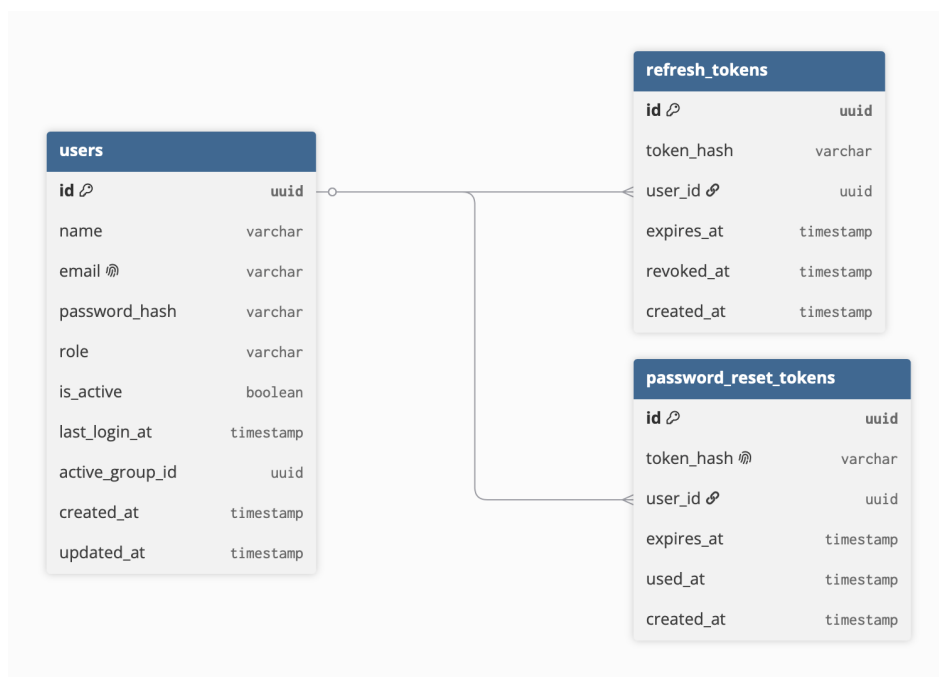


Figura 8.4: Modelo entidad-relación. Bloque de autenticación y seguridad.

8.2.2. Descripción de entidades

La tabla 8.1 describe los atributos clave de cada entidad y su propósito en el sistema.

Cuadro 8.1: Descripción de entidades del modelo de datos.

Entidad	Atributos principales	Descripción
users	id, name, email, password_hash, role, is_active, last_login_at, active_group_id (FK)	Persona registrada en el sistema. La contraseña se guarda como <i>hash</i> y nunca en texto plano. El campo <i>role</i> indica el rol global (SUPERADMIN o USUARIO) y <i>active_group_id</i> apunta al grupo con el que el usuario está trabajando en cada momento.
groups	id, name, description, status, archived_by_role	Grupo de investigación. Un grupo puede archivarse sin borrar sus datos históricos, lo que conserva la trazabilidad de las campañas. El campo <i>archived_by_role</i> registra el rol que lo archivó, para condicionar quién puede restaurarlo.
memberships	user_id (FK), group_id (FK), role, created_at	Tabla de unión entre usuarios y grupos, con clave primaria compuesta. El campo <i>role</i> toma los valores ADMIN o FIELD_RESEARCHER, de modo que un mismo usuario puede tener un rol distinto en cada grupo.
group_invitations	id, group_id (FK), email, invited_by_id (FK), status, responded_at	Invitación de un usuario a un grupo a partir de su correo. El campo <i>status</i> sigue el ciclo PENDING, ACCEPTED o REJECTED.
campaigns	id, name, description, status, starts_at, ends_at, finished_by_archive, group_id (FK)	Proyecto de muestreo asociado a un grupo. Agrupa varias sesiones de campo bajo un objetivo común. El estado evoluciona entre DRAFT, ACTIVE, FINISHED y ARCHIVED.
typology_clusters	id, code, name_es, name_en	Agrupación que relaciona un entorno con el conjunto de tipologías compatibles. Al existir como entidad propia, los entornos y las tipologías se asocian mediante clave foránea en lugar de por texto.

Entidad	Atributos principales	Descripción
<code>environments</code>	<code>id</code> , <code>code</code> , <code>name_es</code> , <code>name_en</code> , <code>typology_cluster_id</code> (FK), <code>is_active</code>	Catálogo global de tipos de entorno de muestreo, como playa, ribera, bosque, carretera o campo de cultivo. Cada entorno cuelga de un <code>typology_cluster</code> y dispone de nombre en español e inglés.
<code>typologies</code>	<code>id</code> , <code>code</code> , <code>name_es</code> , <code>name_en</code> , <code>typology_cluster_id</code> (FK), <code>is_active</code>	Catálogo global de subentornos o tipologías. Comparten <code>typology_cluster</code> con su entorno, lo que permite ofrecer en la app solo las tipologías coherentes con el entorno elegido.
<code>sampling_sessions</code>	<code>id</code> , <code>campaign_id</code> (FK), <code>observer_id</code> (FK), <code>environment_id</code> (FK), <code>typology_id</code> (FK), <code>environment_name_snapshot</code> , <code>typology_name_snapshot</code> , <code>name</code> , <code>sampling_method</code> , <code>grid_cell_size_meters</code> , <code>transect_width_meters</code> , <code>duration_minutes</code> , <code>gps_start_lat/lng</code> , <code>gps_end_lat/lng</code> , <code>start_time</code> , <code>end_time</code> , <code>status</code> , <code>taxonomy</code> , <code>track_points</code> , <code>sampled_area</code> , <code>sync_state</code>	Sesión de muestreo realizada en campo. Referencia el entorno y la tipología del catálogo y, al mismo tiempo, guarda su nombre en los campos <code>snapshot</code> . Registra el método de muestreo (LIBRE, CUADRICULA o TRANSECTO), sus parámetros, la duración y las coordenadas de inicio y fin. Los campos <code>track_points</code> y <code>sampled_area</code> almacenan el recorrido y el área muestreada, y <code>sync_state</code> controla el estado de sincronización.
<code>observations</code>	<code>id</code> , <code>session_id</code> (FK), <code>observer_id</code> (FK), <code>taxonomy_item_id</code> (FK), <code>latitude</code> , <code>longitude</code> , <code>observed_at</code> , <code>sync_status</code> , <code>notes</code> , <code>photo_urls</code>	Registro de un residuo avistado dentro de una sesión. Conserva la posición (<code>latitude</code> y <code>longitude</code>), el instante de captura y la referencia a la taxonomía. Es la unidad básica de dato científico del sistema.
<code>litter_items</code>	<code>id</code> , <code>observation_id</code> (FK), <code>category</code> , <code>quantity</code> , <code>material</code> , <code>notes</code>	Desglose de los residuos contabilizados en una observación. El campo <code>category</code> usa la clasificación por material (PLASTIC, METAL, GLASS y demás categorías) y <code>quantity</code> recoge la cantidad como entero positivo.

Entidad	Atributos principales	Descripción
observation_photos	id, observation_id (FK), uploaded_by_id (FK), file_path, file_name, mime_type, file_size, checksum_sha256, width, height, source, captured_at, exif_data, metadata	Fotografía asociada a una observación. Almacena los datos del fichero, un checksum SHA-256 para verificar la integridad tras la sincronización y los metadatos EXIF de la imagen cuando están disponibles.
taxonomy_items	id, code, name_es, name_en, category, subcategory, parent_id (FK), version, is_active	Catálogo jerárquico de tipos de residuo basado en listas comunes de categorías. La referencia <code>parent_id</code> hacia la propia entidad permite organizar la taxonomía por niveles y descargarla completa para usarla sin conexión.
refresh_tokens	id, token_hash, user_id (FK), expires_at, revoked_at	Token de renovación de sesión. Se guarda únicamente su <i>hash</i> , junto con la fecha de expiración y, en su caso, la de revocación.
password_reset_tokens	id, token_hash, user_id (FK), expires_at, used_at	Token de restablecimiento de contraseña. Es de un solo uso: el campo <code>used_at</code> marca el momento en que se consume y lo invalida.

8.2.3. Decisiones de diseño de la base de datos

UUID como clave primaria en registros sincronizables. Los registros que pueden crearse en el dispositivo móvil antes de comunicarse con el servidor utilizan identificadores UUID generados en el propio cliente, definidos como identificadores de 128 bits sin necesidad de registro central [3]. Esto resuelve las colisiones de claves en escenarios de creación offline: dos dispositivos que creen una sesión o una observación sin conexión obtienen identificadores únicos que no chocan al sincronizarse con el servidor.

Modelo de roles en dos niveles. El control de acceso separa el rol global del usuario, almacenado en `users.role` con los valores `SUPERADMIN` o `USUARIO`, del rol que tiene dentro de cada grupo, almacenado en `memberships.role` como `ADMIN` o `FIELD_RESEARCHER`. El rol operativo no es una propiedad fija de la persona, sino de su relación con un grupo concreto, por lo que un mismo investigador puede administrar un grupo y participar como investigador de campo en otro. El superadmin queda reservado a la administración global de la plataforma, que en principio sería nuestro cliente.

Catálogo normalizado de entornos con copia histórica en la sesión. El catálogo de entornos y tipologías se modela con tres entidades relacionadas por clave foránea,

no por texto. La entidad `typology_clusters` agrupa cada entorno con las tipologías compatibles, y tanto `environments` como `typologies` cuelgan del mismo *cluster*. Así, al elegir un entorno la aplicación ofrece solo las tipologías coherentes con él. Cada registro del catálogo guarda su nombre en español e inglés (`name_es` y `name_en`), lo que permite al superadmin configurar ambos idiomas desde el portal.

Cuando se crea una sesión, esta referencia el entorno y la tipología mediante clave foránea real (`environment_id` y `typology_id`) y, al mismo tiempo, copia su nombre en los campos `environment_name_snapshot` y `typology_name_snapshot`. La clave foránea mantiene la integridad con el catálogo vigente, mientras que la copia conserva el valor utilizado en el momento del muestreo aunque el catálogo cambie después. Esta combinación evita inconsistencias en los datos históricos y simplifica la sincronización de la app, que no necesita resolver referencias contra un catálogo que pudo cambiar mientras el dispositivo estaba sin conexión.

Separación entre observación y desglose de residuos. Una observación representa un punto avistado, con sus coordenadas e instante de captura, mientras que `litter_items` recoge los residuos contabilizados en ese punto, con su categoría por material y su cantidad. Separar ambos conceptos permite asociar varios tipos de residuo a una misma observación sin duplicar la información de posición, y deja la clasificación detallada en manos de `taxonomy_items`.

Precisión geográfica con tipo decimal. Las coordenadas de las observaciones se almacenan con tipo `decimal` de precisión fija (nueve dígitos, seis de ellos decimales) en lugar de coma flotante. Así se obtiene una precisión cercana a los diez centímetros y se evitan los errores de redondeo propios del punto flotante, algo necesario para una herramienta cuya utilidad depende de la calidad de la georreferenciación.

Estados de sincronización en sesiones y observaciones. Tanto `sampling_sessions` como `observations` incluyen un campo de estado de sincronización basado en el enum `SyncStatus` (`SYNCED`, `PENDING`, `FAILED` o `CONFLICT`). Este campo sostiene el flujo de trabajo sin conexión, ya que permite distinguir qué registros están pendientes de subir, cuáles se han confirmado en el servidor y cuáles han fallado.

Estrategias de borrado según el tipo de relación. El esquema aplica reglas de borrado coherentes con el valor de cada relación. Los datos estrictamente dependientes se eliminan en cascada junto a su contenedor: las membresías e invitaciones de un grupo, las sesiones de una campaña y las observaciones, residuos y fotos asociados a una sesión. En cambio, las referencias al autor (`observer_id` y `uploaded_by_id`) usan una regla restrictiva, de modo que no se puede borrar un usuario que tenga datos de campo asociados y se preserva la autoría. Las campañas pueden quedar sin grupo mediante `SetNull` para mantener el histórico si se elimina el grupo, y el ítem taxonómico de una observación también se puede poner a nulo para no perder el registro científico.

En el catálogo ambiental se combinan restricciones y copia histórica. El entorno de una

sesión usa una relación restrictiva, porque toda sesión debe conservar un contexto ambiental válido. La tipología puede ponerse a nulo si se elimina del catálogo, pero la sesión mantiene `typology_name_snapshot`; de forma equivalente, `environment_name_snapshot` conserva el nombre visible del entorno utilizado en campo. Así se evita que las tareas de mantenimiento del catálogo degraden los datos ya registrados.

JSON para datos de estructura variable. Los campos `track_points`, `sampled_area`, `exif_data` y `metadata` se almacenan como JSON. El recorrido y el área muestreada tienen una geometría que depende del método de muestreo, y los metadatos EXIF varían según el modelo de cámara. Guardar estos datos como JSON evita multiplicar columnas opcionales y permite ampliar la información asociada sin migrar el esquema.

8.3. Diseño de la API REST

La API sigue los principios REST: interfaz uniforme, comunicación sin estado, recursos identificados por URI y representaciones en formato JSON. Todos los *endpoints* protegidos requieren el encabezado `Authorization: Bearer <token>`, donde el *token* es un JWT firmado emitido en el *endpoint* de autenticación.

El cuadro 8.2 recoge los *endpoints* principales agrupados por módulo funcional. En lugar de presentar todas las rutas en una única tabla continua, se separan por bloque para que resulte más visible qué parte de la API atiende cada conjunto de operaciones.

Cuadro 8.2: Principales *endpoints* de la API REST de OmniLitter.

Autenticación			
HTTP	Ruta	Acceso	Uso
POST	/api/auth/login	Público	Emite JWT de acceso y refresco
POST	/api/auth/register	Público	Registra una cuenta nueva
POST	/api/auth/recover-password	Público	Inicia la recuperación de contraseña
POST	/api/auth/reset-password	Público	Restablece la contraseña con un <i>token</i>
POST	/api/auth/refresh	Público	Renueva el JWT de acceso
POST	/api/auth/logout	Usuario	Invalida el <i>token</i> de refresco

Usuarios			
HTTP	Ruta	Acceso	Uso
GET	/api/users	Admin	Lista todos los usuarios
POST	/api/users	Admin	Crea un usuario
GET	/api/users/me	Usuario	Datos del usuario autenticado
GET	/api/users/me/groups	Usuario	Grupos del usuario y grupo activo
PATCH	/api/users/:id	Admin	Actualiza datos, rol o estado

Grupos e invitaciones

HTTP	Ruta	Acceso	Uso
GET	/api/groups	Usuario	Lista grupos del usuario
POST	/api/groups	Admin	Crea un grupo
PUT	/api/groups/:id	Admin	Edita un grupo
GET	/api/groups/:id/backup	Admin	Descarga copia de seguridad JSON
POST	/api/group-invitations/:id/accept	Usuario	Acepta una invitación

Campañas

HTTP	Ruta	Acceso	Uso
GET	/api/campaigns	Usuario	Lista campañas del grupo activo
POST	/api/campaigns	Admin	Crea una campaña
PUT	/api/campaigns/:id	Admin	Edita una campaña

Catálogo científico

HTTP	Ruta	Acceso	Uso
GET	/api/catalog/environments	Usuario	Lista entornos del catálogo
POST	/api/catalog/environments	Superadmin	Crea un entorno
GET	/api/catalog/typologies	Usuario	Lista tipologías del catálogo

Sesiones de muestreo

HTTP	Ruta	Acceso	Uso
GET	/api/sessions	Usuario	Lista sesiones accesibles
POST	/api/sessions	Usuario	Crea una sesión
GET	/api/sessions/:id	Usuario	Obtiene una sesión con observaciones
GET	/api/sessions/tracks	Usuario	Recorridos y áreas muestreadas
PATCH	/api/sessions/:id	Usuario	Actualiza metadatos o estado

Observaciones y fotos

HTTP	Ruta	Acceso	Uso
GET	/api/observations	Usuario	Lista observaciones con filtros
POST	/api/observations	Usuario	Registra una observación
GET	/api/observations/:id/photos	Usuario	Lista fotos de una observación
POST	/api/observations/:id/photos	Usuario	Sube una foto (base64 en el cuerpo)
GET	/api/observations/:id/photos/:pid/file	Usuario	Sirve el archivo de imagen

Taxonomía			
HTTP	Ruta	Acceso	Uso
GET	/api/taxonomy/items	Usuario	Descarga la taxonomía filtrable por versión

Métricas			
HTTP	Ruta	Acceso	Uso
GET	/api/metrics	Usuario	Métricas agregadas para el dashboard
GET	/api/metrics/observations	Usuario	Datos por observación para analítica

Sincronización			
HTTP	Ruta	Acceso	Uso
POST	/api/sync/batch	Usuario	Sube el lote de registros pendientes
GET	/api/sync/changes	Usuario	Descarga cambios desde la última sincronización

La exportación científica no dispone de un *endpoint* propio: el portal web genera los ficheros CSV (observaciones, ítems, resumen y un formato científico con diccionario de columnas) y GeoJSON a partir de los datos ya consultados. El contrato completo de la API se documenta con OpenAPI y puede explorarse de forma interactiva en `/api/docs` mediante Swagger UI.

8.4. Seguridad y control de acceso

El sistema se diseña con autenticación basada en JWT [11] y dos *tokens*: un **token de acceso** de corta duración (15 minutos) y un **token de refresco** de larga duración (7 días). El token de acceso se envía en cada petición en el encabezado **Authorization**; el de refresco se utiliza exclusivamente para solicitar un nuevo token de acceso cuando el anterior expira.

La autorización sigue un modelo de control de acceso basado en roles (RBAC) [15] a nivel de grupo. El *middleware* de autorización verifica, para cada petición, que el usuario autenticado posee el rol requerido en el grupo al que pertenece el recurso solicitado. Esto permite que una misma persona sea administradora en un grupo y colaboradora en otro, con permisos diferentes en cada contexto.

8.5. Diseño del portal web

8.5.1. Arquitectura de componentes

El portal web se diseña como una *Single Page Application* (SPA) con React 18 y TypeScript, empaquetada con Vite y estilizada con Tailwind CSS. El enrutado se resuelve en el cliente con la API `History` del navegador (`pushState` y `replaceState`), sin una librería de enrutado externa, y distingue dos niveles de rutas:

- **Rutas públicas:** la página de inicio (/) y el formulario de inicio de sesión (/login), accesibles sin autenticación.
- **Rutas protegidas:** las páginas privadas bajo /app/*, que comprueban el estado de autenticación antes de renderizarse y redirigen al *login* si no existe una sesión activa.

La estructura de componentes sigue una jerarquía de tres niveles: el **Layout** general (barra lateral, encabezado, selector de grupo activo) envuelve las páginas individuales, que a su vez utilizan componentes reutilizables (tarjetas de estadística, tablas filtrables, modales de confirmación). Las vistas geográficas se construyen con Leaflet a través de `react-leaflet`, con agrupación de marcadores y una capa de mapa de calor, y los gráficos del dashboard y de la analítica se generan con Recharts.

8.5.2. Gestión de estado

El portal no utiliza una librería de gestión de estado externa. El estado de la aplicación se mantiene en componentes React y se transmite mediante propiedades, algo suficiente para la escala del portal. La sesión del usuario se conserva entre recargas en el almacenamiento del navegador, donde se guardan los *tokens* de acceso y refresco emitidos por el backend, y se renueva de forma automática cuando el token de acceso expira.

El grupo activo forma parte de ese estado de sesión. Al cambiarlo, las vistas que dependen del grupo (campanas, observaciones, mapa, dashboard y analítica) recargan su contenido para reflejar únicamente los datos del grupo seleccionado.

8.5.3. Flujo de navegación y control de acceso por rol

Las secciones de gestión de grupos y usuarios están reservadas al rol de administrador. El control efectivo de acceso lo aplica el backend, que rechaza cualquier operación no autorizada aunque la vista llegue a cargarse; la interfaz, por su parte, oculta o deshabilita las acciones de administración para los colaboradores. El Dashboard ajusta el contenido según el rol: los administradores ven las estadísticas del grupo activo, mientras que

los colaboradores ven sus propias métricas. La tabla de observaciones muestra a cada colaborador sus registros y al administrador los de todo el grupo.

8.6. Diseño de la aplicación móvil

8.6.1. Stack tecnológico y estructura de pantallas

La aplicación móvil está planteada con **React Native** y **Expo**, usando *development builds* o *prebuild* cuando sea necesario acceder a capacidades nativas del dispositivo. Esta aproximación permite mantener las ventajas del ecosistema Expo sin renunciar a módulos como `expo-location`, `expo-camera` y las tareas de sincronización en segundo plano mediante `expo-task-manager` y `expo-background-task`.

La navegación está organizada en dos niveles:

- **Navegación de primer nivel (tab bar):** cinco pestañas inferiores (Inicio, Sesiones, Mapa, Sincronización, Perfil), visibles únicamente en las pantallas principales.
- **Navegación de pila (*stack*):** flujos de varios pasos como la creación de una nueva sesión o la captura de una observación, que se presentan sin el *tab bar*.

8.6.2. Gestión de estado global

El estado global de la aplicación se gestiona mediante el patrón **Context + useReducer** de React, sin dependencias externas de gestión de estado. El contexto `AppContext` expone el estado completo (sesiones activas, tipos de residuo, datos del usuario, estado de sincronización) y un *dispatcher* con acciones tipadas en TypeScript. Este patrón proporciona la predictibilidad de Redux con una superficie de código significativamente menor, adecuada para la escala del proyecto.

8.6.3. Base de datos local con SQLite

SQLite, integrado en Expo mediante `expo-sqlite`, actúa como base de datos local de la aplicación móvil. Se elige por ser una solución madura, ligera y suficientemente potente para almacenar el volumen de datos esperado durante una jornada de muestreo. Además, permite mantener control directo sobre el esquema local y sobre la cola de sincronización, reduciendo la complejidad respecto a incorporar una capa adicional de abstracción.

El esquema local replica el subconjunto del modelo de datos necesario para la operación en campo: `sampling_sessions`, `observations`, `litter_items` y `taxonomy_items`, junto con las tablas de apoyo de usuarios, grupos, campañas y catálogo (`environments` y `typologies`) que se descargan al iniciar sesión. Las operaciones pendientes de subida se

registran en una tabla *outbox* (**sync_outbox**) que guarda, por cada cambio, el *endpoint* y el *payload* de la petición, el número de intentos, el último error y el estado de sincronización.

8.6.4. Flujo de captura de una observación

El flujo principal de la aplicación sigue esta secuencia:

1. El investigador inicia una nueva sesión de muestreo, especificando el nombre, el tipo de entorno y los parámetros del transecto.
2. Para cada ítem encontrado, abre la pantalla de nueva observación, selecciona el tipo de residuo a través del modal de taxonomía (organizado en cuadrícula visual por categorías e iconos), e introduce la cantidad, el tamaño, el color y comentarios.
3. Opcionalmente captura una foto geolocalizada del residuo.
4. Al finalizar la sesión de campo, revisa el resumen y puede iniciar la sincronización manualmente o dejar que se realice en segundo plano al recuperar cobertura de red.

El tamaño y el color se recogen en el formulario como ayuda a la caracterización del residuo, pero no se modelan como columnas propias. El ítem de residuo (**litter_items**) almacena la categoría, la cantidad y el material, y estas dos anotaciones se serializan dentro de su campo de notas, de donde la aplicación las vuelve a leer al editar la observación.

8.7. Sincronización offline

8.7.1. Patrón Outbox Queue

El mecanismo de sincronización se diseña como una cola de salida local dentro de un enfoque *offline-first*. Este tipo de diseño prioriza una fuente local persistente y la reconciliación posterior con la fuente remota cuando vuelve la conectividad [1]. En OmniLitter, esta idea se concreta en una tabla **sync_outbox** y en estados de sincronización por registro. El funcionamiento es el siguiente:

1. Cuando el usuario crea o modifica un registro (sesión u observación) en el dispositivo sin conexión, el registro se persiste en SQLite con estado **PENDING** y se inserta una entrada en la tabla **sync_outbox** con el *payload* completo del cambio.
2. Cuando el dispositivo recupera conectividad, una tarea de sincronización en segundo plano o una acción manual del usuario lee la cola y realiza una petición **POST** `/api/sync/batch` con todos los registros pendientes.
3. El servidor procesa los registros en orden, los persiste en PostgreSQL y devuelve el estado canónico actualizado.

4. El cliente actualiza el estado de los registros procesados a **SYNCED** y descarga los cambios remotos realizados por otros usuarios mediante el *endpoint* `GET /api/sync/changes`, enviando la marca de tiempo de la última sincronización como parámetro de consulta.
5. Si un registro concreto falla, pasa al estado **FAILED** cuando los datos son inválidos o la subida no se completa, o a **CONFLICT** cuando el servidor detecta una versión más reciente. En ambos casos se notifica al usuario sin bloquear la sincronización del resto.

8.7.2. Estados de sincronización

Cada sesión y observación puede encontrarse en uno de cuatro estados, como muestra la tabla 8.3.

Cuadro 8.3: Estados de sincronización y su significado.

Estado	Significado
PENDING	El registro existe solo en el dispositivo y está pendiente de subir al servidor.
SYNCED	El servidor ha confirmado la recepción y persistencia del registro.
FAILED	La subida no se completó o el servidor rechazó el registro por datos inválidos. Requiere reintento.
CONFLICT	El servidor detectó una versión más reciente del registro. Requiere resolución.

8.7.3. Resolución de conflictos

La estrategia de resolución de conflictos propuesta es **last-write-wins** con granularidad de registro: si dos usuarios modifican la misma sesión de forma independiente mientras están sin conexión, prevalece la versión con el **timestamp** más reciente. Esta estrategia es suficiente para el modelo de uso de OmniLitter, donde cada sesión de muestreo pertenece a un único observador responsable y los conflictos genuinos son infrecuentes.

8.7.4. Garantía de identificadores únicos en entorno offline

Para que la cola de salida local funcione sin depender de un identificador asignado por el servidor en el momento de la captura, los registros creados en modo offline usan **UUID v4 generados en el cliente** [3]. Esto garantiza que las sesiones y observaciones creadas simultáneamente en distintos dispositivos no colisionarán al sincronizarse, independientemente del orden de llegada al servidor.

9. Desarrollo, implantación y validación

La fase de desarrollo e implementación de OmniLitter recoge el proceso seguido para transformar los requisitos definidos previamente en un sistema funcional compuesto por una aplicación móvil, un portal web y un backend centralizado. Esta sección describe cómo se organizó el trabajo a lo largo de los distintos sprints, qué partes del sistema se abordaron en cada etapa y qué decisiones técnicas permitieron construir una solución coherente con las necesidades del proyecto. Además, se detallan las funcionalidades implementadas, la integración entre los distintos componentes, el soporte para trabajo offline en la aplicación móvil, la sincronización con el servidor y la estructura general que permite gestionar campañas, sesiones, observaciones, usuarios y grupos dentro de la plataforma.

9.1. Sprint 1: Base funcional del sistema

9.1.1. Características y objetivos del sprint

El primer sprint tuvo como objetivo establecer la base funcional de OmniLitter. En esta primera iteración se buscó construir una versión mínima pero coherente del sistema, capaz de soportar los flujos principales sobre los que se apoyaría el resto del desarrollo. Para ello, se abordaron las funcionalidades esenciales relacionadas con el acceso de usuarios, la separación de datos por grupos de trabajo, la creación inicial de campañas y sesiones de muestreo, y el registro básico de observaciones desde la aplicación móvil.

Este sprint fue especialmente importante porque permitió definir la estructura operativa inicial del sistema. A partir de esta base, los usuarios podían acceder a la plataforma, trabajar dentro de un grupo concreto, crear campañas como contexto organizativo del trabajo de campo y registrar las primeras observaciones asociadas a una sesión. De este modo, se estableció el flujo principal de uso de OmniLitter, que posteriormente sería ampliado con capacidades de geolocalización, fotografías, trabajo offline, sincronización y explotación de datos.

- **Autenticación y acceso.** Esta funcionalidad constituye el punto de entrada al sistema y permite identificar de forma segura a cada usuario antes de acceder a las zonas privadas. Abarca el registro, el inicio y cierre de sesión, la conservación de la sesión válida y la protección de rutas internas tanto en el portal web como en la aplicación móvil.
- **Usuarios, grupos y permisos.** Esta parte funcional organiza el trabajo colaborativo mediante grupos, membresías y roles. Su finalidad es delimitar qué datos puede consultar o modificar cada usuario, qué grupo actúa como contexto activo de trabajo y cómo se separa la información entre equipos diferentes.
- **Campañas.** La campaña funciona como unidad organizativa del trabajo de campo. Sirve para planificar una actividad de muestreo dentro de un grupo, agrupar las sesiones que pertenecen al mismo contexto y mantener una referencia común para consultar después las observaciones registradas.
- **Sesiones de muestreo.** La sesión representa una salida o actividad concreta de recogida de datos dentro de una campaña. Esta funcionalidad permite relacionar el trabajo realizado con un grupo, un observador, una campaña y el conjunto de observaciones generadas durante esa actividad.
- **Observaciones.** La observación es el registro básico de un residuo encontrado durante una sesión. Esta funcionalidad recoge la información necesaria para describirlo, como taxonomía, coordenadas, cantidades, materiales, categorías y notas, manteniendo además su relación con la sesión, la campaña, el grupo y el usuario responsable.

9.1.2. Implementación realizada

El primer sprint se dedicó a construir la base técnica sobre la que se apoyaría el resto de OmniLitter. La implementación se centró en disponer de una arquitectura operativa mínima, con autenticación, separación de datos por grupos, gestión inicial de campañas y sesiones, y registro básico de observaciones desde la aplicación móvil.

Backend y modelo de dominio. Se estructuró una API REST modular sobre Express [16], con módulos diferenciados para autenticación, usuarios, grupos, campañas, sesiones, observaciones y taxonomía. Las rutas privadas quedaron protegidas mediante middleware de autenticación JWT [11] y las reglas de acceso se articularon alrededor del grupo activo del usuario. Esta decisión permitió separar el rol global del usuario del rol que ejerce dentro de cada grupo, distinguiendo entre administración general, administración de grupo e investigador de campo.

En autenticación se implementaron registro, inicio de sesión, cierre de sesión, refresco de sesión y recuperación de contraseña. Las contraseñas se almacenaron mediante hash y los tokens de refresco quedaron persistidos para poder revocar sesiones. En el modelo de datos consolidado se definieron las entidades principales del sistema: usuarios, tokens, grupos, membresías, invitaciones, campañas, sesiones de muestreo, taxonomía, observaciones e ítems de residuo. No se localizaron migraciones incrementales por sprint; el repositorio conserva una migración inicial con el estado final consolidado.

El flujo de sesión se diseñó con separación entre token de acceso y token de refresco. El access token contiene únicamente el identificador del usuario y su rol global, mientras que el refresh token se almacena en base de datos como hash y se rota en cada refresco. De este modo, el cierre de sesión, el cambio de contraseña o la recuperación de contraseña pueden revocar sesiones existentes sin depender solo de la expiración del JWT. La recuperación de contraseña se implementó con tokens aleatorios de un solo uso, almacenados como hash SHA-256 y con caducidad de 30 minutos.

La autorización por grupos se resolvió con funciones de dominio reutilizadas por campañas, sesiones, observaciones, métricas y administración. El backend distingue entre poder acceder a un grupo, poder administrarlo y resolver el alcance completo de grupos visibles para el usuario autenticado. Esta capa es la que permite que un superadministrador vea información agregada, que un administrador gestione su grupo y que un investigador quede limitado a sus propios datos cuando corresponde.

Portal web. El portal web se organizó como una aplicación React [13] con rutas protegidas, conservación de sesión en el navegador y un layout común para las áreas privadas. Sobre esta base se integraron las primeras pantallas de campañas, grupos y administración de usuarios. La navegación interna quedó preparada para incorporar progresivamente módulos posteriores como observaciones, mapa, analítica y configuración.

Aplicación móvil y paquete compartido. La aplicación móvil se implementó con Expo y React Native [6, 14]. Se definió el flujo de autenticación, la navegación principal y los primeros servicios de consumo de la API. Desde la sección de campañas se incorporó la creación inicial de sesiones y observaciones, utilizando contratos compartidos para evitar divergencias entre clientes y servidor. El paquete común centralizó tipos y esquemas Zod [2] de autenticación, campañas, sesiones, observaciones, ítems y fotografías, de modo que la validación de entrada no dependiera únicamente de cada interfaz.

Integración entre capas. La integración se resolvió mediante servicios REST consumidos por web y móvil, tokens JWT en las peticiones privadas y validación compartida en `packages/shared`. Las campañas y sesiones quedaron vinculadas al grupo activo, mientras que las observaciones se asociaron a una sesión, una taxonomía y un usuario observador. Con ello se estableció el flujo técnico básico: autenticación, selección de contexto de trabajo, planificación de campaña, creación de sesión y registro inicial de residuos.

Cuadro 9.1: Trazabilidad técnica de la implementación del Sprint 1

Bloque implementado	Plataforma	Elementos principales	Finalidad
Autenticación y sesión	Backend, web y móvil	Registro, login, refresh, logout, recuperación de contraseña y rutas protegidas	Establecer acceso seguro y sesión reutilizable en las tres capas.
Grupos y permisos	Backend y web	Usuarios, grupos, membresías, invitaciones, grupo activo y roles por contexto	Separar datos entre equipos y controlar operaciones según permisos.
Campañas y sesiones	Backend, web y móvil	Servicios REST, gestión web de campañas y creación móvil de sesiones	Definir el contexto organizativo del trabajo de campo.
Observaciones iniciales	Backend, móvil y shared	API de observaciones, formulario móvil, taxonomía y esquemas compartidos	Registrar residuos vinculados a sesión, usuario y clasificación científica.
Estructura común	Web, móvil y shared	SPA con rutas protegidas, navegación móvil y contratos Zod	Preparar una base extensible para los módulos de sprints posteriores.

Como resultado técnico, el sprint dejó disponible una versión mínima funcional del sistema, con acceso autenticado, organización por grupos, administración básica, planificación de campañas, creación de sesiones y captura inicial de observaciones. Esta base permitió abordar en los sprints siguientes la captura avanzada de datos, el trabajo offline, la sincronización y la explotación de la información registrada.

9.1.3. Pruebas realizadas

En el primer sprint se validó la base funcional del sistema. Las pruebas se centraron en autenticación, administración inicial de usuarios y grupos, campañas, catálogo, sesiones de muestreo y primeras observaciones. Esta fase era especialmente importante porque establecía los contratos y servicios sobre los que se apoyarían los sprints posteriores.

9.1.3.1. Pruebas automatizadas implementadas

Cuadro 9.2: Pruebas automatizadas vinculadas al Sprint 1

Tarea o funcionalidad	Cobertura automatizada	Comprobación principal	Resultado
Login, registro y cierre de sesión	SCH-01, SCH-02	Validación del registro con contraseña robusta, confirmación coincidente y rechazo de contraseñas débiles o confirmaciones incorrectas.	Correcto
Portal de administración de usuarios	USR-01, USR-03	Bloqueo de correos duplicados y creación de usuarios con o sin membresías iniciales.	Correcto
Gestión de grupos y membresías	GRP-06, GRP-07	Restauración de grupos, permisos de administración y rechazo de acciones realizadas por usuarios sin autorización.	Correcto
Campañas y sesiones backend	SCH-04 a SCH-13, CMP-01	Estados válidos de campañas, referencias de catálogo y reglas de geometría para sesiones libres, por cuadrícula o por transecto.	Correcto
Catálogo científico	CAT-01 a CAT-08	Creación, actualización, detección de duplicados, borrado y restricciones por uso de entornos y tipologías.	Correcto
API de observaciones y adjuntos	SCH-14 a SCH-23, OBS-04 a OBS-06	Observaciones con ítems obligatorios, coordenadas válidas, cantidades positivas, tipos de imagen soportados y control de acceso a observaciones propias o ajenas.	Correcto
Formulario móvil de observación	SCH-14 a SCH-16	Validación compartida de sesión, taxonomía, coordenadas, ítems y cantidades utilizada por el formulario móvil.	Correcto

Estas pruebas protegen reglas que no deben depender únicamente de la interfaz. Aunque los formularios ayuden a evitar datos incorrectos, los esquemas compartidos y los servicios de backend garantizan que la validación se mantiene ante datos reutilizados por distintos clientes o enviados directamente a la API.

9.1.4. Desviaciones e incidencias

Durante el primer sprint se detectaron varios ajustes propios de la puesta en marcha de una arquitectura con backend, portal web y aplicación móvil. Las desviaciones se concentraron principalmente en la definición del flujo de dominio y en la configuración inicial de las distintas plataformas.

Cuadro 9.3: Desviaciones e incidencias del Sprint 1

Tipo	Desviación o incidencia	Causa	Efecto sobre el sprint
Ajuste funcional	Incorporación explícita de campañas al flujo grupo–campaña–sesión–observación.	El backlog inicial no contemplaba con suficiente detalle la campaña como entidad intermedia del flujo de trabajo.	Fue necesario crear una tarea adicional y adaptar parte de lo ya implementado para mantener la coherencia del modelo.
Configuración técnica	Mayor esfuerzo de configuración inicial entre web, móvil y API.	La solución debía coordinar dos clientes distintos conectados a un backend común.	Se produjo una carga inicial superior de configuración y pequeñas refactorizaciones durante la integración.
Validación en entorno móvil	Ralentización de algunas pruebas de la aplicación móvil.	Existía una curva de aprendizaje asociada al uso de emuladores móviles.	Las comprobaciones móviles requirieron más tiempo, sin alterar el alcance funcional previsto para el sprint.
Ajuste de permisos	Revisión del sistema de permisos basado en grupo activo y pertenencia.	Los roles globales no resultaban suficientes para diferenciar investigadores, administradores de grupo y superadministradores.	Se reajustaron filtros de datos, API y pantallas de administración para reflejar correctamente la visibilidad por contexto.

En conjunto, estas desviaciones no comprometieron el objetivo del sprint, pero sí obligaron a consolidar con mayor precisión el modelo de dominio y las reglas de autorización. El efecto principal fue un esfuerzo adicional de ajuste arquitectónico que dejó una base más coherente para los sprints posteriores.

9.1.5. Resultado del sprint

El primer sprint cerró con una versión mínima pero operativa de OmniLitter. Su resultado principal no fue solo disponer de autenticación, grupos, campañas, sesiones y primeras observaciones, sino comprobar que esas piezas podían funcionar como un flujo único: el usuario accede al sistema, trabaja dentro de un grupo, organiza una campaña, abre una sesión y registra información desde la aplicación móvil.

La iteración permitió validar las decisiones de base del proyecto. La separación entre backend, portal web, aplicación móvil y paquete compartido resultó adecuada para construir sobre ella, y el uso de una API común permitió coordinar las distintas capas sin duplicar reglas de negocio. También se identificaron dos ajustes clave: incorporar la campaña como

entidad explícita del modelo y reforzar los permisos a partir de la pertenencia a grupos, no únicamente mediante roles generales.

Como conclusión, el sprint cumplió su función de arranque del desarrollo incremental. Dejó una arquitectura verificable, un modelo de dominio más coherente y un primer recorrido funcional suficiente para abordar con mayor seguridad las capacidades posteriores: captura avanzada de datos, persistencia local, sincronización, análisis y administración más completa.

Cuadro 9.4: Resumen de horas reales del Sprint 1.

Horas del Sprint 1	
Horas planificadas del Sprint	X
Horas reales (Alejandro Castilla)	X
Horas reales (Ignacio Martínez Díaz)	X
Horas reales totales del Sprint	X
Horas acumuladas planificadas hasta este Sprint	X
Horas acumuladas reales hasta este Sprint	X

9.2. Sprint 2: Captura de campo y análisis inicial

9.2.1. Características y objetivos del sprint

El segundo sprint se centró en ampliar el flujo de trabajo de campo y mejorar la calidad de los datos registrados desde la aplicación móvil. Una vez construida la base funcional del sistema, esta iteración permitió enriquecer las sesiones y observaciones con información más precisa, incorporando coordenadas, fotografías, metadatos, validaciones y mecanismos de revisión previa.

Además, este sprint introdujo las primeras funcionalidades de explotación de datos desde el portal web. Esto permitió que la información capturada desde la aplicación móvil no quedara limitada al registro, sino que pudiera consultarse, filtrarse e interpretarse desde la parte web del sistema. Por tanto, este sprint supuso un avance tanto en la calidad de la captura en campo como en la capacidad inicial de análisis de la información registrada.

- **Sesiones de muestreo II.** Esta funcionalidad continúa el trabajo iniciado en las sesiones de muestreo, ampliándolas para que representen una actividad de campo con información espacial y de progreso. Además de identificar la campaña y el observador, la sesión pasa a reflejar el recorrido realizado, la distancia aproximada y el avance del muestreo, datos necesarios para interpretar posteriormente la cobertura del trabajo.
- **Observaciones II.** Esta parte funcional continúa la funcionalidad de observaciones, ampliando el registro básico de residuos con coordenadas, fotografías y metadatos asociados. También contempla la revisión y corrección de datos antes de su consolidación, así como reglas de validación y permisos para mantener la coherencia científica y el control de acceso.
- **Trabajo offline.** Esta funcionalidad permite que la aplicación móvil siga siendo útil cuando la conectividad es limitada. Su objetivo es conservar localmente la información necesaria para navegar, consultar datos ya disponibles y continuar el trabajo de campo sin depender de una conexión constante.
- **Consulta web de observaciones.** Esta funcionalidad convierte el portal web en una herramienta de revisión de los datos capturados. Permite localizar observaciones mediante filtros, abrir su detalle, comprobar la información asociada y consultar fotografías o metadatos relevantes desde un entorno más cómodo para la supervisión.
- **Dashboard y métricas.** El dashboard ofrece una visión agregada de la actividad del grupo activo. Su función es resumir el estado general del trabajo mediante totales, distribuciones e indicadores sobre campañas, sesiones, observaciones, residuos y fotografías.
- **Analítica y exportación.** Esta parte funcional está orientada a explorar los datos registrados y preparar su reutilización. Permite trabajar con consultas filtradas, revisar

subconjuntos de información y establecer la base para generar salidas exportables en fases posteriores del desarrollo.

9.2.2. Implementación realizada

El segundo sprint amplió la base anterior hacia un flujo de campo más completo. La implementación se centró en enriquecer las sesiones y observaciones con coordenadas, recorridos, fotografías y persistencia local, además de incorporar las primeras capacidades web de consulta, métricas y análisis sobre los datos capturados.

Captura móvil y persistencia local. En la aplicación móvil se incorporó la obtención de ubicación mediante `expo-location` [7], con solicitud de permisos y lectura de coordenadas del dispositivo. Para las sesiones se implementó el registro de recorridos mediante puntos GPS, aplicando filtros de precisión, distancia mínima y velocidad máxima. Esta información permitió representar el avance de una salida de campo mediante `trackPoints` y `sampledArea`, mientras que cada observación incorporó sus propias coordenadas.

Durante una sesión activa se registra el recorrido con lecturas periódicas cada tres segundos o cada metro de desplazamiento, según los avisos que proporcione el sistema operativo. Sobre esas lecturas se añadió un filtro específico en el servicio móvil `samplingTrack.ts`. Cada punto se acepta solo si contiene coordenadas válidas, una fecha de registro interpretable, una precisión no superior a 15 metros, una separación mínima de 2,5 metros respecto al punto anterior y una velocidad implícita no superior a 8 m/s. La distancia entre dos puntos se calcula mediante la fórmula de Haversine, por ser suficiente para estimar recorridos cortos sobre coordenadas geográficas sin introducir una dependencia geoespacial adicional. Estos umbrales no proceden de una norma externa, sino de una decisión de diseño del proyecto para reducir saltos espurios del GPS sin perder desplazamientos reales de muestreo.

El backend repite parte de esta limpieza antes de persistir el progreso de la sesión. En `apps/api/src/modules/sessions/sessions.service.ts` se normalizan los puntos, se ordenan temporalmente, se eliminan duplicados y se vuelven a descartar lecturas no plausibles. En servidor se usa una tolerancia algo más amplia para no rechazar datos ya capturados en campo: precisión máxima de 50 metros, separación mínima de 1,5 metros y la misma velocidad máxima de 8 m/s. Además, los esquemas compartidos limitan los recorridos a un máximo de 5000 puntos por actualización y validan latitud, longitud y precisión.

El área muestreada se genera a partir del recorrido filtrado. Para muestreo libre se construye una envolvente rectangular con un margen de 3 metros; para transectos se usa como margen la mitad de la anchura del transecto, con 5 metros como valor por defecto; y para cuadrículas se usa la mitad del tamaño de celda, con 10 metros como valor por defecto, ajustando después los límites a pasos de celda. Esta geometría se guarda como

`sampledArea`, mientras que los puntos crudos aceptados permanecen en `trackPoints`. Así, el sistema conserva tanto la trayectoria original como una superficie simplificada para mapas, métricas y exportaciones.

También se añadió la captura y selección de fotografías desde cámara o galería. Las imágenes se copiaron al almacenamiento local de la aplicación para poder mantenerlas asociadas a observaciones incluso en escenarios de conectividad variable. La base SQLite local [20] se estructuró mediante repositorios por entidad, incluyendo usuarios, grupos, campañas, sesiones, observaciones, taxonomía, fotografías y cola de sincronización. Esta separación facilitó que la interfaz móvil pudiera revisar, editar o eliminar datos creados localmente antes de su envío al servidor.

Backend y validación científica. En el backend se reforzó el módulo de observaciones para aceptar fotografías, registrar sus metadatos y exponer endpoints de listado, descarga, subida y eliminación. Además, se añadieron reglas de validación sobre taxonomía activa, coordenadas, fechas, cantidades y permisos por grupo. La lógica impidió, por ejemplo, registrar observaciones fuera del intervalo temporal permitido para la sesión o asociarlas a grupos a los que el usuario no tuviera acceso.

La gestión de fotografías se implementó con endpoints específicos para no mezclar la edición de campos científicos de la observación con la transferencia de archivos. El backend recibe las imágenes codificadas en base64, valida que el tipo MIME pertenezca a formatos soportados como JPEG, PNG, WebP, HEIC o HEIF, rechaza imágenes vacías o superiores a 10 MB y limita cada observación a cinco fotografías. Antes de guardar el archivo, normaliza el nombre recibido, calcula un `checksum SHA-256`, lo almacena bajo el directorio de subidas configurado y registra metadatos como tamaño, tipo MIME, dimensiones, origen, fecha de captura, EXIF y usuario que subió la imagen. Si falla la persistencia en base de datos, el archivo físico se elimina para evitar residuos inconsistentes.

El modelo consolidado incorporó fotografías de observación, URLs asociadas a observaciones y campos espaciales de sesión. Aunque estos cambios aparecen en la migración inicial consolidada, su uso funcional corresponde a esta fase de captura avanzada.

Portal web y explotación inicial. El portal web incorporó listados y fichas de observaciones con filtros por campaña, sesión, entorno, observador, fecha y presencia de fotografías. La ficha permitió consultar la información científica y contextual del registro, incluyendo taxonomía, ítems, coordenadas, notas y evidencias fotográficas.

Además, se implementó el cálculo de métricas agregadas en backend y su visualización en el dashboard web. Estas métricas resumieron actividad, observaciones e ítems por dimensiones como campaña, sesión, categoría, material, entorno u observador. El módulo de analítica inicial reutilizó esta información para explorar subconjuntos filtrados y preparar exportaciones posteriores.

Integración entre capas. La aplicación móvil construyó payloads compatibles con los esquemas compartidos y los conservó localmente hasta poder enviarlos. El backend validó y persistió observaciones, ítems y fotografías, y el portal web consumió esos datos para revisión y análisis. De esta forma, el dato capturado en campo comenzó a alimentar directamente la supervisión web y las primeras métricas del sistema.

Cuadro 9.5: Trazabilidad técnica de la implementación del Sprint 2

Bloque implementado	Plataforma	Elementos principales	Finalidad
Localización y recorridos	Móvil, backend y shared	Permisos GPS, alta precisión opcional, filtro por precisión/distancia/velocidad, puntos de track, área muestreada y validación compartida	Registrar contexto espacial fiable de sesiones y observaciones.
Fotografías de observación	Móvil y backend	Captura desde cámara/galería, almacenamiento local, subida y metadatos	Asociar evidencia visual a los registros de residuos.
Persistencia local inicial	Móvil	SQLite, repositorios por entidad y cola de operaciones pendientes	Permitir revisión y trabajo con conectividad variable.
Validación de observaciones	Backend y shared	Taxonomía activa, fechas, coordenadas, cantidades y permisos	Mantener coherencia científica y control de acceso.
Consulta y métricas web	Backend y web	Listado de observaciones, fichas, dashboard y analítica inicial	Convertir la captura de campo en información revisable y agregada.

Al finalizar este sprint, OmniLitter disponía de un flujo de captura más representativo del trabajo de campo: sesiones con recorrido, observaciones con fotografías, persistencia local y primeras herramientas web para revisar y analizar la información registrada.

9.2.3. Pruebas realizadas

El segundo sprint reforzó la captura de datos de campo y el análisis inicial. La validación se orientó a comprobar la calidad del dato capturado, la incorporación de fotografías, el uso de ubicación GPS, el control de permisos sobre observaciones y las primeras funcionalidades de análisis del portal web.

9.2.3.1. Pruebas automatizadas implementadas

Cuadro 9.6: Pruebas automatizadas vinculadas al Sprint 2

Tarea o funcionalidad	Cobertura automatizada	Comprobación principal	Resultado
GPS en sesión y observación	WEB-SES-01 a WEB-SES-03	Cálculo de distancia para rutas vacías, rutas con un único punto y suma de distancia Haversine entre puntos consecutivos.	Correcto
Fotografías por observación y metadatos	SCH-17 a SCH-23	Aceptación de formatos de imagen soportados y rechazo de tipos MIME no permitidos.	Correcto
Validación científica de observaciones	SCH-14 a SCH-16	Existencia de ítems, coordenadas válidas y cantidades positivas antes de aceptar una observación.	Correcto
Permisos de observaciones	OBS-01 a OBS-06	Alcance de observaciones por usuario, visibilidad ampliada para gestores y bloqueo de observaciones ajenas sin permisos.	Correcto
Dashboard de métricas científicas	WEB-EXP-01 a WEB-EXP-12	Formatos soportados, persistencia de preferencias en <code>localStorage</code> y recuperación ante valores obsoletos.	Correcto

La cobertura de este sprint resulta relevante para la calidad científica del sistema. Una observación no debe almacenarse sin ítems, con coordenadas imposibles, con cantidades no positivas o con ficheros de imagen no soportados. Además, las pruebas de permisos aseguran que cada usuario consulta únicamente la información que corresponde a su ámbito de autorización.

9.2.4. Desviaciones e incidencias

En el segundo sprint las incidencias estuvieron relacionadas con la evolución del flujo de campo hacia un funcionamiento más próximo al uso real. La introducción de persistencia local, captura con GPS y fotografías exigió ajustes de configuración y validación adicionales.

Cuadro 9.7: Desviaciones e incidencias del Sprint 2

Tipo	Desviación o incidencia	Causa	Efecto sobre el sprint
Configuración técnica	Mayor esfuerzo inicial en la sincronización y configuración de SQLite.	La persistencia local requería preparar repositorios, almacenamiento y gestión de datos pendientes.	La funcionalidad necesitó más tiempo de configuración del estimado inicialmente.
Validación en dispositivo	Dificultad para probar GPS en tiempo real y fotografías desde el emulador.	El entorno emulado no reproducía adecuadamente algunos comportamientos físicos del dispositivo.	Fue necesario dedicar tiempo adicional a probar el proyecto con Expo Go en un dispositivo real.
Ajuste de permisos	Reestructuración de permisos por roles en el portal web.	Se detectó la necesidad de coordinar de forma más homogénea los permisos entre la web y la aplicación móvil.	Se incorporó el ajuste asociado a BUG.02 , reforzando la consistencia entre plataformas.

El impacto del sprint fue moderado y se concentró en la validación técnica de funcionalidades dependientes del dispositivo. Los ajustes realizados permitieron mejorar la fiabilidad del flujo offline y mantener una política de permisos más uniforme entre las distintas capas del sistema.

9.2.5. Resultado del sprint

El segundo sprint supuso el paso de una base funcional a una herramienta más próxima al trabajo real de campo. Al cierre de la iteración, las observaciones ya no eran registros básicos, sino unidades de información contextualizadas mediante coordenadas, fotografías, metadatos y validaciones más estrictas. Esto aumentó la calidad del dato recogido y reforzó su utilidad para la consulta y el análisis posterior.

La principal conclusión técnica fue que la arquitectura definida en el sprint anterior podía absorber funcionalidades más exigentes sin romper el flujo general. La aplicación móvil ganó capacidad de captura y una primera persistencia local; el backend reforzó su papel como capa de validación y almacenamiento; y el portal web comenzó a actuar como espacio de supervisión de la información generada en campo. Las dificultades con SQLite, ficheros locales y pruebas móviles ayudaron a detectar límites del entorno y a ajustar la solución antes de avanzar hacia la sincronización completa.

En conjunto, el sprint tuvo un cumplimiento adecuado y dejó el sistema en un estado más representativo del uso previsto. OmniLitter pasó a registrar datos más completos, verificables y preparados para explotación, y quedó lista la base necesaria para abordar

en la siguiente iteración el ciclo completo entre trabajo offline, sincronización, visualización espacial y exportación.

Cuadro 9.8: Resumen de horas reales del Sprint 2.

Horas del Sprint 2	
Horas planificadas del Sprint	X
Horas reales (Alejandro Castilla)	X
Horas reales (Ignacio Martínez Díaz)	X
Horas reales totales del Sprint	X
Horas acumuladas planificadas hasta este Sprint	X
Horas acumuladas reales hasta este Sprint	X

9.3. Sprint 3: Sincronización, mapas y exportación científica

9.3.1. Características y objetivos del sprint

El tercer sprint tuvo como objetivo completar el ciclo de trabajo entre la aplicación móvil y el servidor, especialmente en escenarios donde la conectividad no estuviera garantizada. En esta iteración se abordó la sincronización de cambios pendientes, la descarga de información remota, la conservación de datos locales hasta su envío y la gestión de resultados individuales dentro de los procesos de sincronización.

Junto a ello, se incorporaron funcionalidades de visualización geográfica y configuración del usuario. Esto permitió que OmniLitter pasase de ser una herramienta centrada en la captura y consulta básica de datos a ofrecer un flujo más completo de trabajo: registro en campo, persistencia local, sincronización con el backend, representación espacial de la información y adaptación de ciertas preferencias a las necesidades del usuario.

- **Trabajo offline y sincronización II.** Esta funcionalidad continúa el trabajo offline desarrollado previamente, ampliándolo con la sincronización entre la aplicación móvil y el backend cuando vuelve a existir conectividad. Gestiona los cambios pendientes, agrupa operaciones, conserva los datos hasta confirmar su envío y permite que sesiones, observaciones y progresos capturados en campo acaben integrados en el sistema central.
- **Sesiones de muestreo III.** Esta parte funcional continúa la evolución de las sesiones de muestreo, ampliándolas para que puedan modificarse localmente y sincronizarse después con el servidor. Incluye la creación, actualización del progreso y finalización de sesiones, además del uso de rutas y áreas muestreadas como información útil para mapas, análisis y exportaciones.
- **Observaciones III.** Esta funcionalidad continúa la ampliación de las observaciones, centrándose ahora en el paso de los registros creados en el dispositivo al backend. Su objetivo es que los datos capturados sin conexión, junto con sus elementos asociados, queden publicados posteriormente y cada operación de sincronización pueda evaluarse de forma individual.
- **Visualización geográfica.** Esta funcionalidad permite interpretar los datos desde una perspectiva espacial. El mapa representa observaciones, rutas y áreas muestreadas, facilitando la identificación de zonas ya recorridas, puntos de registro y posibles concentraciones de residuos.
- **Analítica y exportación II.** Esta parte funcional continúa la base de analítica y exportación del sprint anterior, ampliándola hacia la reutilización externa de los datos de OmniLitter. Las exportaciones en CSV y GeoJSON permiten trasladar observaciones, resúmenes o información geográfica a herramientas de análisis científico, hojas de cálculo o sistemas cartográficos.

- **Perfil y configuración.** Esta funcionalidad agrupa las opciones que permiten adaptar la aplicación al usuario y al contexto de uso. Incluye datos de perfil, cambio de contraseña, selección de idioma y preferencias de sincronización, de forma que el comportamiento del sistema pueda ajustarse a las condiciones del dispositivo y de la conexión.

9.3.2. Implementación realizada

El tercer sprint se orientó a cerrar el ciclo técnico entre la aplicación móvil, el backend y el portal web. La implementación se centró en sincronización, visualización geográfica, exportación de datos y configuración de preferencias, permitiendo que los registros creados sin conexión pudieran consolidarse y explotarse posteriormente desde el sistema central.

Sincronización móvil-backend. En la aplicación móvil se implementó una pantalla específica de sincronización y un servicio encargado de procesar operaciones pendientes. La base local conservó las acciones en una cola persistente, de forma que sesiones, actualizaciones de progreso, finalizaciones y observaciones pudieran mantenerse hasta recuperar conectividad. También se añadió sincronización automática al volver a estar online, considerando preferencias como la restricción a redes Wi-Fi.

El backend incorporó endpoints de sincronización por lotes y consulta de cambios remotos. El procesamiento por lotes permitió enviar varias operaciones en una misma petición y obtener resultados individuales por elemento, evitando que un fallo puntual invalidase necesariamente el conjunto completo. Esta estructura resultó especialmente adecuada para escenarios de campo, donde los datos pueden acumularse en el dispositivo durante periodos sin conexión.

La sincronización no se planteó como una subida genérica de tablas, sino como una cola de operaciones de captura con tipos explícitos. El contrato compartido define operaciones diferenciadas para crear sesiones, editar sus campos, actualizar su progreso, finalizarlas y crear observaciones, agrupadas en lotes de hasta 500 elementos. La app móvil genera identificadores de cliente para relacionar cada operación enviada con la respuesta del servidor; si una operación se acepta, se marca como sincronizada o se elimina de `sync_outbox`, y si falla se incrementa el contador de intentos y se conserva el último error. Para evitar sobrescribir datos locales pendientes, la descarga de cambios remotos se omite cuando existe trabajo sin sincronizar en el dispositivo.

Mapa y visualización espacial. El portal web incorporó un mapa interactivo basado en React Leaflet [18]. Se implementaron marcadores de observaciones, agrupación visual, filtros por campaña, entorno, categoría y fecha, y carga de fotografías en ventanas de detalle. Además, se añadieron modos de visualización de densidad, recorridos y áreas

muestreadas, utilizando los datos espaciales de sesiones y observaciones persistidos por el backend.

Para las sesiones de muestreo, la API expuso rutas y áreas asociadas al grupo activo. El portal pudo así representar recorridos, superficies muestreadas y zonas de concentración de residuos, conectando la captura móvil con una lectura geográfica de los resultados.

En la exportación geográfica, el mapa construye una `FeatureCollection` GeoJSON con coordenadas en el orden `[longitud, latitud]`. Cuando el usuario exporta desde el modo de rutas, se incluyen primero polígonos de área muestreada, después líneas de recorrido y por último puntos de observación, de forma que los visores GIS puedan interpretar por separado superficies, trayectorias y registros puntuales. Las propiedades de cada elemento incorporan contexto de campaña, sesión, entorno, tipología, observador, método de muestreo y distancia cuando procede.

Analítica, exportaciones y configuración. Se ampliaron las utilidades de exportación del portal web con salidas CSV, JSON y GeoJSON. El dashboard pudo exportar datos agregados, el mapa generó colecciones geográficas de observaciones, rutas y áreas, y la analítica produjo ficheros orientados al tratamiento posterior de observaciones, residuos y resúmenes. En backend, las consultas de analítica incluyeron información de campaña, grupo, sesión, entorno, observador, coordenadas, taxonomía, método de muestreo, duración e ítems.

Las exportaciones CSV se generaron sin depender de librerías externas. El servicio web añade marca BOM UTF-8 para conservar acentos al abrir los ficheros en hojas de cálculo, utiliza punto y coma como separador habitual en entornos españoles y escapa comillas, saltos de línea y separadores. La preferencia de formato de exportación se conserva en `localStorage`, por lo que el usuario puede mantener CSV o JSON como formato por defecto entre sesiones.

En la aplicación móvil se incorporaron preferencias de idioma, sincronización y precisión de ubicación. La infraestructura de internacionalización también se observa en el portal web, aunque parte de la configuración web se completó en el sprint siguiente según la trazabilidad disponible.

Integración entre capas. La app móvil trabajó sobre SQLite, agrupó cambios pendientes y los envió al backend cuando la conectividad lo permitió. La API persistió los cambios en la base central y expuso tanto cambios remotos como datos espaciales y analíticos. El portal web consumió esos datos ya consolidados para mapas, métricas y exportaciones. La integración de este sprint unió captura offline, publicación, consulta y reutilización externa de los datos.

Cuadro 9.9: Trazabilidad técnica de la implementación del Sprint 3

Bloque implementado	Plataforma	Elementos principales	Finalidad
Sincronización batch	Móvil y backend	Cola local, pantalla de sincronización, envío por lotes y resultados individuales	Consolidar en servidor datos generados sin conexión.
Descarga de cambios	Backend y móvil	Consulta de cambios remotos por grupo y actualización de la copia local	Mantener el dispositivo alineado con la información central.
Mapa interactivo	Web y backend	Marcadores, filtros, fotografías, rutas, áreas y densidad	Interpretar observaciones y sesiones desde una perspectiva espacial.
Exportaciones	Web y backend	CSV, JSON, GeoJSON y consultas analíticas filtradas	Facilitar reutilización científica y cartográfica de los datos.
Preferencias	Móvil y web	Idioma, sincronización, Wi-Fi y precisión de ubicación	Adaptar el comportamiento del sistema al contexto de uso.

El resultado técnico del sprint fue una plataforma capaz de soportar el ciclo completo de trabajo: captura local, sincronización controlada, consulta centralizada, representación espacial y exportación en formatos reutilizables.

9.3.3. Pruebas realizadas

El tercer sprint estuvo orientado a sincronización, mapas y exportación científica. En esta fase fue necesario diferenciar con claridad entre las pruebas automatizadas existentes y la validación funcional de integración. La sincronización móvil-backend se comprobó mediante recorridos manuales completos, pero no se implementó una prueba automática de integración contra una base de datos real.

9.3.3.1. Pruebas automatizadas implementadas

Cuadro 9.10: Pruebas automatizadas vinculadas al Sprint 3

Tarea o funcionalidad	Cobertura automatizada	Comprobación principal	Resultado
Exportación CSV y soporte de rutas	WEB-SES-01 a WEB-SES-03	Cálculo de distancia de tracks como soporte para visualización, análisis de recorridos y exportación de información asociada a sesiones.	Correcto
Preferencias de exportación científica	WEB-EXP-01 a WEB-EXP-12	Formatos admitidos, persistencia de la preferencia de exportación y recuperación ante formatos no soportados.	Correcto
Configuración y cambio de contraseña	SCH-03, USR-04 a USR-06	Rechazo de reutilización de contraseña, usuario inexistente, contraseña actual incorrecta y guardado seguro del nuevo hash.	Correcto
Sincronización móvil-backend	Sin suite automatizada específica	La sincronización se validó mediante pruebas manuales de flujo completo. No se atribuye cobertura automatizada inexistente.	Manual

La ausencia de una prueba automatizada específica para sincronización se considera una limitación de la validación automática. No obstante, se documenta de forma explícita para mantener la trazabilidad y evitar que la memoria presente como automatizada una comprobación que realmente fue funcional y manual.

9.3.4. Desviaciones e incidencias

Durante el tercer sprint se produjeron ajustes de planificación derivados de la necesidad de equilibrar el cierre técnico con la calidad visual de la aplicación y la preparación de la documentación. También se detectaron inconsistencias de datos al trabajar con información de ejemplo.

Cuadro 9.11: Desviaciones e incidencias del Sprint 3

Tipo	Desviación o incidencia	Causa	Efecto sobre el sprint
Refinamiento visual	Refactorización estética de la aplicación móvil para alinearla mejor con la identidad del proyecto.	La interfaz móvil necesitaba mayor coherencia visual con los componentes definidos en Figma.	Se reajustó el tiempo disponible para otras tareas del sprint, sin modificar los objetivos funcionales principales.
Replanificación	Aplazamiento de la configuración en la aplicación, incluido el módulo de inglés.	Se priorizó disponer de mayor margen para el cierre de funcionalidades y la redacción de la memoria.	La tarea se trasladó a una fase posterior, dejando constancia del reajuste en la planificación.
Consistencia de datos	Inconsistencias entre sesiones, entornos, tipologías y catálogo utilizado por la aplicación móvil.	Los datos de ejemplo y la información devuelta por la API no estaban completamente alineados.	Fue necesario revisar los datos seed y ajustar la información expuesta por la API para evitar discrepancias.

Estas desviaciones supusieron ajustes de alcance interno y ordenación de tareas, pero no alteraron el propósito general del sprint. El resultado fue una mayor coherencia visual y de datos, junto con una planificación más realista del trabajo pendiente.

9.3.5. Resultado del sprint

El tercer sprint fue el punto de consolidación del ciclo principal de OmniLitter. La plataforma pasó a cubrir un recorrido completo: capturar sesiones y observaciones desde la aplicación móvil, conservarlas localmente cuando no hubiera conexión, sincronizarlas con el backend y consultarlas posteriormente desde el portal web mediante listados, métricas, mapas y exportaciones.

El resultado más relevante fue la integración real entre captura, consolidación y explotación de datos. La sincronización conectó el contexto de trabajo en campo con el entorno de revisión, mientras que la visualización geográfica y la exportación dieron utilidad práctica a la información ya registrada. Con ello, la plataforma dejó de depender de funcionalidades aisladas y empezó a comportarse como un sistema coordinado en torno a un flujo de

uso completo.

Las desviaciones del sprint se concentraron en la coordinación entre datos locales y remotos, el refinamiento visual y la validación del flujo de sincronización. No modificaron el objetivo de la iteración, pero sí dejaron una conclusión importante: el recorrido funcional estaba validado manualmente de extremo a extremo, aunque la sincronización seguía siendo una zona donde sería conveniente incorporar pruebas automatizadas específicas. El sprint cerró con las funcionalidades centrales implementadas y con el trabajo final orientado ya a estabilización, refinamiento y despliegue.

Cuadro 9.12: Resumen de horas reales del Sprint 3.

Horas del Sprint 3	
Horas planificadas del Sprint	X
Horas reales (Alejandro Castilla)	X
Horas reales (Ignacio Martínez Díaz)	X
Horas reales totales del Sprint	X
Horas acumuladas planificadas hasta este Sprint	X
Horas acumuladas reales hasta este Sprint	X

9.4. Sprint 4: Cierre funcional y refinamiento

9.4.1. Características y objetivos del sprint

El cuarto sprint tuvo un carácter principalmente orientado a la consolidación del producto. Una vez implementados los principales flujos funcionales, esta iteración se centró en reforzar funcionalidades ya existentes, cerrar aspectos pendientes detectados durante el desarrollo, mejorar la experiencia de usuario y asegurar una versión más estable y presentable para la validación funcional del proyecto.

En esta etapa se revisaron tanto la administración web como el modelo científico que condiciona campañas, sesiones y observaciones. También se realizaron ajustes de interfaz, limpieza de elementos no utilizados y refinamientos en las validaciones. El objetivo final fue reducir inconsistencias, mejorar la coherencia general del sistema y preparar una versión suficientemente completa para su evaluación.

- **Configuración y administración II.** Esta funcionalidad continúa la organización de usuarios, grupos y permisos, ampliándola con ajustes administrativos necesarios para operar la plataforma con coherencia desde el portal web. Abarca la gestión de usuarios, grupos, membresías y permisos, asegurando que cada rol actúe dentro del alcance que le corresponde.
- **Campañas II.** Esta parte funcional continúa la funcionalidad de campañas, reforzando su papel organizativo dentro del grupo y su relación con las sesiones asociadas. La clasificación científica fina no queda fijada en la campaña, sino en cada sesión de muestreo, lo que permite que una misma campaña agrupe salidas realizadas en entornos o tipologías distintas.
- **Sesiones de muestreo IV.** Esta funcionalidad continúa la evolución de las sesiones de muestreo, ampliándolas como registros editables y validados científicamente. Permite ajustar datos de la salida de campo, asociar el entorno correspondiente y aplicar reglas compartidas con el catálogo para mantener coherencia entre la campaña planificada y el muestreo realizado.
- **Observaciones IV.** Esta parte funcional continúa la mejora progresiva de las observaciones, centrándose en la calidad final de los registros de residuos. Las observaciones pueden revisarse y corregirse manteniendo reglas de coherencia, rangos científicos y datos estructurados que después serán utilizados en consultas, métricas y exportaciones.
- **Taxonomía y catálogo científico.** Esta funcionalidad proporciona las referencias controladas que dan uniformidad a la información científica del sistema. Los catálogos de entornos y tipologías permiten clasificar las sesiones con valores consistentes y reutilizables, mientras que los esquemas compartidos validan taxonomía, coordenadas, cantidades y parámetros de muestreo.

- **Interfaz pública y limpieza final.** Esta parte funcional afecta a la presentación general y al cierre del producto. La interfaz pública actúa como primera pantalla de acceso y presentación de la plataforma, mientras que la limpieza final reduce elementos innecesarios y facilita una validación más clara del sistema desarrollado.

9.4.2. Implementación realizada

El cuarto sprint tuvo un carácter de integración final y refinamiento funcional. La implementación se centró en completar la configuración web, reforzar reglas de permisos, ampliar la administración, permitir edición posterior desde móvil, consolidar el catálogo científico y preparar el sistema para una presentación y despliegue más estables.

Backend, permisos y administración. En backend se reforzaron los módulos de usuarios, grupos, campañas, catálogo, sesiones y observaciones. En usuarios se incorporaron operaciones de actualización de perfil y cambio de contraseña. En grupos se ampliaron las capacidades de administración con archivado, restauración, invitaciones, gestión de membresías y generación de copias de seguridad. También se ajustaron reglas de alcance para diferenciar el acceso de superadministradores, administradores de grupo e investigadores.

Las campañas y métricas pasaron a respetar de forma más precisa el contexto del usuario. Cuando no existe grupo activo, el superadministrador puede operar sobre vistas agregadas, mientras que el investigador mantiene un alcance restringido a sus campañas, sesiones u observaciones. Estas reglas completaron la separación de responsabilidades iniciada en los sprints anteriores.

La copia de seguridad administrativa también quedó implementada como una exportación JSON distinta de la analítica científica. Para un grupo concreto incluye campañas, sesiones, observaciones, ítems, fotografías, miembros e invitaciones, junto con metadatos de exportación y un resumen de totales. El superadministrador dispone además de una copia global de todos los grupos activos, útil como respaldo operativo del estado de la plataforma.

Edición móvil y sincronización de cambios. La aplicación móvil incorporó edición local de sesiones y observaciones ya creadas. Los repositorios locales permitieron actualizar campos editables y coordinar esos cambios con la cola de sincronización. Cuando un registro aún tenía una creación pendiente, las modificaciones se integraron sobre el estado local antes de enviarlo, evitando duplicar operaciones innecesarias. En backend, los servicios de sesiones y observaciones añadieron operaciones de actualización posterior sujetas a permisos, estado y titularidad.

En el caso de las sesiones, la edición posterior distingue entre campos de descripción y campos que afectan al muestreo. Si la sesión no está finalizada, pueden actualizarse

método, tamaño de cuadrícula, anchura de transecto, entorno o tipología, y el backend recalcula el área muestreada a partir del recorrido ya almacenado. En observaciones, la actualización se limita a taxonomía, coordenadas, fecha, notas e ítems; las fotografías se gestionan mediante sus endpoints dedicados para mantener separado el ciclo de vida de archivos y datos estructurados.

Catálogo científico y configuración web. El catálogo de entornos y tipologías se gestionó desde un módulo específico de backend y se consumió desde web y móvil. En el modelo consolidado, la tipología y el entorno se vinculan a la sesión de muestreo, lo que permite clasificar de forma más consistente el contexto real de la salida de campo. Los esquemas compartidos mantuvieron validaciones sobre método de muestreo y parámetros específicos, como tamaño de cuadrícula o anchura de transecto.

En el portal web se añadió la página de configuración de cuenta, con actualización de perfil y cambio de contraseña. Además, se consolidaron vistas de administración de grupos y usuarios, ajustes responsive, traducciones y mejoras de presentación en páginas principales como dashboard, campañas, observaciones, mapa y analítica.

Interfaz pública y despliegue. La página pública del portal se rediseñó como pantalla de presentación de la plataforma. En paralelo, se documentó la configuración de despliegue mediante Docker [4], variables de entorno de producción, PostgreSQL [21], API, web estática y build móvil con EAS [5]. La guía de despliegue también deja constancia de que el almacenamiento local de fotografías requiere almacenamiento persistente o un servicio externo en un entorno productivo como Render [19].

Cuadro 9.13: Trazabilidad técnica de la implementación del Sprint 4

Bloque implementado	Plataforma	Elementos principales	Finalidad
Configuración de cuenta	Backend y web	Perfil, cambio de contraseña y preferencias visibles desde portal	Completar la gestión básica del usuario.
Administración avanzada	Backend y web	Archivado, restauración, invitaciones, membresías, backups y alcance por rol	Reforzar operación y control administrativo de grupos y campañas.
Edición posterior	Backend y móvil	Actualización de sesiones y observaciones, repositorios locales y cola de sincronización	Permitir correcciones coherentes antes o después de sincronizar.
Catálogo científico	Backend, web, móvil y shared	Entornos, tipologías y validaciones de muestreo	Uniformar la clasificación del contexto de las sesiones.
Cierre y despliegue	Web, backend y móvil	Landing pública, limpieza final, Docker, Render y EAS	Preparar una versión más presentable y desplegable del sistema.

El resultado del Sprint 4 fue una versión más integrada del producto, con permisos reforzados, administración ampliada, edición móvil posterior, catálogo científico consolidado, configuración de cuenta, landing pública revisada y documentación de despliegue. La parte relativa a una entidad independiente de rangos de tamaño queda como punto pendiente de confirmación manual.

9.4.3. Pruebas realizadas

El cuarto sprint correspondió al cierre funcional y al refinamiento del sistema. La validación tuvo un carácter principalmente regresivo: se reejecutaron las pruebas automatizadas existentes y se revisaron manualmente los flujos principales del portal web y de la aplicación móvil para comprobar que los cambios finales no introducían errores.

9.4.3.1. Pruebas automatizadas implementadas

Cuadro 9.14: Pruebas automatizadas vinculadas al Sprint 4

Tarea o funcionalidad	Cobertura automatizada	Comprobación principal	Resultado
Página de configuración web	USR-02, USR-04 a USR-06, GRP-01 a GRP-15, CMP-02 a CMP-04, OBS-01 a OBS-06	Permisos por rol, vista agregada de superadministrador, participación en campañas, archivado y restauración de grupos, membresías y alcance de observaciones.	Correcto
Entorno en sesión y rangos científicos	CAT-01 a CAT-08, SCH-08 a SCH-13	Validación de entornos, tipologías, referencias de catálogo y reglas compartidas que condicionan campañas y sesiones.	Correcto
Cierre funcional y refinamiento	Suite completa	Reejecución de las pruebas de API, web y shared para comprobar que los ajustes finales no rompen funcionalidades previamente validadas.	Correcto
Aplicación móvil	Sin suite automatizada específica	Los ajustes visuales y funcionales de la aplicación móvil se validaron mediante recorridos manuales.	Manual

Esta validación permitió comprobar que las reglas principales del sistema seguían funcionando tras los cambios finales. Se mantiene como limitación que la aplicación móvil no dispone de una suite automatizada configurada.

9.4.4. Desviaciones e incidencias

El cuarto sprint tuvo un carácter de cierre y refinamiento, por lo que las desviaciones fueron reducidas. Las incidencias detectadas se vincularon principalmente a la actualización de pruebas tras refactorizaciones realizadas en módulos ya implementados.

Cuadro 9.15: Desviaciones e incidencias del Sprint 4

Tipo	Desviación o incidencia	Causa	Efecto sobre el sprint
Cierre funcional	Desviaciones mínimas durante las refactorizaciones finales.	El sprint estaba orientado a cerrar implementación y consolidar funcionalidades existentes.	No se produjo un impacto relevante sobre el desarrollo previsto.
Pruebas automatizadas	Refactorización de tests unitarios de la API.	Las refactorizaciones posteriores a los módulos desarrollados en el Sprint 3 exigieron adaptar parte de la suite de pruebas.	Aumentó la carga dedicada a validación respecto a la estimación inicial, sin afectar al alcance funcional del cierre.

En términos globales, el sprint mantuvo el plan previsto y las incidencias se gestionaron como parte natural del proceso de estabilización. El principal efecto fue un incremento limitado del esfuerzo de pruebas para asegurar la coherencia de la versión final.

9.4.5. Resultado del sprint

El cuarto sprint cerró el desarrollo incremental con una versión más estable, integrada y preparada para evaluación. Después de haber construido el flujo principal en las iteraciones anteriores, esta fase se centró en consolidar el producto: reforzar administración y permisos, completar la configuración de cuenta, revisar la edición posterior de datos, estabilizar el catálogo científico y preparar la documentación de despliegue.

La conclusión principal de la iteración es que OmniLitter alcanzó un estado coherente como plataforma completa. La aplicación móvil, el backend y el portal web quedaron conectados en un recorrido que abarca captura en campo, persistencia local, sincronización, consulta, visualización geográfica, métricas, exportación y administración. Las pruebas regresivas y los recorridos manuales finales permitieron comprobar que los ajustes de cierre no rompían las funcionalidades ya validadas.

Las incidencias fueron reducidas y propias de una fase de estabilización: refactorizaciones, adaptación de pruebas y correcciones menores de coherencia visual o funcional. El sprint se considera cumplido con un grado alto, ya que permitió transformar el prototipo funcional acumulado durante el desarrollo en una versión final más ordenada, documentada y con una base técnica preparada para futuras ampliaciones.

Cuadro 9.16: Resumen de horas reales del Sprint 4.

Horas del Sprint 4	
Horas planificadas del Sprint	X
Horas reales (Alejandro Castilla)	X
Horas reales (Ignacio Martínez Díaz)	X
Horas reales totales del Sprint	X
Horas acumuladas planificadas hasta este Sprint	X
Horas acumuladas reales hasta este Sprint	X

9.5. Pruebas globales y validación funcional

9.5.1. Pruebas manuales funcionales

Las pruebas manuales realizadas sobre OmniLitter tienen como objetivo validar el comportamiento funcional del sistema desde la perspectiva del usuario final, comprobando que los principales flujos de la aplicación se ejecutan correctamente en condiciones reales de uso. Estas pruebas abarcan tanto el portal web como la aplicación móvil, incluyendo autenticación, gestión de grupos y campañas, creación de sesiones de muestreo, registro de observaciones, funcionamiento sin conexión, sincronización posterior de datos y consulta de la información registrada. Para facilitar su trazabilidad dentro de la memoria, las pruebas se organizan por casos de uso, permitiendo justificar de forma clara qué partes del sistema han sido verificadas.

Cuadro 9.17: Pruebas manuales funcionales agrupadas por casos de uso

Caso de uso	Prueba realizada
CU-01	

Caso de uso	Prueba realizada
Registrarse, autenticarse y gestionar la sesión.	PM-CU01-01 – Registro de nuevo usuario ■ Abrimos el formulario de registro del portal web. ■ Enviamos datos incompletos, correo inválido y contraseña débil, y vimos que el sistema mostraba errores de validación. ■ Registramos una cuenta con nombre, correo único y contraseña válida. ■ Comprobamos que el usuario accedía correctamente a la zona autenticada. PM-CU01-02 – Inicio y cierre de sesión en web y móvil ■ Intentamos iniciar sesión con credenciales incorrectas y vimos que se rechazaban. ■ Iniciamos sesión con credenciales válidas y accedimos a pantallas privadas. ■ Recargamos la web y reabrimos la aplicación para comprobar que la sesión se mantenía. ■ Cerramos sesión y verificamos que ya no era posible acceder a zonas protegidas. PM-CU01-03 – Protección de rutas privadas ■ Abrimos el portal sin sesión activa. ■ Accedimos manualmente a rutas privadas como dashboard, campañas, mapa o configuración. ■ Comprobamos que todas redirigían al acceso público. ■ Iniciamos sesión y vimos que las mismas rutas mostraban los módulos permitidos. PM-CU01-04 – Cambio de contraseña del usuario autenticado en portal web ■ Iniciamos sesión con un usuario existente. ■ Intentamos cambiar la contraseña con contraseña actual incorrecta. ■ Intentamos reutilizar la contraseña vigente como nueva contraseña. ■ Introdujimos contraseña actual correcta y nueva contraseña válida. ■ Cerramos sesión e iniciamos sesión con la nueva contraseña correctamente.
CU-02	

Caso de uso	Prueba realizada
Gestionar el perfil y el grupo activo.	PM-CU02-01 – Consulta y edición del perfil básico ■ Iniciamos sesión con un usuario existente. ■ Abrimos la pantalla de perfil o configuración. ■ Modificamos el nombre visible y guardamos los cambios. ■ Recargamos la pantalla y comprobamos que el nuevo nombre se mantenía. PM-CU02-02 – Selección de grupo activo ■ Iniciamos sesión con un usuario perteneciente a más de un grupo. ■ Abrimos el selector de grupo activo y seleccionamos un grupo. ■ Revisamos campañas, observaciones y métricas asociadas. ■ Cambiamos a otro grupo activo y vimos que los datos y acciones se actualizaban. PM-CU02-03 – Configuración de perfil en web y móvil ■ Abrimos la configuración web y editamos datos del usuario. ■ Guardamos los cambios y comprobamos su persistencia. ■ Abrimos el perfil en móvil y vimos los datos actualizados. ■ Cambiamos el grupo activo desde móvil y revisamos que cambiaban las campañas disponibles.
CU-03	

Caso de uso	Prueba realizada
Administrar grupos, miembros e invitaciones.	PM-CU03-01 – Creación y edición de grupo ■ Iniciamos sesión con un usuario. ■ Creamos un grupo con nombre y descripción. ■ Abrimos el detalle del grupo, editamos sus datos y guardamos los cambios. ■ Comprobamos que el grupo aparecía actualizado en el listado. PM-CU03-02 – Gestión de miembros y roles de grupo ■ Añadimos un usuario existente como miembro de un grupo, desde una cuenta de superadministrador. ■ Cambiamos su rol y vimos que las acciones disponibles se ajustaban al nuevo permiso. ■ Expulsamos al miembro del grupo. ■ Y comprobamos que ya no existía en el grupo. ■ Ingresamos como un usuario normal para gestionar un grupo del que somos administradores. ■ Dimos permisos de administrador a un usuario del grupo. ■ Comprobamos que no se le puede cambiar el rol a un administrador de grupo. PM-CU03-03 – Invitación y aceptación desde la app móvil ■ Enviamos una invitación a un usuario desde el portal web. ■ Iniciamos sesión con el usuario invitado. ■ Nos dirigimos a las invitaciones pendientes y aceptamos la invitación. ■ Comprobamos que el nuevo grupo quedaba disponible para trabajar. ■ Del mismo modo lo probamos en la aplicación móvil. PM-CU03-04 – Archivo, restauración y eliminación de grupos ■ Iniciamos sesión como superadministrador o administrador de grupo. ■ Abrimos el detalle de un grupo. ■ Archivamos el grupo y comprobamos que sus campañas activas quedaban cerradas o no disponibles. ■ Restauramos el grupo. ■ Intentamos eliminar definitivamente un grupo activo y vimos que el sistema respetaba permisos y estados.
CU-04	

Caso de uso	Prueba realizada
Administrar usuarios de la plataforma.	PM-CU04-01 – Alta administrativa de usuario ■ Iniciamos sesión como superadministrador. ■ Creamos un usuario con nombre, correo, contraseña temporal y rol global. ■ Lo asociamos a uno o varios grupos durante el alta. ■ Comprobamos que aparecía en el listado con sus datos y membresías. PM-CU04-02 – Cambio de rol y estado de cuenta ■ Abrimos el detalle de un usuario existente. ■ Desactivamos la cuenta y comprobamos que no podía iniciar sesión. ■ Reactivamos la cuenta y modificamos el rol global. ■ Vimos que los módulos disponibles se ajustaban al nuevo rol. PM-CU04-03 – Restricción de administración global ■ Iniciamos sesión con un usuario sin rol de superadministrador. ■ Intentamos acceder a la administración global de usuarios. ■ Comprobamos que las operaciones de superadministrador desaparecen para perfiles no autorizados.
CU-05	

Caso de uso	Prueba realizada
Crear y gestionar campañas de muestreo.	PM-CU05-01 – Creación de campaña desde el portal ■ Iniciamos sesión como usuario. ■ Seleccionamos un grupo activo del que eramos administrador y abrimos el módulo de campañas. ■ Creamos una campaña con nombre, descripción, estado y fechas. ■ Comprobamos que la campaña quedaba asociada al grupo activo, y así le aparecía a los demás miembros del grupo. PM-CU05-02 – Edición, filtrado y eliminación de campaña ■ Filtramos el listado de campañas por texto y estado. ■ Comparamos que se listaban las campañas deseadas. ■ Editamos una campaña existente y guardamos cambios válidos, comprobando que se guardaban correctamente. ■ Intentamos guardar fechas incoherentes y vimos que se rechazaban. ■ Eliminamos una campaña no necesaria para otras pruebas, comprobando que se eliminaba para todos los usuarios. PM-CU05-03 – Disponibilidad de campaña en móvil ■ Iniciamos sesión en móvil con un miembro administrador de grupo. ■ Creamos una campaña para dicho grupo. ■ Comprobamos que se creaba correctamente. PM-CU05-04 – Creación móvil de campaña con catálogo ■ Iniciamos sesión en móvil con permisos de administración sobre el grupo activo. ■ Creamos una nueva campaña desde la aplicación. ■ Seleccionamos entorno del catálogo, tipología, fechas y estado. ■ Guardamos la campaña. ■ Verificamos que aparecía correctamente en móvil y en el portal web.
CU-06	

Caso de uso	Prueba realizada
Crear y actualizar una sesión de muestreo.	PM-CU06-01 – Creación de sesión móvil asociada a campaña ■ Iniciamos sesión en móvil con un usuario del grupo activo. ■ Abrimos una campaña activa y accedimos a sus sesiones. ■ Creamos una sesión. ■ Guardamos la sesión y comprobamos que quedaba asociada a campaña, grupo y observador. PM-CU06-02 – Captura de coordenadas iniciales ■ Activamos la localización del dispositivo o simulador. ■ Creamos una sesión nueva y concedimos permiso de ubicación. ■ Abrimos el detalle o mapa de la sesión. ■ Comprobamos que la sesión almacenaba coordenadas iniciales. PM-CU06-03 – Métodos de muestreo y parámetros específicos ■ Creamos sesiones con método libre, cuadrícula y transecto. ■ Hicimos recorridos para trazar las sesiones ■ Vimos que se muestreaban correctamente según el método escogido. PM-CU06-04 – Registro de recorrido durante la sesión ■ Abrimos una sesión activa en móvil. ■ Mantuvimos la sesión abierta mientras cambiaba la ubicación real o simulada. ■ Revisamos el mapa o resumen de la sesión. ■ Comprobamos que se añadían puntos de recorrido, duración y progreso. PM-CU06-05 – Edición de sesión existente ■ Creamos una sesión de muestreo en móvil. ■ Abrimos el detalle de la sesión. ■ Editamos campos permitidos como nombre, notas, entorno o parámetros de muestreo. ■ Guardamos los cambios y volvimos a abrir la sesión. ■ Comprobamos que se mantenía su asociación a campaña, grupo y observador. PM-CU06-06 – Entorno y rangos científicos en sesión ■ Creamos o editamos una sesión móvil. ■ Seleccionamos un entorno válido e introducimos parámetros científicos compatibles. ■ Intentamos guardar valores fuera de rango. ■ Corregimos los valores y comprobamos que la sesión conservaba datos válidos.
CU-07	

Caso de uso	Prueba realizada
Registrar observaciones de residuos.	PM-CU07-01 – Registro básico de observación ■ Abrimos una sesión activa propia en móvil. ■ Intentamos guardar una observación sin seleccionar residuo y vimos la validación. ■ Seleccionamos un residuo de la taxonomía y rellenamos el resto del formulario. ■ Guardamos la observación y comprobamos que quedaba dentro de la sesión. PM-CU07-02 – Edición y borrado de observación ■ Seleccionamos una observación. ■ Editamos los detalles de la misma y guardamos dichos cambios. ■ Abrimos el detalle de la observación. ■ Comprobamos que los cambios se habían guardado. ■ Por último, seleccionamos la eliminación de la observación, comprobando que se realizaba correctamente. PM-CU07-04 – Observación con fotografía ■ Abrimos el formulario de nueva observación. ■ Seleccionamos un residuo, cantidad, material y notas. ■ Añadimos fotografías desde cámara y galería. ■ Guardamos la observación y comprobamos que las imágenes quedaban asociadas y eran accesibles. PM-CU07-05 – Límite y permisos de fotografías ■ Denegamos temporalmente el permiso de cámara o galería. ■ Intentamos adjuntar una imagen y vimos que la aplicación informaba del permiso denegado. ■ Concedimos permisos y adjuntamos imágenes correctamente. ■ Intentamos superar el límite máximo y comprobamos que la aplicación lo bloqueaba. PM-CU07-06 – Validación y permisos de observaciones ■ Creamos una observación como investigador de campo. ■ Comprobamos la consulta con propietario, administrador de grupo y usuario de otro grupo. ■ Vimos que los permisos permitían consultar solo el alcance correspondiente. PM-CU07-07 – Edición de observación existente ■ Creamos una observación con taxonomía, cantidad, notas y fotografía. ■ Abrimos el detalle de la observación. ■ Editamos campos permitidos, como cantidad o notas. ■ Guardamos los cambios y sincronizamos cuando procedía. ■ Revisamos en web que el detalle reflejaba la versión actualizada. PM-CU07-08 – Rangos de tamaño científicos ■ Abrimos el formulario de observación. ■ Seleccionamos un residuo y un rango de tamaño válido. ■ Guardamos la observación.

Caso de uso	Prueba realizada
CU-08 Revisar y finalizar una sesión de muestreo.	PM-CU08-01 – Revisión de observaciones antes del cierre ■ Abrimos una sesión activa con observaciones registradas. ■ Revisamos el listado y el detalle de las observaciones. ■ Comprobamos taxonomía, cantidad, coordenadas, notas y fotografías. ■ Volvimos al resumen de sesión y verificamos que la información seguía disponible. PM-CU08-02 – Eliminación o corrección antes de sincronizar ■ Creamos una observación en una sesión activa. ■ Abrimos su detalle desde móvil. ■ Eliminamos una fotografía y después la observación completa. ■ Confirmamos la acción y vimos que desaparecía del listado de la sesión. PM-CU08-03 – Finalización de sesión ■ Abrimos una sesión activa y revisamos sus observaciones. ■ Confirmamos la finalización de la sesión. ■ Comprobamos que se guardaba la fecha u hora de cierre y el progreso.
CU-09	

Caso de uso	Prueba realizada
Trabajar sin conexión y sincronizar datos.	PM-CU09-01 – Login offline y consulta de datos cacheados ■ Iniciamos sesión online en móvil y abrimos perfil, grupos y campañas para poblar la caché. ■ Cerramos sesión, desactivamos la conectividad y volvimos a iniciar sesión. ■ Abrimos campañas y sesiones cacheadas. ■ Comprobamos que el acceso offline funcionaba para datos ya disponibles en el dispositivo. PM-CU09-02 – Creación local de datos pendientes ■ Trabajamos en móvil sin conexión. ■ Creamos una sesión y una observación dentro de una campaña cacheada. ■ Adjuntamos una fotografía cuando fue posible. ■ Abrimos sincronización y vimos que los elementos quedaban marcados como pendientes. PM-CU09-03 – Sincronización manual de cola pendiente ■ Creamos una sesión y una observación offline. ■ Recuperamos conectividad y abrimos la pantalla de sincronización. ■ Pulsamos la acción de sincronizar. ■ Comprobamos que el contador de pendientes bajaba. ■ Verificamos en web que la sesión y la observación aparecían. PM-CU09-04 – Restricción de sincronización por Wi-Fi ■ Activamos la opción de sincronizar solo con Wi-Fi. ■ Generamos datos pendientes y simulamos una condición sin Wi-Fi. ■ Intentamos sincronizar y vimos que la aplicación lo bloqueaba. ■ Restablecimos una condición válida y la sincronización se completó. PM-CU09-05 – Descarga de cambios remotos ■ Creamos o modificamos una campaña desde el portal web. ■ Abrimos la aplicación móvil con un usuario de dicha campaña. ■ Establecimos conexión. ■ Comprobamos que los cambios remotos aparecían en el listado móvil.
CU-10	

Caso de uso	Prueba realizada
Consultar sesiones y observaciones.	PM-CU10-01 – Listado web de observaciones ■ Sincronizamos una observación creada desde móvil. ■ Abrimos el módulo de observaciones en el portal web. ■ Filtramos por campaña, sesión, entorno, observador, fecha y presencia de fotos. ■ Abrimos el detalle y comprobamos que los datos se mostraban correctamente. PM-CU10-02 – Consulta según permisos ■ Creamos observaciones con dos usuarios del mismo grupo. ■ Accedimos como investigador, administrador de grupo y usuario de otro grupo. ■ Comprobamos que el listado y el detalle respetaban el alcance autorizado. PM-CU10-03 – Consulta tras edición móvil ■ Editamos una sesión o una observación desde móvil. ■ Sincronizamos los cambios. ■ Abrimos el portal web y consultamos el registro afectado. ■ Aplicamos filtros para localizarlo. ■ Comprobamos que los valores editados eran los vigentes.
CU-11 Visualizar la información en mapa.	PM-CU11-01 – Mapa de observaciones ■ Creamos observaciones con coordenadas válidas y las sincronizamos. ■ Abrimos el módulo de mapa en web. ■ Seleccionamos el modo de marcadores. ■ Abrimos el popup de una observación. ■ Comprobamos que mostraba datos básicos y fotografía cuando existía. PM-CU11-02 – Densidad, rutas y áreas muestreadas ■ Abrimos el mapa con sesiones que tenían recorrido. ■ Cambiamos entre modos de densidad y rutas. ■ Aplicamos filtros por campaña, entorno, categoría y fecha. ■ Revisamos que puntos, rutas y áreas se representaban de forma coherente.
CU-12	

Caso de uso	Prueba realizada
Consultar métricas y analítica.	PM-CU12-01 – Dashboard de métricas agregadas ■ Abrimos el dashboard con un grupo activo. ■ Revisamos totales de campañas, sesiones, observaciones, residuos y fotos. ■ Aplicamos filtros temporales y comparamos los totales antes y después. ■ Vimos que las métricas se recalculaban según el filtro aplicado. PM-CU12-02 – Analítica filtrada por campaña, sesión y residuo ■ Abrimos el módulo de analítica. ■ Filtramos por campaña, sesión, entorno, observador y fecha. ■ Revisamos rankings de materiales o categorías. ■ Comprobamos que el panel reflejaba el subconjunto seleccionado. PM-CU12-03 – Métricas respetando grupo activo y permisos ■ Iniciamos sesión con un usuario perteneciente a varios grupos. ■ Seleccionamos un primer grupo activo y revisamos dashboard y analítica. ■ Cambiamos de grupo activo y volvimos a revisar las métricas. ■ Repetimos la comprobación con distintos perfiles. ■ Vimos que los cálculos respetaban permisos y grupo activo. PM-CU12-04 – Recálculo tras edición o eliminación ■ Registramos varias observaciones en una sesión. ■ Revisamos las métricas del dashboard. ■ Editamos o eliminamos una observación desde móvil. ■ Sincronizamos los cambios. ■ Volvimos a dashboard y analítica, y vimos que totales, rankings y gráficos cambiaban de forma coherente.
CU-13	

Caso de uso	Prueba realizada
Exportar datos.	PM-CU13-01 – Exportación desde dashboard ■ Abrimos el dashboard web. ■ Aplicamos un rango de fechas. ■ Exportamos los datos en CSV y JSON. ■ Abrimos los ficheros y comprobamos que contenían datos coherentes con los filtros. PM-CU13-02 – Exportación científica desde analítica ■ Abrimos analítica y aplicamos filtros por campaña, sesión o categoría. ■ Exportamos observaciones, residuos y resúmenes. ■ Revisamos que los CSV incluían columnas y datos interpretables. ■ Comprobamos que los ficheros respetaban los filtros activos. PM-CU13-03 – Exportación GeoJSON desde mapa ■ Abrimos el mapa y aplicamos filtros. ■ Exportamos GeoJSON. ■ Abrimos el fichero generado. ■ Comprobamos que contenía una <code>FeatureCollection</code> coherente con los datos visibles.
CU-14	

Caso de uso	Prueba realizada
Configurar preferencias de la aplicación.	PM-CU14-01 – Cambio de idioma ■ Abrimos la configuración en web o móvil. ■ Cambiamos el idioma de español a inglés. ■ Navegamos por varias pantallas y vimos que los textos principales cambiaban. ■ Volvimos a español y comprobamos que la preferencia se mantenía. PM-CU14-02 – Preferencias de sincronización y localización ■ Abrimos la configuración móvil. ■ Activamos y desactivamos sincronización automática, solo Wi-Fi y ubicación de alta precisión. ■ Cerramos y reabrimos la configuración. ■ Comprobamos que los valores se conservaban y afectaban al comportamiento de la aplicación. PM-CU14-03 – Configuración web y catálogo científico ■ Iniciamos sesión como superadministrador. ■ Abrimos la configuración web. ■ Creamos un entorno de catálogo y una tipología asociada. ■ Intentamos crear duplicados y comprobamos que se rechazaban. ■ Editamos y eliminamos registros no usados, validando permisos y referencias en uso.

9.5.2. Matriz de trazabilidad

La matriz siguiente relaciona los casos de uso consolidados con las pruebas manuales descritas. La relación se limita a las pruebas definidas para cada caso de uso, aunque durante su ejecución se hayan utilizado datos o flujos pertenecientes a otros casos ya validados.

Cuadro 9.18: Matriz de trazabilidad entre casos de uso y pruebas manuales

PM/CU	01	02	03	04	05	06	07	08	09	10	11	12	13	14
PM-CU01-01	X													
PM-CU01-02	X													
PM-CU01-03	X													
PM-CU01-04	X													
PM-CU02-01		X												
PM-CU02-02		X												
PM-CU02-03		X												
PM-CU03-01			X											
PM-CU03-02			X											
PM-CU03-03			X											
PM-CU03-04			X											
PM-CU04-01				X										

PM/CU	01	02	03	04	05	06	07	08	09	10	11	12	13	14
PM-CU04-02				X										
PM-CU04-03				X										
PM-CU05-01					X									
PM-CU05-02					X									
PM-CU05-03					X									
PM-CU05-04					X									
PM-CU06-01						X								
PM-CU06-02						X								
PM-CU06-03						X								
PM-CU06-04						X								
PM-CU06-05						X								
PM-CU06-06						X								
PM-CU07-01							X							
PM-CU07-02							X							
PM-CU07-03							X							
PM-CU07-04							X							
PM-CU07-05							X							
PM-CU07-06							X							
PM-CU07-07							X							
PM-CU07-08							X							
PM-CU08-01								X						
PM-CU08-02								X						
PM-CU08-03								X						
PM-CU09-01									X					
PM-CU09-02									X					
PM-CU09-03									X					
PM-CU09-04									X					
PM-CU09-05									X					
PM-CU10-01										X				
PM-CU10-02										X				
PM-CU10-03										X				
PM-CU11-01											X			
PM-CU11-02											X			
PM-CU12-01												X		
PM-CU12-02												X		
PM-CU12-03												X		
PM-CU12-04												X		
PM-CU13-01													X	
PM-CU13-02													X	
PM-CU13-03													X	
PM-CU14-01														X
PM-CU14-02														X
PM-CU14-03														X

10. Despliegue y operación

Este capítulo describe cómo se pone en marcha OmniLitter y en qué entornos se ejecuta. El sistema se ha pensado para recorrer tres entornos con propósitos distintos: un entorno local para el desarrollo y las pruebas, un entorno de preproducción accesible desde Internet para validar el sistema completo y mostrarlo desplegado tanto a nuestro tutor como al cliente, y un entorno de producción estable para la entrega final. Los tres comparten las mismas imágenes de contenedor y la misma configuración basada en variables de entorno, lo que reduce las diferencias entre ejecutar el sistema en una máquina de desarrollo y hacerlo en la nube.

10.1. Estrategia de despliegue por entornos

La separación en entornos permite trabajar sin riesgo sobre el código, validar los cambios en un servidor real antes de la entrega y reservar la producción para una versión estable. La tabla 10.1 resume cada entorno.

Cuadro 10.1: Entornos de despliegue de OmniLitter.

Entorno	Plataforma	Propósito y componentes
Local (desarrollo)	Docker Compose o Node en local	Desarrollo y pruebas en la máquina del equipo. Levanta PostgreSQL, la API y el portal web, y la aplicación móvil se ejecuta en un emulador o dispositivo.
Preproducción	Render	Validación e integración sobre un servidor accesible desde Internet, usado también para demostraciones. Aloja la API, el portal web y una base de datos PostgreSQL gestionada.
Producción	Microsoft Azure	Versión estable de la entrega final. Ejecuta los contenedores de la API y del portal web junto con una base de datos PostgreSQL gestionada.

10.2. Entorno local de desarrollo

El entorno local es el punto de partida para cualquier persona que quiera ejecutar el proyecto. La instalación paso a paso, con sus requisitos previos para macOS, Linux y

Windows, se detalla en el anexo B. Aquí se describe cómo está organizado.

10.2.1. Orquestación con Docker Compose

La forma recomendada de levantar el sistema en local es mediante Docker Compose, que arranca los tres servicios con una sola orden (`pnpm docker:up`). El fichero `docker-compose.yml` define:

- **PostgreSQL 16**, que expone el puerto 5433 del equipo para no chocar con una instalación local en el 5432, y conserva los datos en el volumen `postgres_data`.
- **La API**, construida desde `docker/api.Dockerfile`, accesible en el puerto 3000. Las imágenes subidas se guardan en el volumen `api_uploads`.
- **El portal web**, construido desde `docker/web.Dockerfile` y servido por Nginx (configurado en `docker/nginx.conf`) en el puerto 8080.

Esta opción no requiere instalar PostgreSQL en la máquina y reproduce una topología parecida a la de los entornos en la nube, lo que reduce las sorpresas al desplegar.

10.2.2. Ejecución sin contenedores

Durante el desarrollo activo resulta cómodo trabajar con recarga en caliente. Para ello se puede levantar únicamente la base de datos en Docker (`docker compose up postgres -d`), aplicar las migraciones y los datos semilla (`pnpm db:migrate` y `pnpm db:seed`) y arrancar los servicios en modo desarrollo (`pnpm dev`). En esta modalidad el portal web se sirve con Vite en el puerto 5173 y la API en el 3000. Cada paquete puede arrancarse también por separado mediante los filtros de pnpm.

10.2.3. Aplicación móvil y emulador

La aplicación móvil utiliza Expo en *bare workflow* con `expo-dev-client`, por lo que Expo Go no es compatible y es necesario compilar el cliente nativo. La primera ejecución compila e instala la aplicación en un emulador o dispositivo mediante `expo run:android` o `expo run:ios`, y en arranques posteriores basta con iniciar el servidor de desarrollo. La variable `EXPO_PUBLIC_API_URL` indica a la aplicación la dirección de la API, que debe ser la IP local de la máquina cuando se usa un dispositivo físico.

Para probar la aplicación en un dispositivo real sin depender del servidor de desarrollo, se genera un APK de Android con EAS a partir del perfil `preview`, cuya distribución es interna y se instala directamente en el dispositivo. La dirección de la API que utiliza ese APK se fija al compilar mediante la variable `EXPO_PUBLIC_API_URL`, de modo que el mismo procedimiento permite construir una versión que apunte a la API local, a la de

preproducción o a la de producción. Los comandos concretos para generar el APK se recogen en el anexo B.

10.3. Preproducción en Render

El entorno de preproducción está desplegado en Render y permite usar el sistema completo desde Internet sin instalar nada. La API y el portal web se publican como servicios independientes a partir del mismo repositorio, y se apoyan en una base de datos PostgreSQL gestionada por la propia plataforma. Las direcciones públicas son:

- API: <https://omnilitter-api.onrender.com>
- Portal web: <https://omnilitter-web.onrender.com>

Este entorno cumple dos funciones. Por un lado, sirve para validar que el sistema funciona igual en un servidor real que en local, incluyendo la comunicación entre el portal, la API y la base de datos sobre HTTPS. Por otro, ofrece un punto de acceso estable para las demostraciones y para que la aplicación móvil se conecte a una API real sin depender de la máquina de desarrollo.

Para la aplicación móvil se generó además un APK con `EXPO_PUBLIC_API_URL` apuntando a la API de Render. Este APK se instala en cualquier dispositivo Android y trabaja contra el entorno de preproducción sin necesidad de configurar nada en local, lo que resulta práctico para mostrar el sistema al tutor y al cliente.

10.4. Producción en Azure

El entorno de producción se despliega en Microsoft Azure y aloja la versión estable destinada a la entrega final. Las imágenes de contenedor de la API y del portal web, generadas por la integración continua, se ejecutan como servicios gestionados, mientras que la base de datos se aloja en una instancia gestionada de PostgreSQL y los archivos subidos se conservan en almacenamiento persistente.

Reservar un entorno propio para producción permite mantener una versión estable e independiente del trabajo de desarrollo y de las pruebas de preproducción. La elección de Azure responde, además, a su disponibilidad dentro del entorno académico y a su integración con los flujos de contenedores ya utilizados en el proyecto.

Del mismo modo que en preproducción, la aplicación móvil dispone de un APK con `EXPO_PUBLIC_API_URL` apuntando a la API de producción, de manera que la versión instalable de la entrega final trabaja directamente contra el entorno de Azure.

10.5. Integración y entrega continua

El repositorio incorpora varios flujos de trabajo de GitHub Actions que automatizan la validación y la publicación de artefactos. La tabla 10.2 los resume.

Cuadro 10.2: Flujos de integración y entrega continua.

Flujo	Disparador	Función
<code>ci.yml</code>	<i>push</i> y <i>pull request</i> a <code>develop</code> y <code>main</code>	Instala dependencias, ejecuta el análisis estático, genera el cliente de Prisma y lanza la compilación y los tests del monorepo.
<code>docker.yml</code>	<i>push</i> a <code>main</code> y ejecución manual	Construye las imágenes de la API y del portal web y las publica en el registro de contenedores de GitHub (GHCR).
<code>expo-preview.yml</code>	Ejecución manual	Genera una versión de previsualización de Android con EAS para probar la aplicación en un dispositivo real.
<code>release.yml</code>	Etiqueta <code>v*</code>	Crea la <i>release</i> en GitHub asociada a la versión etiquetada.

De esta forma, cada cambio que llega a las ramas principales se valida de forma automática antes de integrarse, y las imágenes de contenedor que consumen los entornos de nube se generan de manera reproducible a partir del código fuente.

10.6. Configuración y gestión de secretos

La configuración del sistema se centraliza en variables de entorno, tomando como plantilla el fichero `.env.example`. Este enfoque evita escribir credenciales en el código y permite que cada entorno use sus propios valores sin modificar la aplicación. Las variables principales son:

- `DATABASE_URL`: cadena de conexión a PostgreSQL.
- `JWT_ACCESS_SECRET` y `JWT_REFRESH_SECRET`, junto con sus tiempos de expiración, para la firma de los *tokens* de acceso y refresco.
- `UPLOADS_DIR`: ruta donde la API almacena las fotografías subidas.
- Parámetros `SMTP_*`: configuración del correo saliente usado en la recuperación de contraseña.
- `VITE_API_URL` y `EXP0_PUBLIC_API_URL`: dirección de la API que utilizan el portal web y la aplicación móvil.

En local estos valores se toman del fichero `.env`, mientras que en Render y en Azure se definen como secretos y variables propias de cada plataforma. Los entornos de nube se

sirven siempre sobre HTTPS, de modo que las credenciales y los datos viajan cifrados en tránsito.

10.7. Limitaciones operativas

El despliegue cubre las necesidades del proyecto, pero conviene dejar constancia de sus límites. La operación no incluye una estrategia de escalado horizontal probada ni pruebas de carga, ya que el volumen previsto es el propio de un Trabajo de Fin de Grado. Las copias de seguridad y la monitorización se apoyan en las funciones que ofrecen las plataformas gestionadas, sin un sistema de observabilidad propio. La aplicación móvil se distribuye como versión de previsualización para Android y no se publica en las tiendas oficiales, tal como se indica en el alcance del proyecto. Por último, el mantenimiento evolutivo posterior a la entrega queda fuera del ámbito académico del trabajo.

11. Conclusiones y trabajo futuro

Este capítulo cierra la memoria. Revisa en qué medida se han cumplido los objetivos planteados al inicio del proyecto, reconoce las limitaciones del sistema entregado, recoge las lecciones aprendidas durante el desarrollo y apunta las líneas por las que OmniLitter podría seguir evolucionando.

11.1. Cumplimiento de objetivos

El objetivo general del proyecto era diseñar, desarrollar e implementar una plataforma que facilitara la recogida y el análisis posterior de datos de monitorización de residuos de forma sencilla, eficiente y precisa. El sistema entregado cumple ese objetivo: integra una aplicación móvil de captura, un portal web de gestión y análisis, y una API central con base de datos PostgreSQL, y está desplegado y accesible desde Internet.

Los objetivos específicos también se han alcanzado. La arquitectura integra los tres componentes alrededor de una única API, lo que evita duplicar la lógica de negocio. La aplicación móvil permite trabajar por sesiones de muestreo, registrar observaciones clasificadas con una taxonomía jerárquica, asociar fotografías y capturar la localización mediante GPS. El funcionamiento sin conexión y la sincronización diferida, que eran el requisito más delicado del proyecto, quedan resueltos mediante una base de datos local y una cola de salida que sube los datos pendientes al recuperar cobertura.

Por el lado del portal web, se cubren la gestión de usuarios, grupos, campañas y sesiones, la consulta de observaciones con filtros, el mapa interactivo, el dashboard de métricas y la exportación científica en CSV, JSON y GeoJSON. La información se centraliza en una base de datos común con control de integridad y aislamiento por grupo. Para cerrar el ciclo de ingeniería del software, el proyecto se ha desarrollado en un monorepo con tipos y validaciones compartidas, pruebas automatizadas sobre las partes críticas e integración continua.

11.2. Limitaciones

El sistema cubre el alcance previsto, pero conviene dejar constancia de sus límites.

La aplicación móvil se validó mediante pruebas manuales funcionales y no dispone de una suite de pruebas automatizadas, a diferencia de la API, el portal web y el paquete compartido. La sincronización entre la aplicación y el servidor se comprobó con recorridos manuales completos, pero no cuenta con una prueba de integración automática contra una base de datos real.

La resolución de conflictos sigue una estrategia de *last-write-wins* con granularidad de registro. Es suficiente para el modelo de uso de OmniLitter, donde cada sesión pertenece a un único observador, pero no contempla la edición concurrente intensa del mismo dato por varios usuarios. En cuanto al despliegue, no se han realizado pruebas de carga ni se ha configurado un escalado horizontal, porque el volumen previsto es el propio de un Trabajo de Fin de Grado.

Por último, hay funcionalidades que se dejaron fuera del alcance de forma consciente: la identificación automática de residuos a partir de imágenes, la publicación en las tiendas de aplicaciones y el soporte de más idiomas además del español y el inglés. El mantenimiento evolutivo posterior a la entrega tampoco forma parte del ámbito académico del trabajo.

11.3. Lecciones aprendidas

La lección más clara es que los requisitos de un proyecto real no se cierran en una única fase inicial. La definición evolucionó a partir de las reuniones con el cliente y de la validación con prototipos, y fueron precisamente los mockups los que permitieron transformar necesidades generales en requisitos concretos y detectar cambios antes de escribir código.

En lo técnico, el trabajo sin conexión y la sincronización resultaron ser la parte más costosa del desarrollo. Garantizar que no se pierden datos ante cortes de red o cierres de la aplicación obligó a diseñar con cuidado la cola de salida, los estados de sincronización y la generación de identificadores en el cliente. El uso de un monorepo con esquemas de validación compartidos entre la aplicación, el portal y la API ayudó a mantener la coherencia de los datos y a reducir errores en la frontera entre cliente y servidor.

Trabajar para un cliente del dominio científico también enseñó a valorar los metadatos. Una observación no vale solo por el residuo que registra, sino por el contexto que la acompaña, y esa idea condicionó buena parte del modelo de datos y de los protocolos de muestreo.

11.4. Líneas de trabajo futuro

OmniLitter deja una base sobre la que es natural seguir construyendo. Entre las posibles líneas de trabajo futuro destaca la incorporación de técnicas de visión artificial para identificar automáticamente residuos a partir de las fotografías registradas, la publicación de la aplicación en las tiendas oficiales con una versión de producción para iOS y Android,

una resolución de conflictos más fina con detección por campo, el soporte de más idiomas y la incorporación de una suite de pruebas automatizadas para la aplicación móvil y de pruebas de integración para la sincronización.

Más allá de la evolución técnica, el proyecto tiene recorrido real. En la última reunión de validación, en la que se mostró a Daniel el funcionamiento completo del sistema, el cliente manifestó su intención de presentar OmniLitter a otros compañeros para proponer su uso en futuros proyectos, y de plantear su aplicación en un proyecto real de la Unión Europea contando con el equipo de desarrollo para su mantenimiento posterior. Es una valoración que confirma que la plataforma responde a una necesidad real y que abre una posible continuidad del trabajo más allá del ámbito académico.

Bibliografía

- [1] Android Developers. Build an offline-first app. <https://developer.android.com/topic/architecture/data-layer/offline-first>.
- [2] Colin McDonnell and Zod contributors. Zod documentation. <https://zod.dev>.
- [3] Kyzer R. Davis, Brad Peabody, and Paul Leach. Universally Unique IDentifiers (UUIDs). RFC 9562. <https://www.rfc-editor.org/rfc/rfc9562>.
- [4] Docker Inc. Docker documentation. <https://docs.docker.com/>.
- [5] Expo. Expo application services documentation. <https://docs.expo.dev/eas/>.
- [6] Expo. Expo documentation. <https://docs.expo.dev>.
- [7] Expo. Expo location. <https://docs.expo.dev/versions/latest/sdk/location/>.
- [8] L. Gallitelli, P. Girard, U. Andriolo, M. Liro, G. Suaria, et al. Monitoring macroplastics in aquatic and terrestrial ecosystems: Expert survey reveals visual and drone-based census as most effective techniques. *Science of The Total Environment*, 955:176528, 2024. doi:10.1016/j.scitotenv.2024.176528. Disponible en <https://www.sciencedirect.com/science/article/pii/S0048969724066841>.
- [9] Daniel Gonzalez, Georg Hanke, Gijsbert Tweehuysen, Bert Bellert, Marloes Holzhauer, Andreja Palatinus, Philipp Hohenblum, and Lex Oosterbaan. Riverine litter monitoring – options and recommendations. Technical Report EUR 28307 EN, Joint Research Centre, European Commission, 2016. doi:10.2788/461233. Disponible en <https://mcc.jrc.ec.europa.eu/documents/201703034325.pdf>.
- [10] Daniel Gonzalez-Fernandez and Georg Hanke. Toward a harmonized approach for monitoring of riverine floating macro litter inputs to the marine environment. *Frontiers in Marine Science*, 4, 2017. doi:10.3389/fmars.2017.00086. Disponible en <https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2017.00086/full>.
- [11] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519. <https://www.rfc-editor.org/rfc/rfc7519>.
- [12] Camille Lacroix and Paul Vriend. Sharing best practices part 1: Monitoring techniques for macrolitter. OSPAR Riverine Litter Workshop, Temse, Belgium,

2025. Disponible en https://www.ospar.org/site/assets/files/49581/annex_2_-_slides_on_monitoring_guidelines.pdf.
- [13] Meta Open Source. React documentation. <https://react.dev>.
 - [14] Meta Open Source. React native documentation. <https://reactnative.dev>.
 - [15] National Institute of Standards and Technology. Role Based Access Control (RBAC). <https://csrc.nist.gov/projects/role-based-access-control>.
 - [16] OpenJS Foundation. Express documentation. <https://expressjs.com>.
 - [17] Prisma Data, Inc. Prisma documentation. <https://www.prisma.io/docs>.
 - [18] React Leaflet contributors. React leaflet documentation. <https://react-leaflet.js.org/docs/start-introduction/>.
 - [19] Render. Render documentation. <https://render.com/docs>.
 - [20] SQLite Consortium. Sqlite documentation. <https://www.sqlite.org/docs.html>.
 - [21] The PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>.
 - [22] Vite. Vite documentation. <https://vite.dev>.

A. Declaración del uso de IA

En cumplimiento con las buenas prácticas académicas y de transparencia, se declara que para la elaboración de este trabajo se ha contado con el apoyo de herramientas de Inteligencia Artificial Generativa. El uso de dichas herramientas se ha limitado a tareas de apoyo, contraste y revisión, manteniendo los autores la responsabilidad final sobre las decisiones técnicas, el código entregado y la redacción definitiva de la memoria.

A.1. Usos realizados

Las herramientas de IA se han utilizado principalmente en las siguientes tareas:

- **Organización inicial del trabajo.** Apoyo para estructurar la memoria, ordenar capítulos y decidir qué artefactos documentales eran necesarios.
- **Contraste de stack tecnológico.** Comparación de alternativas para el portal web, la aplicación móvil, el backend, la base de datos y el mecanismo de sincronización offline.
- **Prototipado de interfaces.** Ayuda en la generación y ajuste de mockups funcionales del portal web y de la aplicación móvil.
- **Diseño de arquitectura.** Apoyo en la elaboración de diagramas, separación de componentes, flujo de sincronización y modelo de datos.
- **Revisión de texto.** Reformulación y mejora de redacción de apartados de la memoria, siempre sobre contenido revisado por los autores.

El Cuadro A.1 resume las partes del trabajo donde se utilizó IA y con qué propósito.

Cuadro A.1: Detalle del uso de IA en el trabajo.

Parte del trabajo	Herramienta	Propósito
Estructura de memoria	Claude / Codex	Proponer organización de capítulos y anexos.
Stack tecnológico	Claude / Codex	Comparar alternativas y razonar decisiones técnicas.
Mockups funcionales	Claude / Codex	Apoyar la generación y corrección de pantallas React.
Diseño del sistema	Claude / Codex	Redactar borradores, diagramas y tablas técnicas.
Revisión de redacción	Claude / Codex	Mejorar claridad, coherencia y estilo académico.

A.2. Ejemplos de prompts utilizados

A continuación se recogen ejemplos representativos de prompts empleados durante el proyecto. No se incluyen todas las conversaciones, sino aquellos tipos de consulta que reflejan el uso realizado.

Elección del stack

Estamos diseñando una aplicación móvil de campo que debe funcionar sin conexión, capturar GPS y fotos, y sincronizar observaciones con un backend cuando recupere conectividad. Compara alternativas para la base de datos local y razona si tiene sentido usar SQLite directamente en este contexto.

Diseño de arquitectura

Necesito plantear el capítulo de diseño de la solución para una plataforma formada por portal web, app móvil offline-first y backend REST. Propone una estructura para explicar arquitectura, componentes, base de datos y sincronización.

Revisión de requisitos

Tenemos requisitos obtenidos en reuniones con el cliente y los hemos contrastado mediante mockups. Ayúdame a organizarlos en requisitos funcionales, no funcionales y de datos científicos, manteniendo trazabilidad con las reuniones.

Revisión de redacción

He redactado un apartado de la memoria y quiero que suene más claro y técnicamente correcto. Reformula el texto manteniendo el contenido y las decisiones tomadas por el equipo.

A.3. Autoría y revisión crítica

Los autores declaran conocer y poder explicar el contenido incorporado al trabajo. Las respuestas obtenidas de las herramientas de IA se trataron como material de apoyo y fueron revisadas críticamente antes de su inclusión. La selección del dominio, la validación con el cliente, la toma de decisiones, la implementación final, la ejecución de pruebas y la entrega definitiva son responsabilidad de los autores.

B. Manual de instalación y despliegue

Este anexo recoge los pasos reproducibles para poner en marcha OmniLitter en una máquina local desde cero. La descripción de los entornos de nube y de la estrategia de despliegue se encuentra en el capítulo de despliegue y operación. Los comandos son válidos para macOS y Linux; cuando Windows requiere una orden distinta, se indica de forma explícita.

B.1. Requisitos previos

Antes de empezar conviene tener instaladas las herramientas de la tabla B.1.

Cuadro B.1: Requisitos previos de instalación.

Herramienta	Versión mínima	Comprobación
Node.js	20.x LTS	<code>node --version</code>
pnpm	9.x	<code>pnpm --version</code>
Docker Desktop	Reciente	<code>docker --version</code>
Git	Cualquiera	<code>git --version</code>

En macOS y Linux, Node.js puede instalarse con `nvm` y `pnpm` de forma global. En Windows se recomienda usar Windows Terminal con PowerShell e instalar las herramientas con `winget`. Docker Desktop necesita activar WSL 2 en Windows durante su instalación. Si PowerShell bloquea la ejecución de scripts, hay que permitirla una sola vez con `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`. En todos los casos, Docker Desktop debe estar en ejecución antes de continuar.

B.2. Puesta en marcha

B.2.1. Clonar el repositorio e instalar dependencias

El siguiente bloque clona el repositorio e instala todos los paquetes del monorepo (web, móvil, API y compartido).

```

1 git clone https://github.com/Omnilitter-TFG/OmniLitter.git
2 cd OmniLitter
3 pnpm install

```

A continuación se copia la plantilla de variables de entorno. El fichero `.env` incluye valores listos para desarrollo local, por lo que no es necesario cambiar nada para arrancar la aplicación. Las credenciales de correo y otros secretos incluidos en la plantilla son valores de ejemplo y deben sustituirse en un despliegue real.

```

1 cp .env.example .env          # En Windows: Copy-Item .env.example .env

```

B.2.2. Opción A: arranque con Docker

Esta opción levanta PostgreSQL, la API y el portal web sin necesidad de instalar PostgreSQL en la máquina. La primera vez tarda unos minutos mientras construye las imágenes.

```

1 pnpm docker:up

```

El comando anterior deja los contenedores en ejecución y muestra sus logs. En otra terminal, una vez PostgreSQL esté disponible, se aplican las migraciones y se cargan los datos semilla:

```

1 pnpm db:migrate
2 pnpm db:seed

```

Una vez listos los contenedores, los servicios quedan disponibles en las direcciones de la tabla B.2. Para detenerlos se usa `pnpm docker:down`.

Cuadro B.2: Servicios disponibles con Docker.

Servicio	Dirección
Portal web	<code>http://localhost:8080</code>
API REST	<code>http://localhost:3000/api</code>
Documentación de la API (Swagger)	<code>http://localhost:3000/api/docs</code>
PostgreSQL	<code>localhost:5433</code>

B.2.3. Opción B: desarrollo sin Docker

Para trabajar con recarga en caliente se puede levantar solo la base de datos en Docker y ejecutar el resto en local.


```
1 docker compose up postgres -d
2 pnpm db:migrate
3 pnpm db:seed
4 pnpm --parallel --filter @omnilitter/api --filter @omnilitter/web dev
```

En esta modalidad el portal web se sirve con Vite en `http://localhost:5173` y la API en `http://localhost:3000/api`. Cada paquete puede arrancarse por separado con los filtros de pnpm, por ejemplo `pnpm --filter @omnilitter/api dev`. El script `pnpm dev` del monorepo también existe, pero arranca web, API y móvil a la vez.

B.3. Aplicación móvil

La aplicación móvil usa Expo en *bare workflow* con `expo-dev-client`, de modo que Expo Go no es compatible y hay que compilar el cliente nativo. La primera vez se compila e instala en el emulador o dispositivo, y en arranques posteriores basta con iniciar el servidor de desarrollo.

```
1 # Primera vez (compila e instala el cliente nativo)
2 pnpm --filter @omnilitter/mobile android # Android (Android Studio o
    dispositivo)
3 pnpm --filter @omnilitter/mobile ios # iOS (solo macOS, requiere Xcode)
4
5 # Arranques posteriores
6 pnpm --filter @omnilitter/mobile dev:client
```

La variable `EXPO_PUBLIC_API_URL` del fichero `.env` debe apuntar a la API. Si se usa un dispositivo físico, hay que sustituir `localhost` por la IP local de la máquina en la red, por ejemplo `http://192.168.1.50:3000/api`. En Windows solo es posible probar en Android, ya que el simulador de iOS no está disponible.

Generar un APK para un dispositivo físico

Para probar la aplicación en un dispositivo Android real sin el servidor de desarrollo, se genera un APK con EAS a partir del perfil `preview`. Requiere una cuenta de Expo para iniciar sesión.

```
1 npm install -g eas-cli
2 eas login # inicia sesion con la cuenta de Expo
3 eas whoami # comprueba la sesion activa
4 pnpm --filter @omnilitter/mobile build:android:apk
```

El APK resultante usa distribución interna, por lo que se instala directamente en el dispositivo sin pasar por la tienda. La API contra la que trabaja se decide al compilar: basta con definir `EXPO_PUBLIC_API_URL` con la dirección deseada (la IP local de la máquina, la API de Render en `https://omnilitter-api.onrender.com/api` o la API de producción en Azure) antes de lanzar la compilación.

B.4. Scripts útiles

```
1 pnpm lint      # Analisis estatico con ESLint en todo el monorepo
2 pnpm typecheck # Validacion de TypeScript en todos los paquetes
3 pnpm format    # Formateo del codigo con Prettier
4 pnpm build     # Compilacion de todos los paquetes
5 pnpm test      # Ejecucion de los tests
```

B.5. Solución de problemas frecuentes

Error de versión de Node al instalar. Comprobar que la versión es 20 o superior con `node --version`. Con nvm, ejecutar `nvm use 20`.

Docker no arranca los contenedores. Asegurarse de que Docker Desktop está abierto. En Windows, si aparece un error de WSL 2, ejecutar `wsl --install` y `wsl --update` en una terminal de administrador y reiniciar.

El puerto 5433 ya está en uso. OmniLitter expone PostgreSQL en el 5433 del equipo para no chocar con una instalación local en el 5432. Si el 5433 también está ocupado, detener el proceso que lo usa o cambiar el mapeo de puerto en `docker-compose.yml` y en `.env`.

La aplicación móvil no conecta con la API. Desde un dispositivo físico, `localhost` no funciona. Buscar la IP local de la máquina (con `ipconfig` en Windows o `ifconfig` en macOS y Linux) y ponerla en `EXPO_PUBLIC_API_URL`.

C. Manual de usuario